الجمهورية الجزائرية الديمقراطية الشعبية
ⵜⴰⴷⵍⴰⵡⵉⵜ ⵜⴰⵎⴰⵡⴰⵙⵜ ⵜⴰⵎⴰⵡⴰⵙⵜ ⵜⴰⵏⵎⵓⵔⵜ
République Algérienne Démocratique et Populaire
وزارة التعليم العالي والبحث العلمي
ⴰⵏⴰⵎⵓⵔ ⵏ ⵓⵙⵙⵍⵎⴷ ⵓⵏⴰⵎⵓⵔ ⴷ ⵓⵙⵎⵓⵜⵜⴳ ⵓⵙⵙⵍⵎⴷ
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

ECOLE NATIONALE SUPÉRIEURE D'INFORMATIQUE

المدرسة الوطنية العليا للإعلام الآلي
ⴰⵏⵓⵎⴰⵔ ⴰⵏⵓⵏⴰⵍ ⵓⵏⴰⵎⵓⵔ ⵏ ⵜⵓⵏⵏⵓⵏⵜⵉⵜ
École nationale Supérieure d'Informatique

Mémoire de Master

Pour l'obtention du diplôme de Master en Informatique

Option : Systèmes d'Information et Technologies (SIT)

Thème :

Approches bio-inspirées pour la conception et l'optimisation automatique des circuits quantiques

---

Réalisé par :
BOUHADOUZA Rania
GHLIB Romaissa

Encadré par :
Pr. HNAIEN Faicel (LIST3N)
Dr. DJENNANE Nabila (ESI)
Dr. CHARABI Leila (ESI)

Organisme d'accueil :

LIST3N - Laboratoire informatique et société numérique

Université de Technologie de Troyes

Promotion : 2024-2025

Dédicace

*À **Allah**, le Tout-Puissant, qui a guidé mes pas, illuminé mon chemin et m'a accordé la force de persévérer.*

*À **ma mère**, flamme bienveillante, lumière et refuge. Par son amour inconditionnel, sa force et ses sacrifices, elle a façonné la personne que je suis aujourd'hui. Elle demeure le souffle qui m'a portée et poussée à ne jamais renoncer.*

*À **mon père**, pilier de sérénité et de chaleur, dont la confiance et les prières m'ont réconfortée dans les instants de doute.*

*À **mes frères**, mes compagnons de vie, et tout particulièrement à **Raouf**, mon bras droit et allié fidèle. À mon petit prince **Abdelmounaim**, source de joie pure, ainsi qu'à ma petite sœur et à ma douce **Nour**.*

*À mes enseignants, **Merazga Abdelkarim** et **Akhenak Assia**, qui ont été bien plus que de simples formateurs : des guides généreux et inspirants.*

*À mon binôme **Romaissa**, partenaire de route dans cette aventure académique, avec qui j'ai partagé les défis et les réussites.*

*À ma cousine **Chourouk**, ainsi qu'à mes amies **Nada**, **Imane** et **Sonia**, pour leur présence, leur soutien et leurs sourires qui ont embelli ce parcours.*

Enfin, à toutes celles et ceux qui, de près ou de loin, ont cru en moi, m'ont offert leur aide précieuse et ont contribué à bâtir le chemin qui m'a menée jusqu'ici.

Rania

Dédicace

*À **Allah, le Tout-Puissant**, qui a guidé mes pas, illuminé mon chemin et m'a accordé la force de persévérer dans les moments de doute.*

*À ma chère **mère** et à mon **père**, piliers de ma vie, pour leur amour inconditionnel, leurs prières sincères et leur soutien qui ont été la source de ma motivation et de ma réussite.*

*À mon frère **Abdou**, ainsi qu'à mes sœurs **Kheira, Souaad, Aïcha, Karima, Fatiha, Chaïma et Douae**, pour leur présence réconfortante et leur encouragement constant, vous avez été une lumière dans mes moments d'effort.*

*À mes beaux-frères **Abdelhak, Youcef, Ahmed** et les autres, pour vos encouragements et votre bienveillance.*

*À mes enseignants de **l'ESI** et de **l'ESTIN**, et particulièrement à **Monsieur D. Sidali Bousla**, pour ses précieux conseils, son encouragement et sa recommandation qui m'ont beaucoup inspiré.*

*À mon promoteur de stage **Faicel Hnaien** et à toute l'équipe administrative de **LUTT**, pour leur encadrement, leur disponibilité et leur aide précieuse qui ont grandement facilité ce travail.*

À mes encadrantes de ce projet, pour leur patience, leur disponibilité et leurs conseils constructifs qui ont enrichi ce travail.

*À ma binôme **Rania**, pour son soutien, sa collaboration sincère et le partage de chaque étape de ce projet avec détermination et complicité.*

*À mes amies **Safa, Houda, Imane**, et à tous mes amis rencontrés à **l'ESTIN**, en particulier **Ouarda** et **Djihen**, pour leur amitié sincère, leur générosité et tous les précieux souvenirs partagés.*

Enfin, à toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce travail, je vous exprime ma profonde gratitude.

Romaissa

Remerciements

Le chemin qui a conduit à l'aboutissement de ce travail a été une véritable aventure, faite de croissance, de persévérance et de découvertes. Rien de tout cela n'aurait été possible sans le soutien, l'inspiration et la bienveillance de nombreuses personnes auxquelles nous adressons nos plus sincères remerciements.

Nous tenons tout d'abord à exprimer notre profonde gratitude à **Pr. Faicel Hnaien**. Son soutien constant, ses conseils éclairés et sa confiance indéfectible ont guidé notre progression et façonné notre parcours de chercheurs.

Nous souhaitons également adresser nos remerciements les plus chaleureux à **Mme. Djennane Nabila** et **Mme. Charabi Leila**, qui ont été bien plus que de simples encadrantes : de véritables mentors. Leur encouragement, leurs analyses pertinentes et leur leadership inspirant ont marqué notre parcours de manière durable.

Nous exprimons aussi notre profonde reconnaissance aux **membres du jury**, pour le temps consacré à l'évaluation de ce travail et pour l'attention portée à nos recherches.

À nos familles, qui ont toujours été notre refuge et notre force, nous disons merci du fond du cœur. Nous souhaitons exprimer notre sincère gratitude à **nos parents**, véritables piliers de cette aventure. Leur présence constante, leurs encouragements sans faille, leur confiance en nous et, surtout, leur *amour inconditionnel* ont été notre force au quotidien. Ils ont cru en nous, parfois davantage que nous-mêmes, et nous ont soutenus sans relâche. Sans eux, rien de tout cela n'aurait été possible.

À nos sœurs et à nos frères, dont les encouragements et la confiance discrète nous ont accompagnés tout au long de ce chemin, nous adressons également toute notre gratitude.

Nous adressons nos sincères remerciements à l'ensemble du **corps enseignant de l'École Nationale Supérieure d'Informatique (ESI, Alger)** pour leur engagement constant et la qualité de leur enseignement. Leur accompagnement a contribué à façonner non seulement notre parcours académique, mais aussi les valeurs et l'éthique professionnelle que nous portons aujourd'hui.

Enfin, à toutes celles et ceux qui, de près ou de loin, par des gestes visibles ou invisibles, ont contribué à la réalisation de ce projet, nous disons un immense MERCI.

Résumé

L'informatique quantique constitue une révolution computationnelle, capable de résoudre certaines classes de problèmes qui échappent aux ordinateurs classiques. Néanmoins, la conception de circuits quantiques efficaces reste un défi majeur, notamment à l'ère des dispositifs quantiques actuels, limités par le bruit, la décohérence et la faible connectivité des qubits. Face à ces contraintes, la conception automatique de circuits apparaît comme une alternative incontournable à la conception manuelle, devenue rapidement impraticable avec l'accroissement de la complexité des systèmes. Ce mémoire étudie les métaheuristiques bio-inspirées pour la synthèse et l'optimisation automatique des circuits quantiques, en mettant l'accent sur les algorithmes évolutionnaires (algorithmes génétiques, NSGA-II) et l'intelligence en essaim (optimisation par essaim de particules). Une analyse comparative des approches existantes est réalisée, en discutant leurs fondements théoriques, leurs applications et leurs performances dans un cadre multi-objectifs. L'étude met en évidence les compromis entre fidélité, profondeur et coût structurel des circuits, tout en identifiant les stratégies les plus prometteuses pour l'optimisation adaptée aux contraintes des processeurs quantiques de nouvelle génération.

Mots-clés : Informatique quantique, Dispositifs quantiques actuels, Optimisation de circuits quantiques, Algorithmes évolutionnaires, Intelligence en essaim, Métaheuristiques bio-inspirées.

Abstract

Quantum computing represents a major paradigm shift in computation, offering the potential to tackle problems that remain intractable for classical machines. However, the design of efficient quantum circuits remains a central challenge, particularly in the current era, where limitations such as decoherence, restricted connectivity, and gate errors significantly impact performance. To address these issues, automatic circuit synthesis has emerged as a promising approach, reducing the impracticality of manual design as system size and complexity increase. This dissertation explores bio-inspired metaheuristics, with a particular focus on evolutionary algorithms (Genetic Algorithms, NSGA-II) and Particle Swarm Optimization, for the automatic generation and optimization of quantum circuits. A comparative study of existing methods is presented, highlighting their theoretical foundations, applications, and performance in a multi-objective context. The work emphasizes the trade-offs between fidelity, circuit depth, and structural cost, and identifies promising strategies for adapting these methods to the constraints of near-term quantum devices.

Keywords: Quantum computing, near-term quantum devices, Quantum circuit optimization, Evolutionary algorithms, Swarm intelligence, Bio-inspired metaheuristics.

ملخص

تُعَدُّ الحوسبة الكمية ثورةً في مجال الحوسبة، قادرةً على حلِّ بعض الفئات من المسائل التي تَعجِز عنها الحواسيب الكلاسيكية. ومع ذلك، تظلّ عملية تصميم دوائر كمية فعّالة تحدياً رئيسياً، خاصةً في ظل الأجهزة الكمية الحالية التي تعاني من الضوضاء، وفقدان التماسك، وضعف الترابط بين الكيوبتات. في مواجهة هذه القيود، يُعَدّ التصميم التلقائي للدوائر بديلاً لا غنى عنه عن التصميم اليدوي، الذي أصبح سريعاً غير عملي مع ازدياد تعقيد الأنظمة.

تتناول هذه الأطروحة دراسة الميتاهيرستيكيات المستوحاة من الطبيعة الحيوية لتوليد وتحسين الدوائر الكمية تلقائياً، مع التركيز على الخوارزميات التطورية (الخوارزميات الجينية) وذكاء السرب (التحسين بواسطة سرب الجسيمات). كما يتم تقديم تحليل مقارن للمناهج الحالية، مع مناقشة أسسها النظرية، وتطبيقاتها، وأدائها في إطار متعدد الأهداف. وتُبرز الدراسة المقايضات بين الموثوقية، والعمق، والكلفة البِنيوية للدوائر، مع تحديد الاستراتيجيات الأكثر وُعوداً من أجل تحسين يتوافق مع قيود المعالجات الكمية من الجيل الجديد.

الكلمات المفتاحية: الحوسبة الكمية، الأجهزة الكمية الحالية، تحسين الدوائر الكمية، الخوارزميات الجينية، ذكاء السرب، الميتاهيرستيكيات الحيوية المستوحاة.

Table des matières

Table des figures **iv**

Liste des tableaux **v**

Liste des acronymes **vi**

Introduction générale **1**

1 De l'informatique classique à l'informatique quantique **3**
Introduction . . . 3
1.1 L'informatique classique : forces et limites . . . 3
1.2 L'émergence de l'informatique quantique . . . 4
1.3 Différences fondamentales entre informatique classique et quantique . . . 5
1.4 Pourquoi la conception automatique devient nécessaire . . . 5
Conclusion . . . 6

2 L'informatique quantique **7**
Introduction . . . 7
2.1 Genèse et évolution de l'informatique quantique . . . 7
2.1.1 Informatique et mécanique quantiques . . . 7
2.1.2 Le calcul quantique et ses origines . . . 7
2.1.3 Évolutions expérimentales récentes . . . 8
2.2 Postulats et notions mathématiques . . . 8
2.2.1 Notion de qubit . . . 8
2.2.2 Représentations d'un qubit . . . 9
2.2.3 Systèmes multi-qubits . . . 10
2.3 Modèles de calcul quantique . . . 11
2.3.1 Opérations sur les états quantiques . . . 11
2.3.2 Portes logiques quantiques . . . 11
2.3.3 Registre quantique . . . 13
2.3.4 Circuits quantiques . . . 13
2.3.5 Propriétés fondamentales du calcul quantique . . . 15
2.4 Algorithmes fondamentaux . . . 16
2.4.1 Algorithme de Deutsch . . . 16
2.4.2 Algorithme de Grover . . . 16
2.4.3 Algorithme de Shor . . . 16
2.4.4 Algorithmes Noisy Intermediate-Scale Quantum (NISQ) : . . . 17
2.5 Les ordinateurs quantiques . . . 17
2.5.1 Les simulateurs quantiques . . . 17
2.5.2 Les ordinateurs quantiques à portes logiques . . . 18
2.5.3 Simulation classique des circuits quantiques . . . 18
2.6 Correction d'erreurs et tolérance aux fautes . . . 19

Conclusion 20

3 Généralités sur les métaheuristiques bio-inspirées 21
Introduction 21
3.1 Classification des métaheuristiques bio-inspirées 21
3.1.1 Les algorithmes évolutionnaires : 22
3.1.2 Les algorithmes d'intelligence en essaim : 29
Conclusion 33

4 Optimisation de circuits quantiques par les métaheuristiques bio-inspirés 34
Introduction 34
4.1 Critères de comparaison 34
4.2 Les algorithmes évolutionnaires pour la conception de circuits quantiques . 35
4.2.1 Rôle des algorithmes évolutionnaires dans la conception des circuits quantiques 35
4.2.2 Origines de l'application des algorithmes génétiques en circuits quantiques 35
4.2.3 Typologie des travaux réalisés 36
4.2.4 Analyse comparative des méthodes étudiées 48
4.3 Les algorithmes d'intelligence en essaim 50
4.3.1 Apport des algorithmes d'intelligence en essaim à la conception de circuits quantiques 50
4.3.2 Origines de l'application des algorithmes d'intelligence en essaim (SI) à la synthèse de circuits quantiques 51
4.3.3 Typologie des travaux réalisés 52
4.3.4 Analyse comparative des méthodes étudiées 56
4.4 Synthèse comparative 58
Conclusion 60

Conclusion et perspectives 61

Table des figures

2.1 Visualisation d'un qubit sur la sphère de Bloch 10
2.2 Les opérateurs quantiques 13
2.3 Circuit quantique préparant un état de Bell. 14
2.4 Représentation de la mesure dans un circuit quantique 14
2.5 Circuit quantique de l'algorithme de Deutsch 16

3.1 Classification des métaheuristiques bio-inspirées 21
3.2 Les opérateurs utilisés dans l'AG 26
3.3 Principe général de l'AG 27
3.4 Les étapes de déroulement de Ant Colony Optimization (ACO) 31
3.5 Les étapes de déroulement de Particle Swarm Optimization (PSO) 33

4.1 Schéma de déroulement de l'algorithme génétique basé sur le modèle en îles 40
4.2 Le principe de fonctionnement de (Quantum Global Optimizer (QGo)). . . 45
4.3 Un exemple d'optimisation de circuit quantique avec (QGo). 46
4.4 Optimisation des circuits quantiques dans le cadre Genetic Algorithm for Quantum Circuit Optimization (GA4QCO). 47
4.5 Schéma de flux du PSO pour la synthèse de circuits quantiques 53
4.6 Schéma de flux du PSO pour la synthèse de circuits quantiques 55

Liste des tableaux

1.1 Comparaison entre informatique classique et informatique quantique 5

2.1 Quelques simulateurs de circuits quantiques 19

3.1 Algorithmes SI importants 29

4.2 Comparaison des approches multi-objectifs pour la synthèse de circuits quantiques 48

4.1 Comparaison des méthodes évolutionnaires pour la synthèse de circuits quantiques 49

4.3 Approches PSO « seules » pour la conception automatique de circuits (quantiques et réversibles). 57

4.4 PSO *vs* algorithmes évolutionnaires (Algorithme Génétique (AG)/Nondominated Sorting Genetic Algorithm II (NSGA-II)) pour la conception/optimisation de circuits quantiques. 59

Liste des acronymes

A*	A-star Search Algorithm	39
ACO	Ant Colony Optimization	iv
AE	Algorithme Évolutionnaire	22
AG	Algorithme Génétique	v
AGQ	Algorithme génétique quantique	49
BFGS	Broyden–Fletcher–Goldfarb–Shanno algorithm	39
CNOT	Controlled-NOT	12
COBYLA	Constrained Optimization BY Linear Approximations	39
CSS	Calderbank-Shor-Steane	38
CU	Controlled-Unitary	12
DPSO	Discrete Particle Swarm Optimization	54
DRL	Deep Reinforcement Learning	54
EQAS-PQC	Evolutionary Quantum Architecture Search for Parameterized Quantum Circuits	43
GASP	Genetic Algorithm for State Preparation	40
GA4QCO	Genetic Algorithm for Quantum Circuit Optimization	iv
GPU	Graphics Processing Unit	39
GWO	Grey Wolf Optimization	50
IBM	International Business Machines	4
iSOMA	Improved Self-Organizing Migrating Algorithm	50
LRSP	Low-Rank State Preparation	42
MEAS-PQC	Multi-Objective Evolutionary Architecture Search for Parameterized Quantum Circuits	43
NISQ	Noisy Intermediate-Scale Quantum	ii
NP	Nondeterministic Polynomial time	3
NSGA-II	Nondominated Sorting Genetic Algorithm II	v
PSO	Particle Swarm Optimization	iv
PQC	Parameterized Quantum Circuit	43
QAOA	Quantum Approximate Optimization Algorithm	17
QECC	Quantum Error Correcting Codes	19
QEQEA	Quantum Encoded Quantum Evolutionary Algorithm	38
QFT	Quantum Fourier Transform	17
QGo	Quantum Global Optimizer	iv
QUBO	Quadratic Unconstrained Binary Optimization	8
RL	Reinforcement Learning	43
RMN	Résonance Magnétique Nucléaire	8
RSA	Rivest–Shamir–Adleman	4
RS	Random Search	38

SDK Software Development Kit 17
SI Swarm Intelligence 29
SLSQP Sequential Least Squares Programming 41
SOFTMAX-PQC Softmax-Based Architecture Search for Parameterized Quantum Circuits 43
UMAD Uniform Mutation by Addition and Deletion 43
VQE Variational Quantum Eigensolver 17

Introduction générale

L'informatique quantique constitue l'une des révolutions majeures du calcul au XXIe siècle. En exploitant des phénomènes fondamentaux comme la superposition et l'intrication, elle offre un potentiel considérable pour résoudre des problèmes qui demeurent inaccessibles aux ordinateurs classiques (Feynman, 1985 ; Nielsen & Chuang, 2010). Cependant, les dispositifs actuels, dits NISQ, restent fortement limités par le bruit, la décohérence et la connectivité réduite des qubits (Preskill, 2018). Dans ce contexte, l'optimisation des circuits quantiques représente un enjeu central pour exploiter au mieux ces ressources. La conception manuelle de circuits quantiques devient rapidement impraticable à mesure que la taille et la complexité des systèmes augmentent. Les contraintes matérielles (connectivité, profondeur, portes coûteuses) rendent cette tâche encore plus difficile (Maslov, 2016 ; Murali et al., 2019).
La problématique qui se pose est donc la suivante : *comment générer automatiquement des circuits quantiques efficaces, optimisés à la fois en termes de fidélité, de profondeur et de coût structurel, afin de les rendre compatibles avec les processeurs NISQ actuels ?* L'objectif principal de ce mémoire est d'étudier et de comparer des **métaheuristiques bio-inspirées**, notamment les **algorithmes évolutionnaires** (AG, NSGA-II) et l'**intelligence en essaim** PSO, pour la conception automatique de circuits quantiques. Plus précisément, ce travail vise à :

— analyser les fondements théoriques de l'informatique quantique et ses contraintes matérielles,

— étudier les principes et variantes des algorithmes évolutionnaires et d'intelligence en essaim,

— appliquer et comparer ces approches dans un cadre multi-objectifs,

— identifier les stratégies les plus prometteuses pour l'optimisation des circuits quantiques.

Pour répondre à ces objectifs, la méthodologie adoptée repose sur une approche en plusieurs étapes :

1. une étude bibliographique approfondie sur les concepts fondamentaux du calcul quantique et les limites des systèmes NISQ,
2. une analyse des métaheuristiques bio-inspirées, en particulier les algorithmes évolutionnaires et d'intelligence en essaim,
3. une comparaison des approches existantes dans la littérature,
4. une discussion critique sur leurs performances et perspectives d'amélioration.

Le présent mémoire est organisé comme suit :

— Le **premier chapitre** introduit le contexte de l'informatique quantique et ses différences fondamentales avec l'informatique classique.

— Le **deuxième chapitre** expose les concepts et définitions de base du calcul quantique, en insistant sur les qubits, portes et circuits.

— Le **troisième chapitre** présente les fondements des métaheuristiques bio-inspirées, en particulier les algorithmes évolutionnaires et d'intelligence en essaim.

— Le **quatrième chapitre** analyse l'application de ces méthodes à la conception automatique de circuits quantiques et propose une comparaison détaillée des résultats obtenus.

— Enfin, la **conclusion générale** récapitule les contributions de ce travail et ouvre sur des perspectives futures.

Ainsi, ce travail s'inscrit dans la dynamique actuelle visant à rapprocher la puissance du calcul quantique et la flexibilité des approches évolutionnaires, afin de contribuer à l'émergence de circuits quantiques plus efficaces et adaptés aux contraintes réelles.

Chapitre 1

De l'informatique classique à l'informatique quantique

Introduction

L'informatique classique, fondée sur l'utilisation de bits et de transistors, a permis des avancées spectaculaires au cours des dernières décennies. Grâce à l'augmentation régulière de la puissance des processeurs, au développement du cloud computing et à l'adoption croissante du parallélisme, elle a su répondre à une grande variété de besoins computationnels.

Cependant, cette progression atteint aujourd'hui ses limites. La miniaturisation des composants se heurte aux contraintes physiques fondamentales — comme la dissipation thermique et les effets quantiques non désirés — tandis que la complexité intrinsèque de certains problèmes, notamment ceux dits *non-polynomiaux* complets (NP-complets) ou liés à la simulation de systèmes quantiques, dépasse les capacités des architectures classiques.

Face à ces défis, l'informatique quantique émerge comme un nouveau paradigme prometteur. En s'appuyant sur les principes de la mécanique quantique — tels que la superposition, l'intrication et l'interférence — elle propose une nouvelle manière de représenter, manipuler et traiter l'information.

Ce chapitre introduit les fondements de cette transition, en présentant d'abord les limites de l'informatique classique, puis l'émergence de l'informatique quantique, leurs différences fondamentales, et enfin la nécessité croissante de la conception automatique des circuits quantiques.

1.1 L'informatique classique : forces et limites

L'informatique classique repose sur une architecture matérielle de type Von Neumann (von Neumann, 1945), dans laquelle les données et les instructions sont stockées en mémoire et traitées séquentiellement. L'information est représentée sous forme binaire, à l'aide d'unités appelées *bits*, chacun pouvant prendre la valeur 0 ou 1. Le traitement est déterministe : pour une entrée donnée, le résultat est entièrement prévisible.

Au fil des décennies, la puissance de calcul a augmenté de manière exponentielle grâce à la miniaturisation des transistors, comme le prévoit la *loi de Moore* (Moore, 1975). Cette tendance a permis le développement de systèmes de plus en plus performants, exploitant le parallélisme, *l'informatique en nuage* (*cloud computing*), et des architectures multi-cœurs.

Cependant, ce modèle atteint aujourd'hui plusieurs types de limites, regroupées en trois grandes catégories :

- **Limites physiques :** à l'échelle nanométrique, la miniaturisation atteint des contraintes fondamentales. D'une part, l'*effet tunnel quantique* permet aux électrons de traverser les barrières d'isolation, ce qui perturbe le fonctionnement logique des transistors (Zhirnov et al., 2003). D'autre part, la *dissipation thermique* devient difficile à contrôler, car la densité de puissance sur les puces entraîne une surchauffe limitant la fréquence des processeurs (Pop, 2010). Enfin, la *variabilité de fabrication* accroît les défauts à ces échelles extrêmes, compromettant la fiabilité (Iwai, 2009).
- **Limites algorithmiques :** certains problèmes restent intrinsèquement complexes pour les ordinateurs classiques. Parmi eux :
  - la **factorisation de grands entiers**, au cœur du chiffrement Rivest–Shamir–Adleman (RSA), nécessitant un temps exponentiel sans solution efficace connue,
  - la **recherche non structurée**, qui nécessite $O(N)$ essais,
  - et la **simulation de systèmes physiques quantiques**, qui exige la manipulation de 2^n amplitudes complexes pour n particules (Arora & Barak, 2009 ; Richard & Feynman, 1982).
- **Limites énergétiques :** les transistors miniaturisés génèrent des courants de fuite même à l'état de repos, entraînant une consommation énergétique accrue. Les centres de données sont devenus très énergivores, avec un impact environnemental croissant (Andrae & Edler, 2015 ; "IEEE Rebooting Computing Initiative", 2019 ; Koomey, 2011).

En résumé, bien que l'informatique classique reste extrêmement performante dans de nombreux domaines, elle atteint aujourd'hui ses limites face à la croissance des besoins de calcul intensif, à la complexité des problèmes rencontrés, et aux contraintes physiques irréductibles. Ces constats motivent l'exploration d'un nouveau paradigme computationnel : l'informatique quantique, qui sera introduite dans la section suivante.

1.2 L'émergence de l'informatique quantique

L'informatique quantique trouve son origine dans une réflexion de Richard Feynman (Richard & Feynman, 1982), qui, dès 1982, soulignait l'impossibilité pratique de simuler des systèmes physiques quantiques à l'aide d'ordinateurs classiques. Ce constat a suscité l'idée d'un nouveau paradigme computationnel, capable de s'appuyer directement sur les lois de la mécanique quantique pour traiter l'information.

L'intérêt scientifique pour cette approche s'est consolidé au fil du temps, notamment dans des domaines comme la chimie quantique, la modélisation de matériaux ou encore l'optimisation combinatoire. Ces problématiques posent en effet des défis considérables aux ordinateurs classiques, tant en termes de complexité qu'en termes de ressources.

Parallèlement, des avancées théoriques majeures ont démontré que certains problèmes spécifiques, tels que la factorisation de grands entiers ou la recherche dans des bases non structurées, pourraient être résolus plus efficacement sur un ordinateur quantique. Des algorithmes comme ceux de Shor ou Grover illustrent ce potentiel.

À partir des années 2010, cet intérêt est relayé par le monde industriel. Face au ralentissement des progrès issus de la miniaturisation des composants électroniques (Theis & Wong, 2017), des acteurs majeurs comme International Business Machines (IBM), Google ou Microsoft, ainsi que des start-ups innovantes (Rigetti, IonQ, Pasqal,D-Wave), investissent dans la mise au point de processeurs quantiques.

Ces dispositifs expérimentaux, encore immatures, reposent sur des principes fondamentalement différents de ceux de l'informatique classique, tels que la superposition des états, l'intrication entre qubits et l'interférence quantique. Ils ouvrent la voie à de nouvelles stratégies de calcul, plus puissantes pour certaines classes de problèmes.

Les concepts physiques, les modèles computationnels et les implications concrètes de cette approche feront l'objet d'une présentation plus détaillée dans le chapitre suivant.

1.3 Différences fondamentales entre informatique classique et quantique

L'informatique quantique repose sur des principes radicalement différents de ceux de l'informatique classique. Dans le **Tableau 1.1**, nous avons synthétisé et résumé les principales distinctions entre ces deux paradigmes afin de mettre en évidence leurs caractéristiques essentielles (Nielsen & Chuang, 2010 ; Tanenbaum & Austin, 2013).

Caractéristique	**Informatique classique**	**Informatique quantique**
Unité de base	Bit : prend la valeur 0 ou 1	Qubit : peut représenter plusieurs états en même temps 0 et 1
Architecture	Traitement séquentiel ou parallèle classique	Exploite des phénomènes physiques spécifiques pour un parallélisme massif
Traitement de l'information	Déterministe	Basé sur des probabilités, le résultat dépend de mesures répétées
Accélération sur certains problèmes	Limité (polynomial)	Très rapide pour certains problèmes spécifiques (ex. : factorisation, recherche)
Maturité technologique	Très élevée, utilisée à grande échelle	En développement, encore expérimentale

Table 1.1 – Comparaison entre informatique classique et informatique quantique

1.4 Pourquoi la conception automatique devient nécessaire

La conception manuelle de circuits quantiques devient rapidement impraticable à mesure que la taille des circuits et le nombre de qubits augmentent. Plusieurs facteurs rendent cette tâche particulièrement complexe :

— **Ordonnancement précis des portes** : l'ordre dans lequel les portes quantiques sont appliquées peut avoir un impact majeur sur la précision du calcul et sur la validité du résultat global. En effet, une permutation incorrecte peut détruire les interférences ou briser les intrications nécessaires au bon fonctionnement de l'algorithme (Nielsen & Chuang, 2000). De telles erreurs d'ordonnancement se traduisent directement par une dégradation de la performance du circuit, réduisant la qualité des résultats obtenus ou rendant l'algorithme incorrect.

— **Réduction de la profondeur du circuit** : sur les processeurs quantiques actuels, dits *NISQ* (Preskill, 2018), chaque opération est sujette au bruit. Il est donc crucial de minimiser la *profondeur* (le nombre d'étapes séquentielles) pour limiter la perte de fidélité globale.

— **Contraintes matérielles spécifiques** : les architectures physiques imposent souvent des contraintes de connectivité entre les qubits (topologie de type grille, ligne, anneau, etc.), obligeant à insérer des portes supplémentaires (comme `SWAP`) pour permettre certaines interactions (Linke et al., 2017). De plus, certaines portes sont plus coûteuses ou plus bruyantes que d'autres selon la technologie employée.

Face à ces défis, des approches de **conception automatique de circuits quantiques** se sont développées, reposant sur des techniques d'optimisation. Parmi elles, les **algorithmes évolutionnaires** — en particulier les *algorithmes génétiques* AG, les *algorithmes à population élitiste* comme NSGA-II (Deb et al., 2002). Ces méthodes permettent :

— d'explorer un espace de solutions très vaste,

— de trouver des circuits à la fois compacts et compatibles avec les contraintes matérielles,

— d'optimiser plusieurs objectifs simultanément (ex. : fidélité, profondeur, coût en portes).

L'utilisation de telles métaheuristiques est donc devenue une nécessité dans un contexte où l'exactitude, la compacité et la faisabilité matérielle sont toutes critiques pour exécuter efficacement les algorithmes quantiques sur du matériel NISQ.

Conclusion

Ce chapitre a mis en évidence les limites actuelles de l'informatique classique, tant sur le plan physique qu'algorithmique et énergétique. L'informatique quantique apparaît comme une réponse prometteuse à ces défis, en s'appuyant sur des principes fondamentalement différents. Toutefois, la conception de circuits quantiques efficaces constitue un enjeu majeur, nécessitant des méthodes d'optimisation automatiques comme les algorithmes évolutionnaires.Les fondements théoriques de l'informatique quantique, ainsi que la structure des circuits et des opérations de base, seront détaillés dans le chapitre suivant.

Chapitre 2

L'informatique quantique

Introduction

Dans ce chapitre, nous définissons les concepts clés de l'informatique quantique, notamment les qubits, les portes logiques et les circuits quantiques. Nous introduisons les bases théoriques liées à l'algèbre linéaire, essentielles pour comprendre le calcul quantique.

2.1 Genèse et évolution de l'informatique quantique

2.1.1 Informatique et mécanique quantiques

La mécanique quantique, développée entre 1900 et 1925 par Planck, Einstein, Schrödinger et d'autres, bouleverse profondément notre intuition classique. À l'échelle microscopique, un système n'occupe pas un état unique, mais peut exister dans une *superposition cohérente* de plusieurs états simultanément (Chuang, 2002 ; Gambs, 2008). Ainsi, au lieu d'être localisé uniquement en p_1 ou p_2, un objet quantique est représenté par la combinaison linéaire $|p_1\rangle + |p_2\rangle$.

L'informatique quantique exploite ces propriétés fondamentales — superposition, intrication, interférence — pour traiter et stocker l'information. Contrairement au bit classique, limité à 0 ou 1, un *qubit* est décrit par l'état $\alpha|0\rangle + \beta|1\rangle$, avec $|\alpha|^2 + |\beta|^2 = 1$, ce qui permet d'encoder simultanément 2^n configurations différentes à l'aide de n qubits (Nielsen & Chuang, 2000). Cette capacité donne naissance au *parallélisme quantique*, permettant d'explorer de nombreux résultats en une seule opération.

2.1.2 Le calcul quantique et ses origines

L'idée d'utiliser des systèmes quantiques pour effectuer des calculs plus puissants que ceux des machines classiques apparaît en 1982, lorsque Richard Feynman propose d'utiliser des dispositifs quantiques pour simuler des phénomènes physiques complexes que les ordinateurs classiques sont incapables de modéliser efficacement (Richard & Feynman, 1982).

Dans le même élan, Benioff (1980) pose les bases du *calcul réversible* dans un cadre quantique (Benioff, 1980), et Deutsch (1985) formalise la notion de *qubit* ainsi qu'un ensemble universel de portes logiques quantiques. Ces travaux ouvrent la voie au développement d'une véritable théorie du calcul quantique.

Le potentiel de l'informatique quantique s'affirme véritablement à partir de 1994 avec l'algorithme de Shor pour la factorisation des entiers, qui montre que certains problèmes autrefois considérés comme inabordables peuvent être résolus en temps polynomial sur un

ordinateur quantique (Shor, 1994). Cet algorithme menace directement la sécurité des systèmes cryptographiques classiques comme RSA.

Deux ans plus tard, Grover (1996) propose un algorithme de recherche quantique, capable de retrouver une entrée dans une base de données non structurée en $O(\sqrt{N})$ opérations, contre $O(N)$ pour la recherche classique (Grover, 1996). Cette accélération quadratique illustre la puissance combinée de la superposition et de l'interférence. En parallèle, la croissance exponentielle des performances des ordinateurs classiques, formulée dans la *loi de Moore* (1975) (Moore, 1975), atteint progressivement ses limites physiques à partir des années 2010, notamment à cause de la miniaturisation extrême des composants électroniques (Theis & Wong, 2017). Face à cette impasse annoncée, l'informatique quantique apparaît comme un nouveau paradigme capable de poursuivre l'amélioration exponentielle des capacités de calcul.

2.1.3 Évolutions expérimentales récentes

Les premières expériences de manipulation d'états quantiques datent de 1998 avec l'utilisation de la Résonance Magnétique Nucléaire (RMN) pour implémenter une porte logique élémentaire (Gershenfeld & Chuang, 1997). En 2001, la première démonstration d'un qubit supraconducteur contrôlé est réalisée à l'Université Yale (Nakamura et al., 1999). Puis, en 2005, la mise en œuvre d'une porte logique entre ions piégés marque une étape décisive dans la fiabilité du contrôle quantique (Nielsen, 2003).

Avec l'émergence de l'ère dite NISQ , IBM propose en 2016 le premier accès public à un processeur quantique de 5 qubits via le cloud (Research, 2016). Depuis, les avancées se sont accélérées : IBM dévoile en 2021 un processeur de 127 qubits baptisé Eagle (Research, 2021), tandis que Rigetti annonce un processeur de 128 qubits pour 2022 (Computing, 2022).

D'autres entreprises, telles que Google, Quantinuum et IonQ, se positionnent également sur le marché, tandis que D-Wave Systems développe des machines basées sur le recuit quantique, spécialisées dans la résolution de problèmes d'optimisation combinatoire, notamment des problèmes de type Quadratic Unconstrained Binary Optimization (QUBO) (Systems, 2019). Aujourd'hui, l'informatique quantique vise à simuler des systèmes physiques complexes, optimiser des problèmes industriels, et renforcer la sécurité informatique grâce aux propriétés intrinsèques du monde quantique. Bien que les dispositifs actuels soient encore bruyants et limités, les perspectives ouvertes par ce nouveau paradigme laissent entrevoir des applications majeures dans la cryptographie, la chimie moléculaire, l'intelligence artificielle, et la logistique.

2.2 Postulats et notions mathématiques

2.2.1 Notion de qubit

Le *qubit*, ou bit quantique, constitue l'unité élémentaire d'information dans un système quantique, tout comme le bit en est l'équivalent dans les systèmes classiques (Jaffali, 2020). Un qubit peut exister dans une superposition de ses deux états de base. Cette propriété fondamentale découle directement des principes de la mécanique quantique.

Dans la base canonique, les deux états de référence (ou *états de base*) sont notés $|0\rangle$ et

$|1\rangle$ selon la **notation de Dirac**, également appelée *ket*[1]. Ils sont définis comme suit :

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}. \tag{2.2.1}$$

Un état général d'un qubit est alors représenté comme une combinaison linéaire complexe de ces deux états (Nielsen & Chuang, 2000) :

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle, \qquad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1, \tag{2.2.2}$$

où $|\alpha|$ et $|\beta|$ sont les *modules* des nombres complexes α et β respectivement[2].

Cette contrainte de normalisation garantit que la somme des probabilités de mesure soit toujours égale à 1. Lors d'une mesure dans la base $\{|0\rangle, |1\rangle\}$, l'état $|\psi\rangle$ s'effondre aléatoirement sur l'un des deux états de base avec des probabilités données par $|\alpha|^2$ et $|\beta|^2$.

2.2.2 Représentations d'un qubit

2.2.2.1 Représentation dans l'espace de Hilbert

Le qubit appartient à l'espace vectoriel complexe $\mathcal{H}_1 = \mathbb{C}^2$. On démontre mathématiquement qu'un état de qubit $|\psi\rangle$ peut être exprimé à l'aide de deux paramètres réels, appelés **angles d'Euler** (θ, φ), selon la relation suivante (Yepez, 2013) :

$$|\psi\rangle = \cos\frac{\theta}{2}\,|0\rangle + e^{i\varphi}\sin\frac{\theta}{2}\,|1\rangle, \qquad 0 \le \theta \le \pi,\ 0 \le \varphi < 2\pi. \tag{2.2.3}$$

Pour un système de n qubits, l'espace de Hilbert associé possède la dimension 2^n et tout état se décompose dans la base computationnelle (Gambs, 2008) :

$$|\Psi\rangle = \sum_{i=0}^{2^n-1} a_i\,|i\rangle, \qquad a_i \in \mathbb{C}, \ \sum_i |a_i|^2 = 1. \tag{2.2.4}$$

où chaque coefficient $a_i \in \mathbb{C}$ est une amplitude complexe. L'état $|i\rangle$ représente ici l'encodage binaire de l'entier i sur n bits (par exemple $|3\rangle = |011\rangle$ pour $n = 3$).

2.2.2.2 Sphère de Bloch :

La sphère de Bloch de la figure 2.1 fournit une représentation tridimensionnelle intuitive d'un qubit : les états $|0\rangle$ et $|1\rangle$ occupent les pôles, tandis qu'un état arbitraire $|\psi\rangle$ se situe au point de coordonnées (θ, φ).
Cette représentation facilite la compréhension des opérations unitaires appliquées aux qubits.

1. La notation de Dirac (ou notation bra-ket) est un formalisme utilisé en mécanique quantique pour représenter les vecteurs d'état : un *ket* $|\psi\rangle$ désigne un vecteur colonne (état quantique), tandis qu'un *bra* $\langle\psi|$ désigne son conjugué hermitien (vecteur ligne).

2. Le module d'un nombre complexe $z = x + iy$ est défini par $|z| = \sqrt{x^2 + y^2}$, et représente sa distance à l'origine dans le plan complexe.

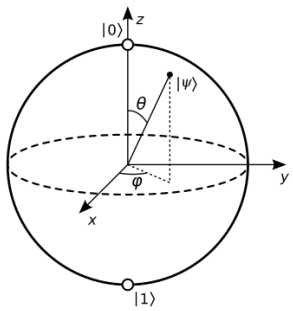


FIGURE 2.1 – Visualisation d'un qubit sur la sphère de Bloch (NIELSEN & CHUANG, 2000)

2.2.3 Systèmes multi-qubits

Lorsque plusieurs qubits sont combinés dans un même système, l'espace d'état résultant croît de manière exponentielle. Un système de deux qubits, par exemple, est représenté dans l'espace de Hilbert $\mathcal{H}_2 = \mathbb{C}^2 \otimes \mathbb{C}^2 = \mathbb{C}^4$, ce qui donne lieu à une base computationnelle composée de quatre états orthonormés :

$$\{|00\rangle,\ |01\rangle,\ |10\rangle,\ |11\rangle\}.$$

la représentation vectorielle de ces ces états :

$$|00\rangle = \begin{pmatrix}1\\0\\0\\0\end{pmatrix}, \quad |01\rangle = \begin{pmatrix}0\\1\\0\\0\end{pmatrix}, \quad |10\rangle = \begin{pmatrix}0\\0\\1\\0\end{pmatrix}, \quad |11\rangle = \begin{pmatrix}0\\0\\0\\1\end{pmatrix}.$$

Un état général de ce système peut alors être exprimé comme une superposition linéaire de ces quatre états :

$$|\psi_{AB}\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle, \qquad \sum_{i,j} |\alpha_{ij}|^2 = 1, \tag{2.2.5}$$

où chaque $\alpha_{ij} \in \mathbb{C}$ représente l'amplitude de probabilité associée à l'état $|ij\rangle$ (JAFFALI, 2020 ; NIELSEN & CHUANG, 2000).

Ce formalisme s'étend naturellement à n qubits, dont l'état quantique est décrit comme une superposition de 2^n états de base. Par exemple, pour un registre de n qubits, l'état général s'écrit :

$$|\psi_N\rangle = \sum_{k=0}^{2^n-1} \alpha_k\, |k\rangle, \tag{2.2.6}$$

où $|k\rangle$ désigne les états binaires possibles, et α_k les amplitudes complexes normalisées.

Le comportement d'un système multi-qubits repose aussi sur le produit tensoriel entre états individuels. Si les qubits A et B sont respectivement dans les états $|\psi_A\rangle = \alpha_A|0\rangle + \beta_A|1\rangle$ et $|\psi_B\rangle = \alpha_B|0\rangle + \beta_B|1\rangle$, alors l'état global du système est donné par :

$$|\psi_{AB}\rangle = |\psi_A\rangle \otimes |\psi_B\rangle = \alpha_A\alpha_B|00\rangle + \alpha_A\beta_B|01\rangle + \beta_A\alpha_B|10\rangle + \beta_A\beta_B|11\rangle. \tag{2.2.7}$$

Cet état appartient à l'espace de Hilbert $\mathcal{H}_{AB} = \mathcal{H}_A \otimes \mathcal{H}_B$, de dimension $2^2 = 4$ (JAFFALI, 2020).

2.3 Modèles de calcul quantique

2.3.1 Opérations sur les états quantiques

Maintenant que nous avons défini les qubits et les systèmes quantiques, il est essentiel de comprendre comment manipuler ces états au moyen d'opérations quantiques.

Produit tensoriel. Pour modéliser plusieurs qubits, on utilise le *produit tensoriel* (ou de Kronecker). Si $M \in \mathbb{C}^{p\times q}$ et $N \in \mathbb{C}^{x\times y}$, alors :

$$M \otimes N \in \mathbb{C}^{(px)\times(qy)}, \tag{2.3.1}$$

qui agit sur l'espace de Hilbert $\mathcal{H}_p \otimes \mathcal{H}_x$. Pour deux qubits :

$$\begin{aligned} |0\rangle \otimes |0\rangle &= |00\rangle, & |0\rangle \otimes |1\rangle &= |01\rangle, \\ |1\rangle \otimes |0\rangle &= |10\rangle, & |1\rangle \otimes |1\rangle &= |11\rangle. \end{aligned}$$

2.3.2 Portes logiques quantiques

Les portes logiques quantiques manipulent les qubits, de manière analogue aux portes logiques classiques. Elles sont représentées par des matrices *unitaires* U satisfaisant la relation $U^\dagger U = I$, où $U^\dagger$ désigne la transposée conjuguée (ou adjointe) de U (Nielsen & Chuang, 2000).

2.3.2.1 Portes à un qubit

Pauli-X (NOT) : inverse l'état d'un qubit (analogue à une porte NOT classique) :

$$X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle, \qquad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Pauli-Y : combine une inversion d'état d'un qubit et une inversion de phase :

$$Y|0\rangle = i|1\rangle, \quad Y|1\rangle = -i|0\rangle, \qquad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}.$$

Pauli-Z : change la phase du qubit $|1\rangle$:

$$Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle, \qquad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Hadamard (H) : crée une superposition d'états :

$$H|0\rangle = \tfrac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad H|1\rangle = \tfrac{1}{\sqrt{2}}(|0\rangle - |1\rangle), \qquad H = \tfrac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

Rotation $R_x(\theta)$: effectue une rotation d'angle θ autour de l'axe x de la sphère de Bloch :

$$R_x(\theta) = \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & -i\sin\left(\frac{\theta}{2}\right) \\ -i\sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}.$$

Rotation $R_y(\theta)$: effectue une rotation d'angle θ autour de l'axe y :

$$R_y(\theta) = \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & -\sin\left(\frac{\theta}{2}\right) \\ \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}.$$

Rotation $R_z(\theta)$: effectue une rotation d'angle θ autour de l'axe z (modifie uniquement la phase relative des états) :

$$R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}.$$

2.3.2.2 Portes multi-qubits

Les portes multi-qubits agissent sur deux qubits ou plus. Elles sont cruciales pour générer l'intrication et effectuer des calculs complexes.

Controlled-NOT (CNOT) : applique X sur le qubit cible si le qubit de contrôle est $|1\rangle$:

$$\mathrm{CNOT}|c,t\rangle = \begin{cases} |0,t\rangle & \text{si } c=0, \\ |1,Xt\rangle & \text{si } c=1 \end{cases} \qquad \mathrm{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

SWAP : échange l'état de deux qubits :

$$\mathrm{SWAP}|q_1,q_2\rangle = |q_2,q_1\rangle, \qquad \mathrm{SWAP} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Toffoli (CCNOT) : applique X sur la cible si les deux qubits de contrôle sont $|1\rangle$.

Controlled-Unitary (CU) Porte contrôlée-U : applique U sur la cible si le contrôle est $|1\rangle$;

$$\mathrm{CU} = \begin{pmatrix} I_2 & 0 \\ 0 & U \end{pmatrix} \in \mathbb{C}^{4\times 4}$$

Le tableau de la figure 2.2 présente un récapitulatif des opérateurs quantiques les plus utilisés.

Gate	Symbol	Matrix
Pauli-X (NOT) Bit flip	X	$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$
Pauli-Y Bit + phase flip	Y	$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$
Pauli-Z Phase flip	Z	$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$
S Phase shift (π/2)	S	$\begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$
T Phase shift (π/4)	T	$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$
Hadamard Superposition	H	$\frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$
Rotation Z Z-axis rotation	$R_z(\theta)$	$\begin{pmatrix} e^{-\frac{i\theta}{2}} & 0 \\ 0 & e^{\frac{i\theta}{2}} \end{pmatrix}$
Rotation Y Y-axis rotation	$R_y(\theta)$	$\begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & -sin\left(\frac{\theta}{2}\right) \\ sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}$
Rotation X X-axis rotation	$R_x(\theta)$	$\begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & -isin\left(\frac{\theta}{2}\right) \\ -isin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}$
CNOT Conditional flip		$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$
Swap Swap qubits		$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

FIGURE 2.2 – Les opérateurs quantiques d'après (**jha2023quantum**).

2.3.3 Registre quantique

Un *registre quantique* est un système composé de plusieurs qubits, permettant de manipuler des états quantiques de dimension supérieure. Formellement, un registre de n qubits est un vecteur d'état dans un espace de Hilbert de dimension 2^n, noté $\mathcal{H} = \mathbb{C}^{2^n}$. L'état général d'un tel registre est une superposition linéaire des 2^n états de base, exprimée par la formule 2.2.4.

2.3.4 Circuits quantiques

Un *circuit quantique* est une représentation schématique des calculs effectués avec des qubits. Il est composé de lignes horizontales représentant les qubits, et de portes quantiques qui agissent sur ces lignes.

Les circuits se lisent de gauche à droite, mais les opérations sont exprimées mathématiquement de droite à gauche (composition fonctionnelle inverse).

Préparation d'un état de Bell : La figure 2.3 montre un circuit simple utilisant une porte Hadamard suivie d'une porte CNOTpour préparer l'état intriqué :

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \tag{2.3.2}$$

Ce circuit encode l'équation suivante :

$$\text{CNOT}(H \otimes I)|00\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

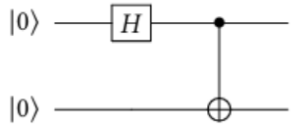


FIGURE 2.3 – Circuit quantique préparant un état de Bell.

Mesure dans les circuits quantiques : En mécanique quantique, mesurer un qubit change son état de manière irréversible. La mesure dans la *base computationnelle* $\{|0\rangle, |1\rangle\}$ revient à observer un état comme $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, ce qui donne :

— le résultat 0 avec probabilité $|\alpha|^2$ (le qubit devient $|0\rangle$),

— le résultat 1 avec probabilité $|\beta|^2$ (le qubit devient $|1\rangle$).

Généralement, le calcul quantique se termine par une mesure afin de réduire l'état quantique dans l'un des états de base. Dans le circuit quantique, cette opération est représentée comme le montre la figure 2.4 .La mesure est symbolisée dans les circuits par une icône en forme de compteur ou de mètre. Elle effondre l'état du qubit vers l'un des états de base.

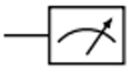


FIGURE 2.4 – Représentation de la mesure dans un circuit quantique (UMEANO, 2021).

Propriétés importantes :

— **Mesure différée** : une mesure peut souvent être reportée à la fin du circuit sans modifier le résultat final.

— **Mesure implicite** : les qubits non explicitement mesurés à la fin du circuit sont supposés l'être dans la base computationnelle.

2.3.5 Propriétés fondamentales du calcul quantique

L'informatique quantique se distingue fondamentalement de l'informatique classique grâce à des propriétés uniques issues des lois de la mécanique quantique (National Academies of Sciences, Engineering, and Medicine, 2019 ; Nielsen & Chuang, 2000). Ces propriétés confèrent aux ordinateurs quantiques un pouvoir de calcul inédit que l'on ne peut pas reproduire efficacement avec des systèmes classiques.

2.3.5.1 Parallélisme quantique

Le parallélisme quantique fait référence à la capacité d'un ordinateur quantique à évaluer simultanément une fonction sur un ensemble d'entrées en une seule opération, en exploitant la *superposition* et la linéarité des transformations unitaires.
Considérons une fonction $f : \{0,1\}^n \to \{0,1\}$. En appliquant l'opérateur unitaire U_f à la superposition uniforme de tous les états de base, on obtient

$$U_f \sum_{x\in\{0,1\}^n} |x\rangle|0\rangle \;=\; \sum_{x\in\{0,1\}^n} |x\rangle|f(x)\rangle \tag{2.3.3}$$

Une seule évaluation quantique suffit donc pour encoder les valeurs de f sur toutes les entrées, ce qui constitue la brique de base d'algorithmes comme Deutsch-Jozsa, Simon ou Grover (Deutsch & Jozsa, 1992 ; Grover, 1996).

2.3.5.2 Superposition

La superposition décrit la capacité d'un système quantique à se trouver dans une combinaison linéaire d'états de base 2.2.2. Prenons, par exemple, un photon pouvant être polarisé Nord ou Est ; avant toute mesure, il peut exister dans une superposition de ces deux polarisations. L'expérience de pensée du chat de Schrödinger pousse cette idée à l'extrême, illustrant qu'un système peut conserver plusieurs issues macroscopiquement distinctes tant qu'aucune mesure n'est effectuée (Schrödinger, 1935).

2.3.5.3 Intrication (Entanglement)

Deux ou plusieurs qubits sont dits *intriqués* lorsque l'état global ne peut pas être factorisé en états individuels ; la connaissance de l'un détermine instantanément l'état des autres, quelle que soit la distance qui les sépare (Bell, 1964 ; Einstein et al., 1935). L'état de Bell 2.3.2 en constitue l'exemple canonique. La mesure d'un qubit entraîne alors la disparition de la superposition et fixe corrélativement l'état du partenaire. L'intrication est le cœur de la téléportation quantique, de la cryptographie quantique (BB84, réseaux EPR) et des codes de correction d'erreurs (Bennett et al., 1993).

2.3.5.4 Interférence

Lorsque plusieurs trajectoires d'amplitudes quantiques mènent au même état, celles-ci se combinent ; elles peuvent se renforcer (interférence constructive) ou s'annihiler (interférence destructive), à l'image des franges claires et sombres observées dans l'expérience des doubles fentes de Young (Young, 1804). Dans les algorithmes quantiques, cette propriété est exploitée pour amplifier les probabilités des résultats souhaités et supprimer les indésirables : l'algorithme de Grover, par exemple, amplifie progressivement la probabilité de l'état solution jusqu'à l'obtenir avec une amplitude maximale, tandis que les trajectoires non solutions interfèrent destructivement (Grover, 1996).

2.4 Algorithmes fondamentaux

Dans cette section, nous présentons quelques-uns des algorithmes quantiques les plus emblématiques, qui illustrent la supériorité potentielle du calcul quantique pour certaines classes de problèmes.

2.4.1 Algorithme de Deutsch

Proposé par David Deutsch en 1985, c'est l'un des premiers algorithmes montrant un avantage quantique (Deutsch, 1985). Il permet de déterminer, avec une seule évaluation de la fonction, si une fonction booléenne $f : \{0, 1\} \to \{0, 1\}$ est **constante** ou **équilibrée**.

Exemples :

— Fonction constante : $f(x) = 0$ ou $f(x) = 1$.

— Fonction équilibrée : $f(x) = x$ ou $f(x) = 1 \oplus x$.

L'algorithme commence avec l'état $|\psi\rangle = |0\rangle|1\rangle$. On applique la porte Hadamard H aux deux qubits, produisant une superposition. Ensuite, une porte unitaire U_f encode la fonction f comme suit :

$$U_f|x, y\rangle = |x, y \oplus f(x)\rangle \tag{2.4.1}$$

Puis on applique à nouveau H au premier qubit, avant de mesurer. Le résultat indique si f est constante (résultat 0) ou équilibrée (résultat 1), avec une seule évaluation de f. La Figure 2.5 illustre le circuit quantique associé à l'algorithme de Deutsch.

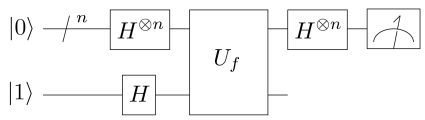


Figure 2.5 – Circuit quantique de l'algorithme de Deutsch (Deutsch, 1985).

2.4.2 Algorithme de Grover

Développé par Lov Grover en 1996, cet algorithme est destiné à la recherche dans une base de données non structurée (Grover, 1996). Il permet de retrouver un élément parmi N avec une complexité $O(\sqrt{N})$, contre $O(N)$ pour un algorithme classique.

L'algorithme commence par la création d'une superposition uniforme sur tous les états possibles, puis applique de manière répétée un oracle (qui marque la solution) suivi d'une opération de diffusion qui amplifie les amplitudes des états solution. Ce processus est répété $O(\sqrt{N})$ fois. L'algorithme est utile dans des problèmes d'optimisation, de recherche exhaustive, ou d'analyse combinatoire.

2.4.3 Algorithme de Shor

Proposé par Peter Shor en 1994, cet algorithme résout efficacement le problème de factorisation d'entiers, menaçant des systèmes de cryptographie comme RSA(Shor, 1994). Il repose sur :

— **Réduction à un problème de périodicité** : factoriser N revient à déterminer la période r d'une fonction $f(x) = a^x \mod N$.

— **Utilisation de la Quantum Fourier Transform (QFT)** pour estimer efficacement cette période dans l'espace quantique.

L'algorithme fonctionne en temps polynomial et pourrait casser la sécurité de systèmes classiques, d'où le développement actuel de la cryptographie post-quantique.

2.4.4 Algorithmes NISQ :

Avec l'émergence de dispositifs quantiques bruyants à échelle intermédiaire NISQ, des algorithmes hybrides ont été proposés pour exploiter la puissance quantique malgré les limitations matérielles actuelles. Deux des plus importants sont : Variational Quantum Eigensolver (VQE) et Quantum Approximate Optimization Algorithm (QAOA).

VQE – Variational Quantum Eigensolver : est utilisé pour approximer l'état fondamental d'un Hamiltonien donné, un problème central en chimie quantique. Il combine un circuit quantique paramétré (appelé ansatz) avec un optimiseur classique pour ajuster les paramètres afin de minimiser l'énergie moyenne mesurée (Peruzzo et al., 2014).

QAOA – Quantum Approximate Optimization Algorithm : introduit par Farhi et al. (2014), a été conçu pour résoudre des problèmes d'optimisation combinatoire . Il utilise une alternance de circuits paramétrés basés sur l'Hamiltonien du problème et un Hamiltonien de mélange. Comme pour le VQE, un optimiseur classique ajuste les paramètres pour maximiser la probabilité d'obtenir la solution optimale.

Ces deux algorithmes incarnent l'approche hybride classique-quantique typique de l'ère NISQ, où la partie quantique explore des espaces de solutions complexes et la partie classique guide la convergence vers l'optimum.

2.5 Les ordinateurs quantiques

Un ordinateur quantique fonctionnel ne repose pas uniquement sur la création d'un matériel à faible taux d'erreur. Il nécessite également un écosystème logiciel complet comprenant des langages de programmation, des compilateurs, des simulateurs, des kits de développement (Software Development Kit (SDK)) ainsi que des solutions d'accès via le cloud.

Dans ce qui suit, nous présentons les principales catégories d'ordinateurs quantiques ainsi que leurs spécificités. Essentiellement, on distingue deux grandes familles :

— les **simulateurs quantiques**, destinés principalement à l'étude de phénomènes physiques complexes,

— les **ordinateurs quantiques à portes logiques**, plus proches du modèle universel et capables d'exécuter des algorithmes quantiques généraux (Gondran & Gondran, 2020).

2.5.1 Les simulateurs quantiques

Les simulateurs quantiques regroupent deux sous-catégories principales :

1. **Les simulateurs analogiques** (ou « ordinateurs à la Feynman ») : ils ont été conçus pour reproduire directement l'évolution de systèmes quantiques complexes, difficiles à simuler même sur les supercalculateurs classiques (Gondran & Gondran, 2020 ; Richard & Feynman, 1982). Leur fonctionnement est comparable aux anciens ordinateurs analogiques classiques. Pour l'heure, aucune supériorité systématique par rapport aux supercalculateurs n'a été démontrée.
2. **Les ordinateurs adiabatiques**, popularisés par la société D-Wave : ils sont adaptés à une classe particulière d'algorithmes, dits *algorithmes de recuit quantique*(Vert, 2021). Leur principe repose sur une évolution lente et contrôlée du système depuis un état initial vers un état final représentant une solution approchée au problème étudié(Ezratty, 2019). Bien qu'intéressants, ces dispositifs sont souvent qualifiés de « mi-quantiques, mi-classiques » (Gondran & Gondran, 2020).

2.5.2 Les ordinateurs quantiques à portes logiques

La deuxième grande catégorie repose sur le modèle universel d'ordinateur quantique. Ces machines utilisent des portes quantiques pour manipuler les qubits et peuvent, en principe, exécuter tout algorithme quantique (Ezratty, 2019). Elles présentent un gain de puissance et de vitesse considérable par rapport aux simulateurs quantiques.

Cependant, leur utilisation reste aujourd'hui limitée :

— le nombre de qubits disponibles demeure faible (de l'ordre d'une centaine),

— le bruit quantique impacte fortement la fiabilité des résultats (Gondran & Gondran, 2020).

On distingue deux situations :

— les ordinateurs **sans correction d'erreur**, utilisés lorsque des solutions approchées suffisent,

— les ordinateurs **avec codes correcteurs d'erreurs**, indispensables pour des calculs exacts, par exemple la factorisation de grands entiers premiers. Dans ce cas, l'intégration d'un schéma de correction est primordiale afin d'obtenir des résultats corrects (Gondran & Gondran, 2020).

2.5.3 Simulation classique des circuits quantiques

En attendant des machines corrigées, les simulateurs classiques restent incontournables pour valider le matériel, explorer les algorithmes et tester la robustesse au bruit (National Academies of Sciences, Engineering, and Medicine, 2019). La mémoire requise croît comme 2^n amplitudes complexes ; simuler 50 qubits en double précision nécessite déjà ~ 2 Po (Pednault et al., 2017). Les plus gros supercalculateurs atteignent aujourd'hui 50–60 qubits en *full-state*.

le tableau 2.1 présente un ensemble de simulateurs de circuits quantiques parmi les plus utilisés dans la communauté, en précisant leur capacité maximale en nombre de qubits, la possibilité d'exécution locale ainsi que le framework logiciel associé.

Simulateur	Qubits max.*	Exécution locale	Framework
`qasm_simulator` (Qiskit)	32	Oui	Python / Qiskit (Team, 2023)
`aer_simulator` (stabilizer)	63	Non	Qiskit Aer (Team, 2023)
`QVM` (Rigetti)	30	Non	PyQuil / Forest SDK (Smith et al., 2016)
ProjectQ back-end	28–30	Oui	ProjectQ (Steiger et al., 2018)
QDK `full_state`	30	Oui	Microsoft Q# / QDK (et al., 2018)
`qsim` (Google Cirq)	40†	Oui	Cirq / Google (AI, 2020)
Qubit Toaster	> 40†	Oui	Stand-alone C++ (Kolodziej, 2021)

* Limite indicative sur un poste de travail classique. † La limite dépend de la mémoire RAM et du parallélisme.

Table 2.1 – Quelques simulateurs de circuits quantiques largement utilisés.

2.6 Correction d'erreurs et tolérance aux fautes

Les systèmes quantiques sont extrêmement sensibles aux perturbations environnementales et aux imperfections de contrôle, ce qui entraîne des erreurs dans les calculs quantiques. Contrairement aux bits classiques, les qubits ne peuvent pas être clonés à volonté (théorème de non-clonage), ce qui rend la correction d'erreurs plus complexe.

Pour garantir un calcul fiable, même en présence de bruit, l'informatique quantique a développé des Quantum Error Correcting Codes (QECC), tels que le code de Shor, le code de Steane, ou encore les codes topologiques comme le code de surface (Fowler et al., 2012 ; Shor, 1995 ; Steane, 1996).

Principes des codes correcteurs

— Les informations sont encodées de manière redondante sur plusieurs qubits physiques pour former un qubit logique.

— Des mesures de syndrome permettent de détecter les erreurs sans effondrer l'état logique.

— Les opérations correctrices sont ensuite appliquées en fonction du syndrome mesuré.

La mise en œuvre efficace de la correction d'erreurs nécessite également des architectures de calcul dites tolérantes aux fautes, qui permettent d'exécuter des algorithmes complets

même avec des qubits imparfaits, à condition que le taux d'erreur reste en dessous d'un certain seuil (Preskill, 1998 ; Terhal, 2015).

Conclusion

À travers les concepts fondamentaux abordés dans ce chapitre, nous comprenons que le calcul quantique repose sur des principes physiques totalement différents du calcul classique, impliquant des phénomènes comme la superposition, l'intrication, et l'interférence. L'exploitation de ces propriétés, conjuguée à des algorithmes adaptés et à des mécanismes de correction d'erreurs, constitue le socle sur lequel reposera la prochaine révolution technologique.Les chapitres suivants approfondiront les applications ciblées de ces principes, en particulier dans le cadre du conception automatique des circuits quantique .

Chapitre 3

Généralités sur les métaheuristiques bio-inspirées

Introduction

L'optimisation de problèmes complexes, souvent non linéaires, à grande dimension ou à contraintes multiples, constitue un enjeu majeur dans de nombreux domaines scientifiques et techniques. Dans ce contexte, les approches classiques atteignent rapidement leurs limites, tant en termes de performance que de flexibilité. C'est pourquoi les *métaheuristiques bio-inspirées* suscitent un intérêt croissant : elles offrent une alternative robuste et adaptable, capable de fournir des solutions de haute qualité, même approximatives, dans des espaces de recherche vastes et mal structurés. (Blum & Roli, 2003 ; Talbi, 2009)
Dans ce chapitre, nous présenterons les fondements théoriques des *métaheuristiques bio-inspirées*, en exposant leurs principales catégories, leurs mécanismes de fonctionnement ainsi que les principes spécifiques à chaque paradigme. Cette base conceptuelle vise à fournir les éléments nécessaires pour appréhender leur potentiel dans la résolution de problèmes complexes, à l'instar de ceux abordés dans les sections suivantes.

3.1 Classification des métaheuristiques bio-inspirées

Les métaheuristiques bio-inspirées désignent une famille de méthodes d'optimisation stochastiques fondées sur des mécanismes observés dans la nature, tels que l'évolution biologique, le comportement des essaims ou l'adaptation dynamique. Elles simulent des phénomènes naturels comme la sélection naturelle (dans les algorithmes évolutionnaires), ou les comportements collectifs (dans les algorithmes d'intelligence en essaim) (Katoch et al., 2021).
Nous illustrons via la figure 3.1 comment les métaheuristiques bio-inspirées sont organisées en différentes approches.

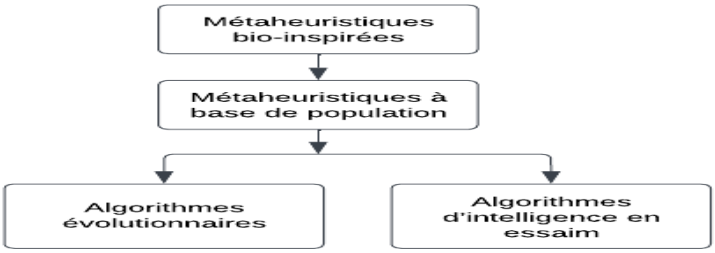


Figure 3.1 – Classification des métaheuristiques bio-inspirées. (Katoch et al., 2021)

Parmi l'ensemble des métaheuristiques bio-inspirées, certaines approches comme l'optimisation par essaims particulaires ou par colonies de fourmis connaissent un succès notable. Néanmoins, l'algorithme génétique (AG) occupe une place centrale dans ce travail. En effet, ses mécanismes d'évolution, sa capacité à explorer efficacement de grands espaces de recherche et son aptitude à traiter des objectifs multiples en font un outil particulièrement adapté à la problématique étudiée. Les sections suivantes présenteront donc les différentes familles de métaheuristiques, avec un accent particulier mis sur les AG, qui constitueront le fondement méthodologique de notre démarche.

3.1.1 Les algorithmes évolutionnaires :

L'*Algorithme Évolutionnaire (AE)* est une méthode d'optimisation inspirée des principes de la théorie de l'évolution biologique, traduite en langage algorithmique. Ces algorithmes opèrent sur une population de solutions potentielles, qu'ils font évoluer au fil des générations dans le but de trouver des solutions optimales ou satisfaisantes à un problème donné.(Bäck & Schwefel, 1993) Plus précisément, un algorithme évolutionnaire suit un cycle itératif basé sur les étapes suivantes :

— **Génération initiale** : Création aléatoire d'une population de structures (ou individus), chacune représentant une solution potentielle.

— **Évaluation** : Chaque structure est analysée via une fonction d'évaluation (ou *fitness*) qui mesure sa performance ou son aptitude à résoudre le problème.

— **Sélection** : Les structures les mieux évaluées sont sélectionnées pour engendrer la nouvelle génération. Cette étape favorise la survie des plus aptes.

— **Variation (croisement et mutation)** : Des opérateurs de variation sont appliqués aux structures sélectionnées pour créer de nouvelles solutions.

— **Remplacement** : La population est mise à jour, partiellement ou totalement, avec les nouvelles structures générées.

Ce processus n'est réellement efficace que s'il repose sur des fondations solides. Deux éléments sont particulièrement déterminants dans la réussite d'un algorithme évolutionnaire. Le premier concerne la représentation des solutions, qui doit être adaptée à la nature du problème et aux opérateurs de variation employés. Le second est la fonction d'évaluation, véritable moteur de l'évolution, qui guide la recherche en mesurant la qualité des solutions et en orientant l'algorithme vers les zones les plus prometteuses de l'espace de solutions (*Evolutionary Algorithms in Engineering Design Optimization*, 2022) .

Dans ce qui suit, nous abordons deux principales catégories d'algorithmes évolutionnaires à savoir l'algorithme génétique classique et l'algorithme génétique multi-objectifs.

3.1.1.1 Algorithme génétique classique :

L'*AG* a été développé par John Holland en 1975 (Holand, 1975 ; Holland, 1984). Il constituent une méthode d'optimisation métaheuristique s'inspirant des principes de l'évolution biologique darwinienne. En reproduisant des mécanismes naturels tels que la sélection, le croisement et la mutation, ces algorithmes sont conçus pour résoudre efficacement une grande variété de problèmes complexes dans divers domaines. Leur capacité à explorer de vastes espaces de recherche tout en évitant les pièges des optima locaux en fait un outil particulièrement puissant pour la recherche de solutions optimales

(Barnier & Brisset, 1999). Nous présenterons ci-dessous les composants fondamentaux d'un algorithme génétique :

Gène : représente une unité élémentaire d'information, correspondant à un paramètre du problème (par exemple, une variable de décision).

Chromosome : séquence ordonnée de gènes constituant une solution candidate complète. Selon l'application, il peut être représenté sous forme de chaîne binaire, vecteur numérique ou autre structure adaptée.

au-delà d'une simple chaîne binaire, un chromosome peut encoder un circuit quantique de plusieurs façons selon le niveau d'abstraction recherché et les contraintes matérielles. Les schémas les plus courants sont :

— **Séquence linéaire de portes (gate list) :** Chaque gène code une porte G (type, qubits cibles/contrôle, paramètres d'angle), et le chromosome est la concaténation ordonnée de ces gènes :

$$C = (g_1, g_2, \ldots, g_L), \quad g_i = (G_i, q_i^{\text{ctrl}}, q_i^{\text{tgt}}, \theta_i)$$

où L est la longueur du circuit. Utilisée pour la synthèse directe d'unitaires ou d'états cibles (Creevey et al., 2023 ; Miranda et al., 2023).

— **Encodage par blocs / couches (modulaire) :** Le circuit est structuré en une séquence de blocs unitaires :

$$C = (B_1, B_2, \ldots, B_K), \quad B_j = \prod_{i=1}^{m_j} G_{j,i}$$

où B_j regroupe un ensemble local de portes. Favorise des croisements structurés (échange de blocs) (Potoček et al., 2018).

— **Grille $n \times m$ (qubits $\times$ profondeur) :** Un individu est représenté comme une matrice :

$$C = [c_{i,j}] \in \mathbb{G}^{n \times m}, \quad c_{i,j} \in \{\text{porte}, \varnothing\}$$

où n est le nombre de qubits, m la profondeur, et chaque cellule $c_{i,j}$ décrit une opération (ou vide). Bien adapté aux contraintes NISQ (Sünkel et al., 2023).

Individu : désigne une solution candidate au sein de la population. Dans le contexte des AG, un individu est équivalent à un chromosome, mais le terme « individu » insiste sur son appartenance à la population évolutive.

Population : Ensemble d'individus sur lequel s'opère l'évolution. Sa taille influence l'exploration (diversité) et l'efficacité computationnelle. L'initialisation est souvent aléatoire pour couvrir un large espace de recherche.

Croisement : Le croisement consiste à combiner deux parents pour produire un ou plusieurs nouveaux individus (fils), dans le but de mélanger leurs caractéristiques et ainsi explorer de nouvelles combinaisons de solutions. Cette opération repose souvent sur des mécanismes probabilistes qui déterminent la contribution de chaque parent à l'enfant. Dans le cadre de la synthèse de circuits quantiques, cela revient à combiner des séquences de portes afin de générer des circuits potentiellement plus performants.

L'objectif est de réutiliser des blocs efficaces tout en introduisant de la diversité pour améliorer la qualité globale de la population.

Plusieurs types de croisements sont couramment utilisés :

— **Croisement à k-points** : une généralisation du croisement à un point où plusieurs points de coupure sont choisis. Les segments sont alors alternés entre les parents. Cette variante introduit une recombinaison plus riche mais peut parfois perturber des schémas génétiques favorables (Mitchell, 1998).

— **Croisement uniforme** : chaque gène de l'enfant est choisi aléatoirement à partir de l'un des deux parents, selon une probabilité fixée (souvent 50/50). Ce schéma favorise un mélange très diversifié mais peut détruire des structures globales utiles (Syswerda, 1989).

— **Croisement par ordre** : utilisé principalement pour les problèmes de permutation. Une sous-séquence de gènes est copiée directement du premier parent, tandis que les autres positions sont complétées en respectant l'ordre relatif des gènes du second parent. Cela garantit la validité de la permutation tout en préservant une partie de la structure de chaque parent (Goldberg, 1989).

— **Croisement par cycle** : les gènes sont hérités de manière cyclique entre les deux parents. Ainsi, chaque position dans l'enfant provient d'un seul parent, en suivant une logique de cycles. Ce type de croisement est particulièrement adapté aux représentations permutationnelles (Goldberg, 1989).

— **Croisement à appariement partiel** : largement utilisé dans les problèmes combinatoires comme le voyageur de commerce. Une portion de la séquence est copiée directement du premier parent, puis les gènes restants sont remplis en respectant une correspondance entre les deux parents. Ce mécanisme permet de préserver certaines positions clés tout en maintenant une permutation valide (Goldberg & Lingle, 1985).

Ces différentes variantes permettent d'équilibrer la préservation des bonnes solutions déjà découvertes et l'exploration de nouvelles combinaisons génétiques.

Mutation : La mutation constitue un mécanisme central des algorithmes génétiques, jouant un rôle crucial dans le maintien de la diversité génétique au sein de la population. Elle agit en introduisant des changements aléatoires dans certains individus, ce qui permet de perturber légèrement leurs structures génétiques. Cette opération vise à éviter la convergence prématurée vers des solutions sous-optimales et à explorer de nouvelles régions de l'espace de recherche qui n'ont pas encore été visitées.

L'efficacité de la mutation repose en grande partie sur un réglage adapté de son taux. Un taux trop élevé risque de transformer l'algorithme en une simple recherche aléatoire, tandis qu'un taux trop faible peut mener à la stagnation dans un optimum local. Ainsi, ce paramètre clé est généralement maintenu à une valeur faible afin d'éviter des perturbations excessives, tout en assurant une exploration suffisante de l'espace des solutions.

Plusieurs types de mutation sont couramment utilisés dans la littérature :

— **Mutation par déplacement** : un gène est extrait de sa position et inséré ailleurs dans la séquence, modifiant l'ordre des éléments.

— **Mutation par inversion** : une portion de la séquence est inversée, ce qui change localement l'ordre des gènes.

— **Mutation par brouillage (scramble)** : les gènes d'un sous-ensemble choisi aléatoirement sont mélangés de manière aléatoire.

— **Mutation inversée** : variante où certains gènes sont remplacés par leurs valeurs inverses (utile si les représentations le permettent, par exemple codage binaire).

Ces différentes approches permettent d'introduire des perturbations variées, plus ou moins fortes, afin d'équilibrer l'exploration et l'exploitation au sein de l'algorithme.

Selection : La sélection est une étape clé dans le fonctionnement d'un algorithme génétique. Elle détermine quels individus seront retenus pour participer aux opérations de croisement *(sélection des parents)* et lesquels seront conservés dans la génération suivante *(sélection des survivants)*. Cette étape doit assurer un équilibre entre deux objectifs contradictoires :

— **Exploitation** : favoriser les individus ayant la meilleure fitness, afin de guider rapidement la recherche vers des solutions de qualité.

— **Exploration** : maintenir une diversité génétique suffisante pour éviter la convergence prématurée vers des minima locaux.

Plusieurs types de stratégies de sélection sont couramment utilisées dans la littérature :

— **Sélection par roulette (Roulette Wheel Selection)** : Chaque individu reçoit une probabilité proportionnelle à sa fitness par rapport à la somme totale des fitness de la population. Plus un individu est performant, plus la portion de « roulette » qui lui est attribuée est grande. Cette méthode favorise les bons individus, mais peut parfois accentuer la perte de diversité si la distribution des fitness est trop déséquilibrée.

— **Sélection par rang (Rank Selection)** : Les individus sont triés selon leur fitness, puis leurs chances de sélection dépendent uniquement de leur rang et non de leur valeur absolue de fitness. Cette approche réduit les effets de domination des meilleurs individus et maintient une pression sélective plus stable au fil des générations.

— **Sélection par tournoi (Tournament Selection)** : Un sous-groupe de taille k est choisi aléatoirement dans la population, puis le meilleur individu de ce groupe est sélectionné pour la reproduction. Plus k est grand, plus la pression sélective est élevée. Cette méthode est simple à mettre en œuvre, robuste et couramment utilisée dans les algorithmes évolutionnaires modernes.

— **Sélection de Boltzmann (Boltzmann Selection)** : Inspirée de la physique statistique, elle associe à chaque individu une probabilité définie par une loi exponentielle :

$$P(i) \propto \exp\left(\frac{f(i)}{T}\right),$$

où $f(i)$ est la fitness de l'individu et T une « température » qui décroît avec les générations. Au début (T élevée), même les individus moins performants ont une chance d'être sélectionnés, favorisant l'exploration. Progressivement (T faible), la sélection devient plus stricte, privilégiant les meilleurs individus.

— **Sélection stochastique universelle (Stochastic Universal Sampling, SUS)** : Au lieu de tirer plusieurs fois une roulette, cette méthode place des pointeurs

équidistants sur la roue et sélectionne les individus correspondants. Elle assure une meilleure représentativité de la population, réduit l'effet de hasard et maintient la proportionnalité à la fitness, tout en garantissant une pression sélective plus régulière.

La figure 3.2 présente les différents types d'opérateurs évoqués précédemment : croisement, mutation et sélection.

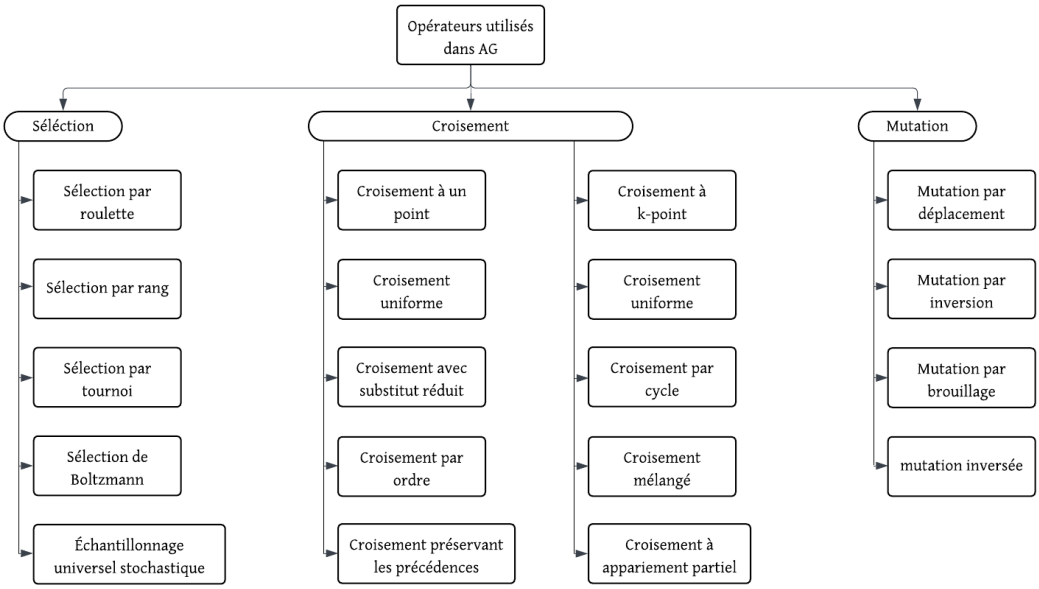


FIGURE 3.2 – Les opérateurs utilisés dans l'AG. (KATOCH et al., 2021)

Fonction de fitness : Fonction qui mesure la qualité d'une solution candidate en indiquant dans quelle mesure elle satisfait l'objectif fixé. Elle guide l'évolution en classant les individus, permettant de privilégier les meilleurs lors de la sélection. Cette fonction est définie selon le problème étudié et peut correspondre, par exemple, à la minimisation d'un coût ou à l'amélioration d'une performance. Dans le cas particulier de la synthèse de circuits quantiques, l'aptitude correspond à la fidélité avec laquelle un circuit candidat C reproduit le comportement d'un circuit cible G, ce qui permet de comparer les différentes solutions générées par l'algorithme. Les principales familles de fonctions de fitness utilisées sont :

— **Fidélité état (State-vector fidelity)** : Utilisée lorsque l'on vise un état cible $|\psi_{\text{target}}\rangle$.

$$F_{\text{state}} = \left|\langle \psi_{\text{target}} \mid \psi_C \rangle\right|^2$$

Avantages : intuitive, directement liée à la probabilité de retrouver l'état cible. *Inconvénients :* dépend de l'état d'entrée choisi ; ne capture pas toute l'unitaire. (CREEVEY et al., 2023)

— **Fidélité unitaire (Trace overlap / Unitary fidelity)** : Comparaison globale entre deux opérateurs unitaires U_G (cible) et U_C (candidat).

$$F_{\text{unitary}} = \frac{1}{2^n}\left|\text{Tr}\left(U_G^\dagger U_C\right)\right|$$

Avantages : évalue l'équivalence fonctionnelle complète du circuit. *Inconvénients :* coûteux en calcul matriciel pour n grand. (Potoček et al., 2018)

Processus évolutionnaire : Comme illustré dans la **Figure 3.3**, l'algorithme génétique commence par une population initiale à la génération k. Lors de la phase de *reproduction*, certains individus P sont sélectionnés selon une probabilité donnée. Deux parents, notés $P1$ et $P2$, sont choisis pour participer à l'opération de croisement (*crossover*), réalisée avec une probabilité P_c. Celle-ci génère de nouveaux descendants, notés $C1$ et $C2$. Parallèlement, un individu P peut subir une *mutation* avec une probabilité P_m, donnant naissance à un nouvel individu modifié P'. L'ensemble des individus issus du croisement ($C1$, $C2$) et de la mutation (P'), ainsi que les individus non modifiés, passent ensuite par une phase d'*évaluation*. Cette étape consiste à calculer leur valeur d'aptitude (*fitness*) afin de déterminer leur qualité.

Enfin, les meilleurs individus sont retenus pour former la population de la génération suivante $k+1$. Ce cycle est répété jusqu'à atteindre un critère d'arrêt défini, tel qu'un nombre maximal de générations ou une performance cible (Barnier & Brisset, 1999).

La figure 3.3 décrit le déroulement de l'AG.

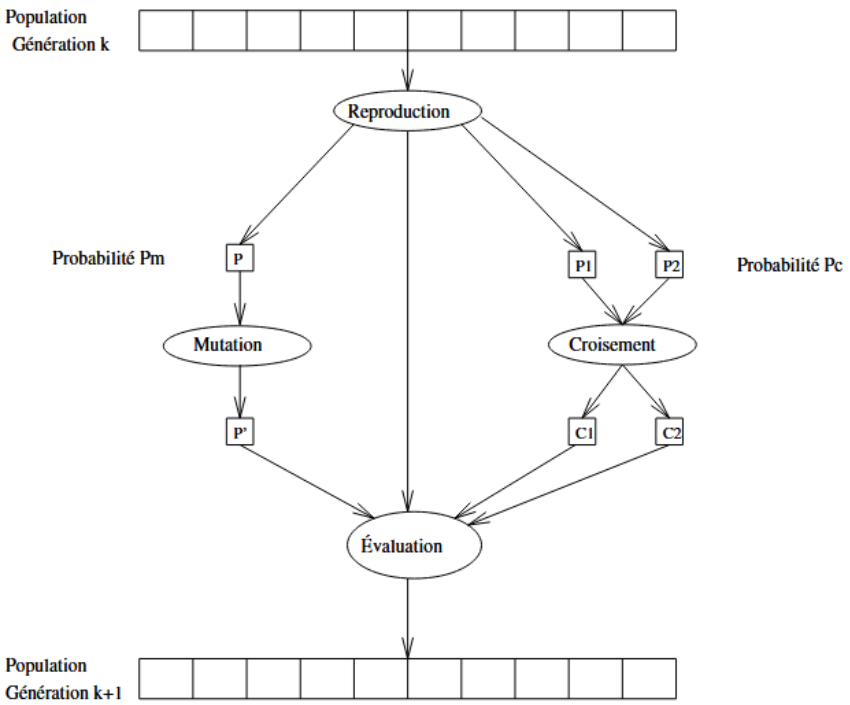


Figure 3.3 – Principe général de l'AG. (Barnier & Brisset, 1999)

3.1.1.2 Algorithme génétique multi-objectif (NSGA-II) :

NSGA-II est une extension de l'algorithme génétique classique, adapté à l'optimisation simultanée de plusieurs objectifs souvent contradictoires. Il conserve les mécanismes de base des algorithmes génétiques — sélection, croisement, mutation — mais se différencie principalement par la manière dont la fonction de fitness est évaluée. (Katoch et al., 2021). NSGA-II repose sur la notion de dominance de Pareto pour classer les solutions, et vise à identifier un ensemble de solutions non dominées appelé front de Pareto, représentant différents compromis entre les objectifs. Pour éviter la convergence prématurée vers un

seul optimum, ces algorithmes intègrent souvent des mécanismes de préservation de la diversité, tels que la crowding distance(Abdou et al., 2013).

1. **Dominance de Pareto**
   La base de comparaison entre solutions dans un NSGA-II repose sur la *dominance de Pareto*. Une solution A domine une solution B si :
   — A est au moins aussi bonne que B sur tous les objectifs ;
   — A est strictement meilleure que B sur au moins un objectif.

   Une solution est dite *non dominée* si aucune autre ne la domine. Le **front de Pareto** est alors l'ensemble de toutes les solutions non dominées de la population. L'algorithme cherche à s'approcher et à densifier ce front au fil des générations (Deb, 1999).
2. **Tri non dominé**
   Dans *NSGA-II*, les solutions sont classées en niveaux de dominance :
   — Le premier niveau contient les solutions non dominées (front 1).
   — Une fois celles-ci retirées, on extrait les non dominées restantes (front 2), etc.

   Ce classement permet une sélection préférentielle des individus dans les meilleurs fronts (Deb, 1999).
3. **Distance de densité (crowding distance)**
   Pour maintenir une bonne diversité des solutions sur le front de Pareto, NSGA-II utilise une métrique appelée **distance de densité**, qui évalue l'isolement d'un individu dans l'espace des objectifs. Cette distance permet de favoriser les solutions éloignées de leurs voisines (et donc de couvrir uniformément le front).

   Formule de distance de densité : (Imani et al., 2024)

$$d_i = \sum_{m=1}^{M} \frac{f_{i+1}^{(m)} - f_{i-1}^{(m)}}{f_{\max}^{(m)} - f_{\min}^{(m)}} \tag{3.1.1}$$

   où :
   — $f_{i+1}^{(m)}$ et $f_{i-1}^{(m)}$ sont les valeurs du m^{e} objectif pour les voisins de l'individu i (dans un front trié par objectif),
   — $f_{\max}^{(m)}$ et $f_{\min}^{(m)}$ sont les valeurs extrêmes du m^{e} objectif dans le front.
   — M est le nombre total d'objectifs.

   Les solutions situées aux extrémités du front de Pareto (celles qui atteignent, pour au moins un objectif, la valeur $f_{\min}^{(m)}$ ou $f_{\max}^{(m)}$) se voient attribuer une distance infinie afin d'être toujours conservées, car elles définissent l'étendue du front.
4. **Sélection avec tournoi et distance de densité**
   La sélection dans NSGA-II s'effectue par *tournoi binaire* entre deux individus (Imani et al., 2024) :
   — Si l'un appartient à un front de dominance inférieur, il est choisi ;
   — Si les deux sont dans le même front, celui avec la plus grande **distance de densité** est retenu.

   Cela assure un équilibre entre convergence vers le front et diversité des solutions.

3.1.2 Les algorithmes d'intelligence en essaim :

Swarm Intelligence (SI) est un paradigme d'optimisation inspiré des comportements collectifs observés dans la nature (colonies de fourmis, essaims d'abeilles, bancs de poissons), dans lequel des agents simples interagissent localement, sans contrôle centralisé, pour faire émerger une solution globale. Grâce à ses propriétés d'auto-organisation, de coopération et de décentralisation, elle permet de concevoir des méthodes heuristiques efficaces pour l'exploration de grands espaces de recherche, en assurant un bon compromis entre exploration et exploitation (Benkirat, 2019 ; Kumar & Pathak, 2019). Nous présentons ci-dessous les principes de ce paradigme :

Coordination : Les agents doivent échanger des informations sur leur environnement local afin d'agir collectivement de manière cohérente vers un objectif commun.

Coopération : Les agents doivent unir leurs efforts pour accomplir des tâches complexes qu'ils ne peuvent résoudre individuellement.

Diversité de réponse : Les ressources et agents sont répartis de manière à éviter toute concentration excessive, assurant ainsi une plus grande résilience face aux variations de l'environnement.

Stabilité : Le système maintient une certaine constance de comportement malgré des perturbations environnementales mineures.

Adaptabilité : La dynamique collective évolue en réponse aux changements significatifs dans l'environnement, permettant au système de s'ajuster de manière efficace.

Auto-organisation : Les comportements collectifs émergent des interactions simples et locales entre les agents, sans nécessiter de contrôle centralisé.

La tableau 3.1 présente les algorithmes les plus importants d'intelligence en essaim

Algorithmes	**Auteur**	**Année**
Optimisation par colonie de fourmis	Dorigo, M, et al.	1991
Optimisation par essaims particulaires	Eberhart & Kennedy (1995)	1995
Optimisation par colonie d'abeilles	Dervis Karaboga (2005)	2005
Gouttes d'eau intelligentes	Shah-Hosseini	2007
Algorithmes des lucioles	Xin-She Yang (2008)	2008
Recherche du Coucou	Xin-She Yang et Suash Deb	2009
Algorithme des chauves-souris	Xin-She Yang	2010

Table 3.1 – Algorithmes SI importants (Kumar & Pathak, 2019)

Deux principales catégories d'algorithmes d'intelligence en essaim sont présentées ci-dessous :

3.1.2.1 Algorithme de colonie de fourmis (ACO) :

L'*ACO* est une méthode d'optimisation inspirée du comportement social des fourmis, notamment leur manière de découvrir et renforcer les chemins les plus courts vers une ressource à l'aide de marques chimiques appelées phéromones. Proposée initialement par Marco Dorigo dans les années 1990, cette approche modélise un ensemble d'agents

coopératifs (les “fourmis artificielles”) qui explorent un espace de solutions et s’appuient sur des mécanismes d’apprentissage indirect. L’ACO est particulièrement efficace pour les problèmes d’optimisation combinatoire. Elle a ensuite été enrichie par des variantes hybrides, notamment en combinant des techniques classiques et quantiques pour élargir ses capacités de résolution dans des contextes complexes ou contraints (Blum, 2024 ; de Andoin & Echanobe, 2022).
Les principes fondamentaux régissant cet algorithme sont présentés dans ce qui suit :

Construction des solutions : Un ensemble de "fourmis artificielles" construit des solutions en explorant un graphe représentant le problème à résoudre. Chaque nœud du graphe peut être interprété comme un choix ou une étape dans la solution.

Utilisation des phéromones : Les fourmis sélectionnent les étapes suivantes en fonction d’une probabilité influencée par la quantité de phéromones déposées sur les chemins disponibles ainsi que par une fonction heuristique locale.

Mise à jour des phéromones : Après chaque itération, les niveaux de phéromones sont ajustés afin de renforcer les chemins correspondant aux solutions de meilleure qualité et d’affaiblir ceux associés aux solutions moins performantes, orientant ainsi l’exploration vers des régions prometteuses de l’espace de recherche.

Évaporation des phéromones : Un mécanisme d’évaporation réduit progressivement les niveaux de phéromones sur les chemins, permettant d’éviter une convergence prématurée vers un optimum local.

Dans ce qui suit, nous décrivons les étapes clés de l’ACO(de Andoin & Echanobe, 2022) :

1. **Initialisation** :
   - **Paramètres** : Définir les paramètres principaux tels que :
     - m : nombre de fourmis.
     - ρ : taux d’évaporation des phéromones.
     - α : poids de l’influence des phéromones.
     - β : poids de l’information heuristique.
   - **Traçage initial** : Initialiser les niveaux de phéromones τ_{ij} sur tous les chemins possibles à une valeur de base τ_0.
2. **Construction des solutions** :
   - **Déplacement des fourmis** : Chaque fourmi commence à une position initiale et construit une solution pas à pas.
   - Le choix de l’étape suivante est basé sur une règle probabiliste (Dorigo & Socha, 2018) :

$$p_{ij} = \begin{cases} \dfrac{\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}}{\sum\limits_{k \in N_i} \tau_{ik}^{\alpha} \cdot \eta_{ik}^{\beta}}, & \text{si } j \in N_i \\ 0, & \text{sinon} \end{cases} \tag{3.1.2}$$

   où :
   - p_{ij} est la probabilité de choisir le chemin de i vers j,
   - τ_{ij} est la quantité de phéromones sur le chemin (i, j),

— η_{ij} est l'information heuristique (par exemple, $\eta_{ij} = \frac{1}{d_{ij}}$ pour le TSP),
— N_i est l'ensemble des nœuds encore accessibles depuis i.

3. **Évaluation des solutions** :
   — Chaque solution construite par une fourmi est évaluée selon une fonction objectif spécifique au problème traité.
4. **Mise à jour des phéromones** :
   — **Évaporation globale** :

$$\tau_{ij} \leftarrow (1-\rho) \cdot \tau_{ij} \quad \text{(Dorigo \& Socha, 2018)} \tag{3.1.3}$$

   — **Renforcement des bonnes solutions** :

$$\tau_{ij} \leftarrow \tau_{ij} + \Delta\tau_{ij} \quad \text{(Dorigo \& Socha, 2018)} \tag{3.1.4}$$

   avec $\Delta\tau_{ij}$ proportionnelle à la qualité de la solution (par exemple, $\Delta\tau_{ij} = \frac{Q}{L}$, où L est la longueur de la solution et Q une constante).
5. **Condition d'arrêt** :
   — Répéter les étapes 2 à 4 jusqu'à ce que l'un des critères suivants soit atteint :
      — un nombre maximal d'itérations,
      — ou la convergence vers une solution satisfaisante.

La figure 3.4 décrit le déroulement de l'ACO.

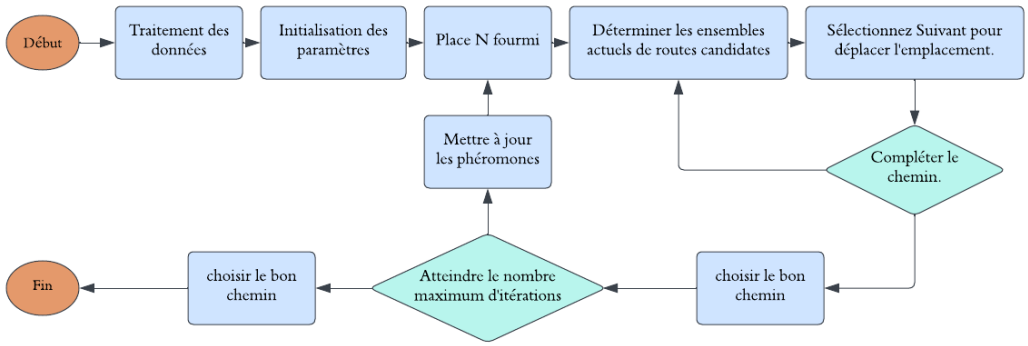


Figure 3.4 – Les étapes de déroulement de ACO

3.1.2.2 Algorithme d'optimization par essaim de particules (PSO) :

L'algorithme *PSO* est une méthode d'optimisation inspirée par le comportement de groupe observé chez des animaux tels que les oiseaux et les poissons. Chaque solution potentielle est modélisée sous forme de particule évoluant dans l'espace de recherche. Ces particules adaptent leur trajectoire selon leur propre expérience et celle des autres particules, dans le but de converger vers une solution optimale (Kumar & Pathak, 2019 ; Shami et al., 2022).

Les paramètres principaux de cet algorithme sont :

Taille de l'essaim (Swarm Size) : Nombre total de particules dans l'algorithme. Une grande taille permet d'explorer plus largement l'espace de recherche, mais augmente les coûts de calcul et le temps d'exécution (Kumar & Pathak, 2019).

Position de la particule (x_i) : Coordonnées représentant la position actuelle d'une particule dans l'espace de recherche. Cette position constitue une solution candidate à évaluer (Kumar & Pathak, 2019).

Vitesse (v_i) : Vecteur qui détermine la direction et la vitesse avec lesquelles une particule se déplace dans l'espace de recherche. Il est mis à jour à chaque itération pour guider l'exploration (Kumar & Pathak, 2019).

Meilleur personnel ($pbest_i$) : Position antérieure qui a donné la meilleure valeur de la fonction objectif pour la particule i. Elle sert de mémoire individuelle pour orienter les futurs déplacements (Kumar & Pathak, 2019).

Meilleur global ($gbest$) : Position globale de l'ensemble des particules qui a fourni la meilleure valeur de la fonction objectif jusqu'à présent. Elle guide collectivement toutes les particules (Kumar & Pathak, 2019).

Poids d'inertie (w) : Coefficient appliqué à la vitesse précédente. Il joue un rôle central dans l'équilibre entre l'exploration de nouvelles zones et l'exploitation des meilleures zones connues (Shami et al., 2022).

Coefficients d'accélération (c_1, c_2) : c_1 représente l'attirance vers le meilleur personnel (composante cognitive), tandis que c_2 mesure l'influence du meilleur global (composante sociale). Leur bonne calibration influence fortement la performance (Shami et al., 2022).

Fonction objectif $f(x)$: Fonction mathématique définie en fonction du problème à résoudre. Elle évalue la qualité d'une solution donnée et oriente l'optimisation (Kumar & Pathak, 2019).

L'algorithme PSO suit les étapes suivantes :

1. **Initialisation** : Positions et vitesses des particules sont générées aléatoirement. Chaque particule évalue sa solution, initialise $pbest_i$, et l'algorithme identifie le $gbest$ initial (Shami et al., 2022).
2. **Mise à jour de la vitesse** :

$$v_i(t+1) = w \cdot v_i(t) + c_1 \cdot r_1 \cdot (pbest_i - x_i(t)) + c_2 \cdot r_2 \cdot (gbest - x_i(t)) \tag{3.1.5}$$

   avec r_1 et r_2 des valeurs aléatoires dans $[0, 1]$ (Kumar & Pathak, 2019 ; Shami et al., 2022).
3. **Mise à jour de la position** :

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad \text{(Kumar \& Pathak, 2019)} \tag{3.1.6}$$

4. **Évaluation** : On réévalue $f(x_i)$. Mise à jour de $pbest_i$ et de $gbest$ si de meilleures solutions sont trouvées (Shami et al., 2022).

5. **Critère d'arrêt** : L'algorithme s'arrête après un nombre d'itérations donné ou si la convergence est atteinte (Shami et al., 2022).

La figure 3.5 décrit le déroulement de l'algorithme de PSO.

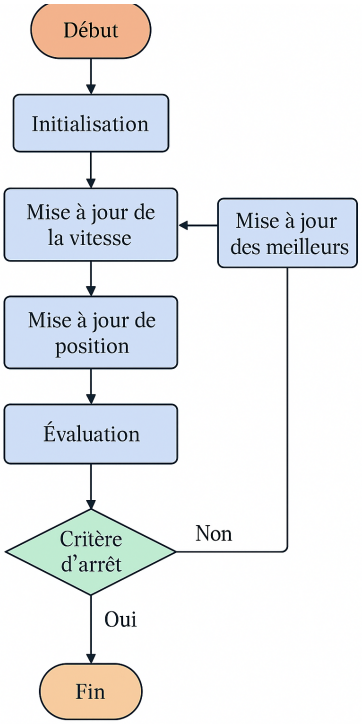


Figure 3.5 – Les étapes de déroulement de PSO

Conclusion

Dans ce chapitre, nous avons introduit les métaheuristiques bio-inspirées en présentant leurs principales familles, notamment les algorithmes évolutionnaires et ceux issus de l'intelligence en essaim. Nous nous sommes particulièrement focalisés sur les algorithmes génétiques, dans leur version classique et multi-objectif, car notre approche d'optimisation pour la conception de circuits quantiques repose directement sur ces méthodes.Cette base nous permettra, dans le chapitre suivant, d'explorer plus en détail comment ces algorithmes peuvent être exploités pour générer des circuits quantiques optimisés.

Chapitre 4

Optimisation de circuits quantiques par les métaheuristiques bio-inspirés

Introduction

Ce chapitre propose une vue d'ensemble des principales approches d'optimisation appliquées à la conception de circuits quantiques. Nous commençons par rappeler les fondements des méthodes évolutionnaires, puis nous présentons les stratégies de réduction de la complexité des circuits en lien avec les contraintes matérielles actuelles. Nous abordons ensuite l'apport des approches inspirées de l'intelligence en essaim, avant de comparer leurs performances avec celles des algorithmes génétiques, en particulier dans un cadre multi-objectifs. L'objectif est de mettre en évidence les tendances générales et les choix méthodologiques les plus adaptés aux architectures NISQ.

4.1 Critères de comparaison

Afin d'assurer une évaluation cohérente des différentes approches présentées, nous avons retenu un ensemble de critères de comparaison communs, adaptés au contexte des circuits quantiques sur architectures NISQ :

— **Fidélité ($\mathcal{F}$)** : mesure de la proximité entre l'unitaire/circuit généré et l'unitaire/circuit cible, exprimée par le chevauchement ou la trace-fidelity.

— **Profondeur du circuit (D)** : nombre de couches séquentielles de portes, directement lié à la robustesse face au bruit et au temps d'exécution.

— **Nombre de portes / coût structurel (Cost)** : total de portes quantiques utilisées, avec pondération particulière pour les portes multi-qubits (ex. CNOT), plus coûteuses.

— **Vitesse de convergence** : rapidité avec laquelle l'algorithme atteint une solution satisfaisante, en nombre d'itérations ou temps de calcul.

— **Robustesse au bruit** : capacité de l'approche à maintenir de bonnes performances dans des environnements bruités simulant des machines NISQ réelles.

— **Scalabilité** : aptitude de la méthode à s'adapter à des circuits de taille croissante (nombre de qubits, profondeur).

— **Facilité de mise en œuvre** : complexité de la configuration (paramètres, opérateurs génétiques, réglages), critère important pour une utilisation pratique.

Ces critères constituent la grille d'analyse utilisée pour comparer les méthodes évolutionnaires classiques, les approches multi-objectifs et les algorithmes d'intelligence en essaim dans les sections suivantes.

4.2 Les algorithmes évolutionnaires pour la conception de circuits quantiques

4.2.1 Rôle des algorithmes évolutionnaires dans la conception des circuits quantiques

L'application des algorithmes évolutionnaires à la conception de circuits quantiques constitue une approche prometteuse pour répondre aux défis liés à la complexité croissante des systèmes quantiques. Ces méthodes permettent non seulement de générer automatiquement des circuits adaptés à des comportements cibles, mais aussi d'optimiser des architectures en maximisant certaines propriétés essentielles telles que l'intrication. À titre d'illustration, l'étude menée par Shailendra Bhandari et ses collaborateurs (Bhandari et al., 2024) met en évidence deux axes principaux d'application des algorithmes évolutionnaires dans ce domaine :

— **Premier axe** : la synthèse de circuits quantiques destinés à reproduire des comportements spécifiques, en particulier ceux d'automates cellulaires déterministes et stochastiques. Les algorithmes évolutionnaires sont utilisés pour assembler automatiquement des circuits dont la performance est évaluée à l'aide de la divergence de Kullback-Leibler, , un outil de la théorie de l'information qui mesure la différence entre deux distributions de probabilité. Plus cette divergence tend vers zéro, plus la distribution des sorties produites par le circuit est fidèle au comportement cible attendu, garantissant ainsi la justesse de la modélisation.

— **Deuxième axe** : l'optimisation de circuits quantiques afin de générer des états fortement intriqués, élément fondamental pour le traitement de l'information quantique. L'évaluation de ce degré d'intrication repose notamment sur deux mesures complémentaires. D'une part, la mesure de Meyer-Wallach fournit une estimation de l'intrication globale dans un système multi-qubits : une valeur nulle indique un état séparable (sans corrélations quantiques), tandis qu'une valeur proche de 1 reflète un état hautement intriqué. D'autre part, l'entropie de von Neumann permet de quantifier l'incertitude ou le désordre d'un sous-système à partir de sa matrice de densité réduite ; une entropie nulle traduit l'absence d'intrication, tandis qu'une valeur positive révèle la présence d'une corrélation quantique non triviale entre les parties du système. En intégrant ces mesures comme fonctions d'évaluation dans le processus évolutionnaire, il devient possible d'orienter automatiquement la recherche vers des circuits quantiques compacts, robustes et hautement performants, adaptés aux exigences des architectures quantiques contemporaines.

4.2.2 Origines de l'application des algorithmes génétiques en circuits quantiques

L'application des algorithmes génétiques (AG) à la conception automatique de circuits quantiques a émergé à la fin des années 1990 et au début des années 2000, à un moment où la complexité croissante des systèmes quantiques rendait difficile la conception manuelle de circuits efficaces. L'une des premières démonstrations concrètes de l'application des algorithmes génétiques à la conception automatique de circuits quantiques est le travail de Yabuki et Iba (2000). Les auteurs y présentent une méthode originale utilisant les algorithmes génétiques (AG) pour la conception automatique de circuits de téléportation

quantique. Leur approche repose sur une structure génétique qui reflète les étapes fonctionnelles du protocole : génération de la paire **EPR** [1], opérations d'Alice, et corrections de Bob. En utilisant uniquement des contraintes élémentaires (type de portes limité, division en étapes bien définies), leur algorithme a été capable de générer un circuit correct contenant seulement huit portes. Ce résultat est notable car il améliore le circuit de référence de Brassard, qui en contenait onze.

La qualité des circuits générés est mesurée à l'aide d'une fonction d'erreur qui compare les amplitudes du qubit téléporté (cf. Eq. 2.2.2). Cette fonction est définie par :

$$\text{error}_j = \frac{1}{n} \sum_{i=0,2,4,6} \left| \frac{a_{j,i}}{a_{j,i+1}} - \frac{p}{q} \right| \tag{4.2.1}$$

où $a_{j,i}$ représente les amplitudes mesurées et $\frac{p}{q}$ le ratio souhaité du qubit téléporté. L'intérêt de cette méthode réside dans le fait qu'il est possible d'évoluer un circuit fonctionnel sans fournir explicitement la matrice unitaire cible. Il suffit de spécifier le comportement attendu à la sortie, ce qui rend l'approche flexible dans les cas où les spécifications sont incomplètes ou complexes à modéliser mathématiquement.

Quelques années plus tard,Lukac et Perkowski (2002) ont ensuite généralisé cette approche à la synthèse de circuits quantiques arbitraires. Leur travail repose sur un AG classique, mais introduit un encodage très efficace pour représenter les circuits sous forme de chromosomes. Chaque circuit est représenté comme une séquence de blocs parallèles contenant des portes quantiques élémentaires (Hadamard, CNOT, Swap, etc.). Cette structure permet d'évaluer rapidement les matrices unitaires correspondantes, réduisant considérablement le temps de calcul.

Dans ce cadre, la méthode vise à reproduire une matrice unitaire cible, c'est-à-dire une matrice unitaire représentant l'opération quantique que le circuit doit approximer. En d'autres termes, la matrice unitaire cible décrit la transformation idéale à appliquer sur un registre de qubits, tandis que l'algorithme cherche à construire un circuit qui en soit la réalisation la plus fidèle possible. La fonction de fitness est donc fondée sur la comparaison élément par élément entre la matrice cible et celle obtenue à partir du chromosome. Elle est donnée par la formule suivante :

$$F = \frac{1}{1 + \Lambda + \sum_{i,j} |U_{ij} - S_{ij}|} \tag{4.2.2}$$

où U est la matrice unitaire cible, S celle du circuit candidat, et Λ un facteur pénalisant les circuits trop longs (en nombre de blocs parallèles). Cette formulation permet d'encourager à la fois la justesse et la compacité du circuit.
Ce travail se distingue par sa généralité : il ne se limite pas à un problème particulier comme la téléportation, mais peut s'appliquer à tout type de circuit quantique, pourvu qu'on en connaisse la matrice unitaire. De plus, les résultats expérimentaux montrent que l'approche converge vers des solutions efficaces en un temps raisonnable, même pour des circuits à trois ou quatre qubits.

4.2.3 Typologie des travaux réalisés

Dans ce qui suit, nous présentons une synthèse de l'état de l'art portant sur les applications des algorithmes évolutionnaires pour la conception automatique de circuits

1. EPR : Einstein–Podolsky–Rosen. Désigne une paire de qubits intriqués, utilisée dans les protocoles de téléportation quantique.

quantiques.

4.2.3.1 Synthèse automatique de circuits :

Giraldi et al. (2004) proposent une étude approfondie sur l'application des algorithmes génétiques (AG) au calcul quantique, tout en ouvrant des perspectives vers les algorithmes génétiques quantiques (AGQs). Leur travail se distingue par deux contributions principales.

D'une part, ils introduisent une méthode originale d'apprentissage d'opérateurs unitaires à partir d'exemples d'entrée-sortie, sans exiger que les vecteurs d'entraînement soient orthogonaux. L'opérateur unitaire A à apprendre est optimisé de sorte que, pour chaque paire d'entraînement $(|\chi_i\rangle, |\psi_i\rangle)$, on minimise l'erreur de transformation. La fonction de fitness est définie comme la somme des normes ℓ_1 des différences entre les sorties produites et les sorties cibles :

$$\mathcal{F}(A) = \sum_{i=1}^{N} \|A|\chi_i\rangle - |\psi_i\rangle\|_1 \tag{4.2.3}$$

où N est le nombre d'exemples, et la norme ℓ_1 est utilisée pour quantifier l'écart entre la sortie produite par l'opérateur candidat A et l'état cible. Ce cadre permet une optimisation directe de matrices continues, et les mutations sont modélisées comme de légères perturbations locales appliquées aux coefficients de A.

D'autre part, ils formalisent un modèle théorique de AGQ, où les individus sont représentés par des superpositions quantiques, intriqués avec leurs valeurs de fitness, et où la sélection naturelle est simulée par un processus de mesure quantique. Cette approche introduit une dynamique évolutive fondamentalement non classique, où la diversité génétique et la convergence sont directement influencées par les lois de la mécanique quantique, notamment à travers la mesure.

Enfin, les auteurs critiquent les représentations classiques fixes des circuits (par exemple sous forme d'arbre ou de liste), et suggèrent d'explorer des structures plus flexibles, notamment celles fondées sur la théorie des groupes abéliens finis. Cette proposition vise à mieux capturer la richesse combinatoire de l'espace des circuits quantiques. L'ensemble de ces contributions étend significativement les capacités des algorithmes évolutionnaires traditionnels, en ouvrant la voie vers des modèles plus puissants et véritablement quantiques d'optimisation évolutionnaire.

Tandeitnik et Guerreiro (2022) proposent d'utiliser des algorithmes génétiques AG pour la conception automatique de circuits quantiques, en particulier pour l'évolution de circuits correcteurs d'erreurs stabilisateurs (QECC). Leur méthode repose sur plusieurs innovations fortes.

Tout d'abord, les circuits sont représentés génétiquement sous forme de matrices de portes Clifford [2], ce qui permet une simulation efficace et évite le recours à des approximations coûteuses. Chaque circuit candidat est ainsi un génotype structuré, facilitant la manipulation par les opérateurs génétiques standards (crossover, mutation).

La fonction de fitness qu'ils introduisent combine deux critères fondamentaux : la capacité du circuit à corriger les erreurs, mesurée par un degré de corrigibilité C, et la profondeur D du circuit. La fonction de fitness est donnée par :

$$F = w \times C - D \tag{4.2.4}$$

2. **Clifford** : Ensemble de portes quantiques qui stabilisent les états de type Pauli. Il comprend notamment les portes Hadamard (H), Phase (S), et CNOT.

où w est un poids élevé (par exemple, $w = 1000$) favorisant prioritairement la capacité de correction sur la compacité. Ce choix guide efficacement l'évolution vers des circuits à la fois performants et utilisables dans des architectures réelles.

Ils adaptent aussi la fonction de fitness pour orienter l'évolution vers des codes spécifiques, comme les codes topologiques (Calderbank-Shor-Steane (CSS)), en introduisant un terme correctif supplémentaire valorisant les stabilisateurs particuliers recherchés.

Une évaluation systématique est menée en comparant leur approche à une recherche aléatoire (Random Search (RS)). Les résultats montrent que le AG converge beaucoup plus rapidement et efficacement vers des circuits performants, et que cet avantage s'amplifie avec l'augmentation du nombre de qubits.

Ces éléments font de leur contribution un jalon important dans la conception automatique de circuits quantiques adaptés aux contraintes pratiques du calcul quantique bruyant.

4.2.3.2 Optimisation structurelle des circuits :

Krylov et Lukac (2018) proposent le Quantum Encoded Quantum Evolutionary Algorithm (QEQEA) , une approche hybride visant à la synthèse de circuits quantiques à partir d'une population de portes codées en qubits et qutrits. Contrairement aux algorithmes génétiques classiques AG qui manipulent des circuits entiers, leur méthode repose sur l'évolution indépendante de segments de circuit, chacun correspondant à une porte élémentaire codée quantiquement.

Chaque segment est représenté par deux registres quantiques : un qubit encodant l'angle de rotation θ (via ses amplitudes complexes α, β) et un qutrit spécifiant l'axe de rotation $r \in \{x, y, z\}$. Lors de la construction du circuit, une séquence de mesures répétées sur ces registres permet de générer des portes unitaires appliquées successivement sur les qubits cibles.

La mutation est réalisée via des opérateurs de rotation de type $SU(3)$ [3] sur l'état du qutrit, modifiant ainsi dynamiquement l'axe de la future rotation unitaire. Elle est proportionnelle à l'erreur actuelle du segment, rendant la mutation *adaptive* selon la performance.

Le groupe $SU(3)$, ou groupe unitaire spécial de degré 3, désigne l'ensemble des matrices unitaires 3×3 de déterminant 1. Il représente mathématiquement l'ensemble des transformations unitaires possibles appliquées à un qutrit, c'est-à-dire à un système quantique à trois niveaux. En utilisant des opérations dans $SU(3)$, l'algorithme assure que les mutations appliquées à l'état du qutrit respectent les contraintes physiques de l'évolution quantique, tout en explorant l'espace des rotations possibles sur les axes x, y ou z.

L'évaluation des circuits repose sur une fonction de fitness exprimée en fonction de la fidélité entre la matrice unitaire cible T et la matrice synthétisée S :

$$\mathcal{F}(S) = 1 - \sqrt{\frac{2^n - |\mathrm{Tr}(S^\dagger T)|}{2^n}} \tag{4.2.5}$$

où n est le nombre de qubits du circuit, et Tr désigne la trace matricielle. Cette expression mesure l'écart unitaire entre la solution trouvée et l'objectif, en le normalisant par la dimension de l'espace de Hilbert.

3. **SU(3)** : Groupe unitaire spécial de degré 3, représentant l'ensemble des matrices unitaires 3×3 de déterminant 1, utilisé pour les rotations dans un espace à trois niveaux (qutrits).

Bien que les performances de QEQEA soient légèrement inférieures à celles d'un AG classique optimisé sur Graphics Processing Unit (GPU), l'intérêt majeur de cette approche réside dans sa compatibilité avec des architectures quantiques physiques. Elle ouvre la voie à des modèles d'évolution partiellement exécutables sur ordinateur quantique, notamment dans les paradigmes de type RMN ou calcul optique.

Davis et al. (2020) introduisent une approche pragmatique et efficace de synthèse de circuits quantiques, conçue pour produire des circuits de faible profondeur en minimisant le nombre de portes CNOT, tout en tenant compte des contraintes topologiques des architectures NISQ. L'algorithme repose sur une stratégie de recherche inspirée de A-star Search Algorithm (A*), combinée à une optimisation numérique des paramètres des portes.

L'objectif est de minimiser une fonction de distance unitaire entre l'unitaire cible U et l'unitaire synthétisé U_S :

$$\min_{U_S} D(U, U_S) \quad \text{où} \quad D(U, U_S) = 1 - \frac{|\mathrm{Tr}(U^\dagger U_S)|}{2^n} \tag{4.2.6}$$

La particularité de cette méthode est d'intégrer directement la topologie physique de la puce dans le processus de synthèse, qu'il s'agisse de topologies linéaires ou en « bowtie ». Cela permet d'éviter le recours à des portes SWAP coûteuses après coup, en produisant des circuits directement exécutables sur le matériel cible.

La recherche dans l'espace des circuits est guidée par une heuristique de type A*, dans laquelle chaque nœud est évalué selon une fonction de coût $f(n) = g(n) + h(n)$, où $g(n)$ représente le nombre de portes CNOT et $h(n)$ correspond à une pénalité proportionnelle à la distance unitaire résiduelle, pondérée par un facteur empirique a.

Une fois la structure du circuit déterminée, les paramètres continus des portes de type `U3` sont optimisés numériquement à l'aide d'algorithmes tels que Broyden–Fletcher–Goldfarb–Shanno algorithm (BFGS) ou Constrained Optimization BY Linear Approximations (COBYLA), afin de minimiser finement la distance à l'unitaire cible.

L'approche est hautement configurable : elle prend en charge différents types de topologies (comme IBM Q5 ou QNL8QR-V5) et différents ensembles de portes natives (Rx, Rz, CNOT). Les résultats empiriques sont particulièrement convaincants, avec une réduction allant jusqu'à 5.3× du nombre de portes CNOT par rapport à UniversalQ, une diminution de 36 % de la longueur des circuits pour des tâches de préparation d'état face à Qiskit, et la génération de circuits optimaux pour des portes classiques comme Toffoli ou Fredkin.

Miranda et al. (2023) proposent une approche innovante pour la conception automatique de circuits quantiques en utilisant un algorithme génétique AG basé sur le modèle en îles (Island Genetic Algorithm). Leur principale contribution est d'accroître la diversité génétique au sein de la population en faisant évoluer plusieurs sous-populations $\{P_1, P_2, \ldots, P_k\}$ de manière indépendante avant d'autoriser des migrations contrôlées, ce qui réduit le risque de convergence prématurée vers des optima locaux.

Leur méthode vise la décomposition directe d'une matrice unitaire cible U_t sans recourir à des qubits auxiliaires, en générant un circuit candidat U_a composé d'une séquence de blocs unitaires $\{B_1, B_2, \ldots, B_m\}$, chacun représentant un ensemble de portes quantiques.

La qualité d'un circuit est évaluée à l'aide de la **fidelité de trace**, définie par :

$$f(U_a, U_t) = \frac{1}{2^n} \left| \mathrm{Tr}\left(U_a U_t^\dagger\right) \right| \tag{4.2.7}$$

où n est le nombre de qubits, et Tr désigne la trace matricielle. L'erreur associée est donnée par :

$$\varepsilon = 1 - f(U_a, U_t)^2 \tag{4.2.8}$$

et l'optimisation vise à minimiser ε jusqu'à atteindre un seuil d'arrêt fixé à 10^{-6}.

Les opérations de croisement et de mutation sont adaptées à la structure modulaire des circuits : les blocs sont échangés ou modifiés selon leur nature (contrôlé, non-contrôlé ou auxiliaire). Comme illustré dans la figure 4.1, l'algorithme repose sur une évolution parallèle de plusieurs sous-populations indépendantes, suivie de migrations contrôlées entre îles, ce qui permet d'augmenter la diversité génétique tout en évitant la convergence prématurée vers des optima locaux.Cette approche structurée permet de limiter la dégradation locale du circuit tout en favorisant une exploration efficace de l'espace des solutions.

L'efficacité de leur méthode est démontrée par la synthèse automatique des portes complexes telles que la porte Toffoli, la porte Fredkin, et l'opérateur Hadamard Coin, obtenant des circuits de haute fidélité compatibles avec les contraintes des dispositifs quantiques de type NISQ. La figure 4.1 illustre le schéma de déroulement de l'algorithme génétique basé sur le modèle en îles.

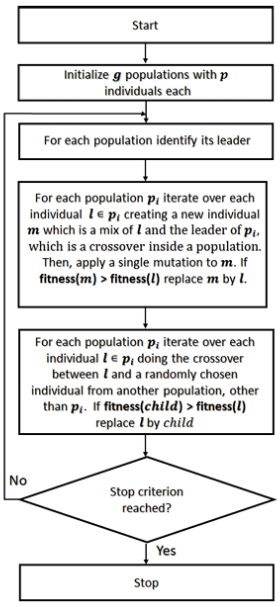


FIGURE 4.1 – Schéma de déroulement de l'algorithme génétique basé sur le modèle en îles (MIRANDA et al., 2023).

Creevey et al. (2023) proposent Genetic Algorithm for State Preparation (GASP), une approche évolutionnaire conçue pour générer des circuits quantiques capables de préparer un état cible $|\psi_{\text{target}}\rangle$ avec une haute fidélité. Cette méthode vise à produire des circuits courts et efficaces, mieux adaptés aux contraintes des dispositifs NISQ, en comparaison avec les synthèses exactes comme celle de Qiskit , basée sur la méthode(**SBM** (Shende, Bullock et Markov) (SHENDE et al., 2006)).

Chaque individu dans GASP est un circuit quantique, représenté comme une séquence de gènes de la forme $[q_t, G, q_c, \theta]$, où $G \in \{\text{Rx}, \text{Ry}, \text{Rz}, \text{CNOT}\}$ et θ est un angle de rotation. L'évaluation de la qualité d'un circuit repose sur la fidélité de l'état généré $|\psi(\vec{\theta})\rangle$ avec l'état cible :

$$\mathcal{F}\left(|\psi(\vec{\theta})\rangle\right) = \left|\langle\psi_{\text{target}}|\psi(\vec{\theta})\rangle\right|^2 \tag{4.2.9}$$

GASP combine une sélection par roue de roulette, un croisement à un point, une mutation (5%) et une optimisation classique des angles par Sequential Least Squares Programming (SLSQP). En cas de stagnation, le nombre de gènes est augmenté pour accroître la complexité du circuit.

Les résultats expérimentaux montrent que GASP produit des circuits de bien moindre profondeur que ceux générés par Qiskit, tout en maintenant la fidélité cible, notamment pour les états Gaussian et W. Cette réduction de la profondeur rend les circuits générés plus robustes au bruit, ce qui est essentiel dans le contexte des architectures NISQ.

4.2.3.3 Avancées vers l'optimisation multi-objectifs :

Potoček et al. (2018) introduisent un cadre d'optimisation évolutionnaire multi-objectifs pour la découverte automatique de circuits quantiques à partir d'exemples d'entrée-sortie (**PotocÌŇek2018**). Leur algorithme génétique est capable de retrouver non seulement des circuits canoniques comme la QFT ou Grover, mais également des variantes plus compactes ou approximatives, optimisées selon différents critères d'implémentation.

L'objectif général consiste à automatiser la découverte de circuits quantiques corrects à partir de spécifications fonctionnelles (I/O), sans hypothèse sur leur structure. Le modèle d'optimisation utilisé repose sur un vecteur d'objectifs multi-critères :

$$\min_{C_{\text{cand}}} [E_{\text{avg}},\ E_{\text{max}},\ \text{num gates},\ \text{num.oracle},\ \text{type-specific counts}] \tag{4.2.10}$$

où E_{avg} est l'erreur moyenne mesurée sur un ensemble d'états d'entrée, et E_{max} l'erreur maximale sur les pires cas testés. Le nombre total de portes (num gates) est utilisé comme indicateur direct de complexité structurelle, impactant la profondeur du circuit, sa robustesse au bruit et son exécution sur matériel NISQ. Les deux autres critères permettent de contrôler respectivement l'utilisation de portes spécifiques coûteuses (comme les oracles), ainsi que le profil de distribution des portes selon leur type (par exemple CNOT, Toffoli ou Rx).

Ce cadre permet à l'algorithme d'explorer des compromis pertinents en générant un *front de Pareto* de solutions, où chaque circuit représente un équilibre entre fidélité, taille et coût physique. Cela s'avère particulièrement pertinent dans les architectures bruitées, où l'optimisation structurelle est souvent plus déterminante que la précision absolue.

L'erreur de sortie est calculée différemment selon le contexte. Pour les circuits de type Grover, elle est définie par :

$$E = 1 - |\langle \psi_{\text{target}} | \psi_{\text{output}} \rangle|^2 \tag{4.2.11}$$

tandis que pour la QFT, elle est évaluée comme une erreur moyenne sur tous les vecteurs :

$$E = 1 - \frac{1}{N} \sum_i |\langle \psi_i | \chi_i \rangle| \tag{4.2.12}$$

Plusieurs contributions marquent l'originalité de leur approche. Premièrement, la découverte des circuits se fait sans connaissance a priori de leur structure, uniquement à partir d'exemples d'entrée-sortie. Deuxièmement, la recherche multi-objectifs permet de générer des variantes adaptées à différentes contraintes d'implémentation, notamment sur des architectures NISQ. L'algorithme utilise une représentation génétique souple, où les individus sont des séquences de portes de longueur variable. Il combine des mutations discrètes (permutation de qubits, suppression/insertion de portes) et continues (ajustement

d'angles), ainsi que des opérations avancées comme l'insertion de séquences inverses ou la fusion automatique de portes adjacentes.

Enfin, les résultats empiriques montrent que l'approche permet de générer des circuits pour la QFT avec moins de 10 portes et une erreur inférieure à 0.001, ainsi que des versions optimisées de l'algorithme de Grover plus performantes que l'originale dans certains cas. Des circuits originaux à 4 qubits sont également découverts par une stratégie de décomposition modulaire.

Cette méthode illustre le potentiel des algorithmes évolutionnaires pour explorer des structures inédites dans l'espace des circuits quantiques. Elle est aussi proposée comme outil de *découverte*, capable de générer des circuits corrects dans des situations où la solution analytique n'est pas connue.

Rindell et al. (2023) présentent une méthode d'optimisation évolutionnaire pour la préparation approximative d'états quantiques sur des dispositifs NISQ, en tirant parti d'un algorithme génétique multi-objectifs (Rindell et al., 2023). Leur approche vise à générer des circuits quantiques courts qui, bien qu'approximatifs, offrent une meilleure fidélité finale que les circuits exacts lorsque le bruit physique est pris en compte.

Le processus d'optimisation repose sur deux objectifs antagonistes. Le premier est la maximisation de la fidélité entre l'état cible $|\psi_{\text{target}}\rangle$ et l'état préparé $|\psi_{\text{output}}\rangle$, mesurée par :

$$\mathcal{F} = |\langle\psi_{\text{target}}|\psi_{\text{output}}\rangle|^2 \tag{4.2.13}$$

Le second objectif est la minimisation du coût structurel du circuit, défini par :

$$\text{cost} = N_1 + 10 \times N_2 \tag{4.2.14}$$

où N_1 est le nombre de portes à un qubit, et N_2 le nombre de portes CNOT, plus coûteuses et sensibles au bruit.

Pour résoudre ce compromis, les auteurs utilisent l'algorithme NSGA-II, bien adapté à l'optimisation multi-objectifs dans des espaces de solutions complexes. L'évaluation des circuits intègre un modèle réaliste de bruit correspondant à l'architecture IBM `FakeVigo`, simulant les conditions d'exécution sur des processeurs quantiques actuels.

Une contribution théorique importante du travail est la mise en évidence du compromis optimal entre la fidélité théorique $\mathcal{F}_0$ et la robustesse au bruit. Ils montrent que la fidélité réelle $\mathcal{F}_{\text{noisy}}$ d'un circuit profond l s'exprime approximativement par :

$$\mathcal{F}_{\text{noisy}} \approx (1-p)^l \cdot \mathcal{F}_0 = (1-p)^l(1-\varepsilon^2) \tag{4.2.15}$$

où p représente le taux de bruit par porte, et ε l'erreur d'approximation de l'état cible. Ce modèle met en évidence que, dans un environnement bruité, un circuit plus court et légèrement approximatif ($\varepsilon > 0$) peut surpasser en performance pratique un circuit exact mais plus long ($\varepsilon = 0$).

Sur le plan expérimental, la méthode proposée surpasse la méthode Low-Rank State Preparation (LRSP), améliorant la fidélité effective jusqu'à +38 % dans certains cas. De plus, l'approche est capable d'adapter automatiquement la structure des circuits aux contraintes topologiques réelles du matériel cible.

En résumé, ce travail démontre que dans les environnements quantiques bruités, la recherche d'une précision structurelle maximale est moins importante que l'optimisation globale prenant en compte les effets physiques réels. Il ouvre la voie à une génération de circuits spécifiquement conçus pour la robustesse plutôt que pour l'exactitude abstraite.

Ding et Spector (2023) introduisent Multi-Objective Evolutionary Architecture Search for Parameterized Quantum Circuits (MEAS-PQC), un cadre d'optimisation évolutionnaire pour la conception automatique de circuits quantiques paramétrés (Parameterized Quantum Circuit (PQC)) (Ding & Spector, 2023). Ce cadre repose sur un algorithme génétique multi-objectifs de type NSGA-II, spécifiquement adapté aux particularités du calcul quantique.

L'objectif d'optimisation est triple : maximiser la performance moyenne $R(C)$ de l'agent contrôlé par le circuit C, tout en minimisant simultanément l'estimation du bruit $\Delta(C)$ et la taille du modèle $N(C)$. Formellement, le problème est formulé comme suit :

$$\min_C \left[-R(C),\ \Delta(C),\ N(C)\right] \tag{4.2.16}$$

Chaque circuit candidat est représenté par une séquence de blocs élémentaires x_i, parmi lesquels x_1 désigne les rotations paramétrées (R_x, R_y, R_z), x_2 représente les opérations d'intrication par portes CZ , x_3 encode les données d'entrée via une activation de type $\sigma(\lambda d)$, tandis que x_0 correspond aux mesures projectives.

Pour favoriser l'exploration des architectures de longueur variable, l'algorithme utilise une mutation appelée Uniform Mutation by Addition and Deletion (UMAD) (Uniform Mutation by Addition and Deletion), définie par des taux :

$$r_{\text{del}} = \frac{1}{\ell}, \quad r_{\text{add}} = \frac{1}{\ell + 1} \tag{4.2.17}$$

où ℓ est la longueur actuelle du circuit.

Des simplifications sont appliquées pour éliminer les redondances structurelles, en exploitant des identités telles que :

$$R_x(\theta_1) \cdot R_x(\theta_2) = R_x(\theta_1 + \theta_2), \quad \text{et} \quad CZ \cdot CZ = I \tag{4.2.18}$$

ce qui permet de réduire efficacement la complexité des circuits générés.

L'évaluation des architectures est effectuée dans un contexte d'apprentissage par renforcement (Reinforcement Learning (RL)), sur plusieurs environnements standards d'OpenAI , tels que `CartPole`, `MountainCar` et `Acrobot`. L'optimisation des paramètres continus θ des circuits est réalisée par descente de gradient classique, suivant la mise à jour :

$$\theta \leftarrow \theta - \eta \nabla_\theta L \tag{4.2.19}$$

où L est la fonction de perte quantifiant la performance de l'agent.

Sur le plan expérimental, MEAS-PQC surpasse nettement les approches antérieures comme Evolutionary Quantum Architecture Search for Parameterized Quantum Circuits (EQAS-PQC) et Softmax-Based Architecture Search for Parameterized Quantum Circuits (SOFTMAX-PQC) en termes de rapidité d'apprentissage, de robustesse au bruit et d'efficacité des architectures produites. Le cadre multi-objectif permet en particulier de réduire simultanément le bruit et la taille du modèle sans compromettre les performances. L'analyse des circuits générés révèle que certaines structures récurrentes, telles que la succession d'un bloc x_1 suivi d'un bloc x_2, sont particulièrement associées aux meilleures performances obtenues.

Rahman et al. (2025) proposent une approche novatrice pour la synthèse automatique de circuits quantiques en utilisant un algorithme génétique multi-objectifs assisté par

logique floue (Rahman et al., 2025). Leur méthode vise à générer des circuits compatibles avec les contraintes matérielles des dispositifs NISQ, tout en optimisant simultanément trois critères essentiels : la fidélité de la transformation, la profondeur du circuit, et le nombre de portes multi-qubits.

Le problème est formalisé comme une optimisation à trois objectifs, définie par la fonction :

$$F(C_{\text{syn}}) = [E(U_{\text{in}}, C_{\text{syn}}),\ D(C_{\text{syn}}),\ G_2(C_{\text{syn}})] \tag{4.2.20}$$

où $E(U_{\text{in}}, C_{\text{syn}})$ mesure l'erreur d'approximation du circuit synthétisé C_{syn} par rapport à l'unitaire cible U_{in}, à l'aide de la distance de Hilbert-Schmidt :

$$E = 1 - \frac{\left|\text{Tr}(U_{\text{in}}^{\dagger} U_{\text{syn}})\right|}{2^n} \tag{4.2.21}$$

Le terme $D(C_{\text{syn}})$ représente la profondeur du circuit, et $G_2(C_{\text{syn}})$ quantifie le nombre de portes multi-qubits utilisées, telles que les portes `CNOT`.

L'algorithme utilisé repose sur une version enrichie du NSGA-II, intégrant un système de logique floue pour ajuster dynamiquement le taux de mutation. Deux variables d'entrée sont surveillées : le taux d'amélioration observé sur les dernières générations et la valeur actuelle du taux de mutation. À partir de ces observations, des règles floues comme *« Si amélioration faible ET taux de mutation élevé, alors réduire mutation »* permettent d'adapter de manière intelligente la stratégie évolutive en fonction de la dynamique de l'optimisation.

Plusieurs contributions techniques renforcent l'efficacité de la méthode. L'encodage des circuits repose sur des séquences modulaires de couches parallèles, avec une optimisation automatique du mapping entrée-sortie des qubits. Les opérateurs génétiques sont spécialisés pour manipuler ces séquences : ajout ou suppression de couches, permutation de portes, et ajustement des qubits ciblés. Une fusion automatique des portes redondantes est également mise en œuvre, en exploitant des identités telles que :

$$R_z(\theta_1) \cdot R_z(\theta_2) = R_z(\theta_1 + \theta_2), \quad X \cdot X = I,$$

afin de réduire la taille effective des circuits.

Enfin, les paramètres continus des portes de rotation θ sont optimisés localement en utilisant la méthode BFGS, en résolvant :

$$\theta^{\star} = \arg\min_{\theta}\ E(U_{\text{in}}, C_{\text{syn}}(\theta)) \tag{4.2.22}$$

pour minimiser l'erreur finale du circuit.

Cette approche hybride améliore à la fois la diversité de la population explorée et la qualité de convergence des solutions trouvées, tout en respectant strictement les contraintes topologiques et les restrictions sur les ensembles de portes imposées par les architectures NISQ. Elle constitue ainsi une avancée significative vers une compilation quantique robuste et adaptée au matériel réel.

4.2.3.4 Développement de frameworks spécialisés :

Wu et al. (2020) présentent *QGo*, un cadre de compilation hiérarchique pour l'optimisation de circuits quantiques reposant sur la synthèse automatisée de blocs. Dans le contexte des dispositifs NISQ, où les portes à deux qubits (comme les CNOT) sont plus

bruyantes que les portes à un seul qubit, il est essentiel de minimiser leur nombre pour améliorer la fidélité du circuit.

Le principal défi de la synthèse quantique réside dans sa complexité exponentielle en fonction du nombre de qubits (la taille de la matrice unitaire étant $2^n \times 2^n$). Pour rendre la synthèse applicable à de grands circuits, les auteurs introduisent une méthode bloc par bloc, où le circuit est partitionné en sous-circuits de petite taille (3 ou 4 qubits), optimisés individuellement par un moteur de synthèse, puis recomposés.

Le cadre QGo repose sur une approche hiérarchique en quatre étapes pour optimiser les circuits quantiques tout en tenant compte de la connectivité matérielle des dispositifs NISQ :

— **Mapping physique** : QGo commence par mapper les qubits logiques sur le matériel cible à l'aide du compilateur `t|ket⟩`, en insérant les portes SWAP nécessaires dès le départ pour éviter de les ajouter après coup.

— **Partitionnement du circuit** : le circuit est représenté sous forme d'un graphe de dépendances. Il est ensuite découpé en blocs indépendants de 3 à 4 qubits, en maximisant localement le nombre de portes CNOT, tout en respectant la connectivité physique.

— **Synthèse des blocs** : chaque bloc est optimisé indépendamment via une méthode de synthèse numérique qui minimise les CNOT tout en gardant une approximation précise de l'unitaire d'origine.

— **Recomposition du circuit** : les blocs optimisés sont recombinés. Si la version synthétisée d'un bloc n'est pas meilleure, l'original est conservé. Une dernière passe fusionne les portes monobit adjacentes.

La figure 4.2 présente l'approche hiérarchique du cadre QGo

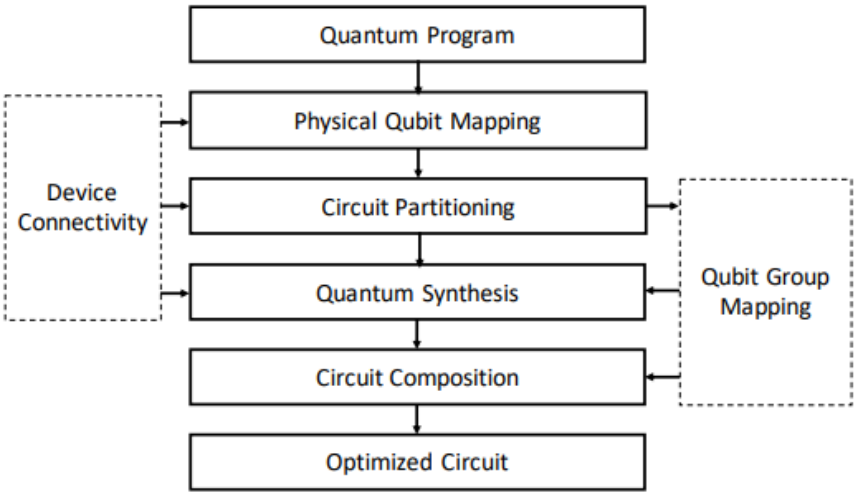
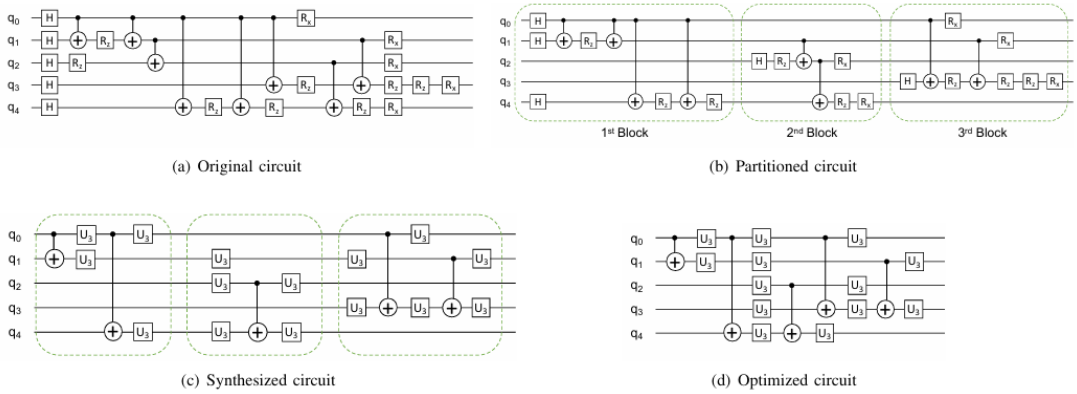


FIGURE 4.2 – Le principe de fonctionnement de (QGo).

Des expériences menées sur trois catégories de circuits (petits, moyens et grands) montrent que QGo permet de réduire en moyenne de 29,9 % le nombre de CNOTs par rapport à `t|ket⟩`, avec des pics allant jusqu'à 50 %. Les circuits optimisés par QGo

démontrent une amélioration de la fidélité mesurée sur l'IBM Athens, corrélée à la réduction de CNOT (coefficient de 78 %). Une simulation bruitée confirme cette tendance avec une corrélation de 76 % pour les circuits moyens. L'analyse de sensibilité indique que plus l'erreur des portes est élevée, plus l'impact de QGo sur la fidélité est significatif.
La figure 4.3 suivante, illustre un exemple d'optimisation d'un circuit quantique à 5 qubits en utilisant l'outil QGo. Le circuit initial est tout d'abord décomposé en trois blocs, chacun contenant uniquement des portes opérant sur un sous-ensemble de trois qubits. Par exemple, le premier bloc agit sur les qubits q0, q1 et q4 ; le second sur q1, q2 et q4 ; et le troisième sur q0, q1 et q3. Cette partition respecte rigoureusement les dépendances de portes du circuit d'origine. Ensuite, une synthèse quantique est effectuée indépendamment sur chaque bloc, ce qui permet de réduire le nombre de portes CNOT dans les deux premiers blocs, tandis que le dernier bloc conserve deux portes CNOT. Finalement, les portes à un seul qubit sont combinées lorsque cela est possible, menant à un circuit optimisé, fonctionnellement équivalent au circuit de départ, mais avec une complexité réduite en termes de portes CNOT.



(a) Original circuit (b) Partitioned circuit (c) Synthesized circuit (d) Optimized circuit

FIGURE 4.3 – Un exemple d'optimisation de circuit quantique avec (QGo).

Enfin, l'optimisation de circuits de grande taille (60+ qubits) confirme la **scalabilité** du cadre, avec un gain moyen de 28,9 % en CNOTs pour QGo-4 (blocs de 4 qubits), au prix d'un temps de compilation plus élevé. L'approche proposée ouvre ainsi une voie vers une intégration efficace de la synthèse dans les compilateurs quantiques industriels, avec des bénéfices concrets pour les dispositifs NISQ actuels.

Sünkel et al. (2023) proposent *GA4QCO*, un cadre modulaire et configurable permettant de générer automatiquement des circuits quantiques adaptés aux contraintes des dispositifs NISQ, via un algorithme génétique. Ce cadre repose sur l'optimisation de circuits en fonction d'objectifs définis par l'utilisateur à travers une fonction de fitness personnalisable.
Dans ce framework, Chaque individu (chromosomes) appartenant à la population représente un circuit quantique et est encodé sous forme d'une liste bidimensionnelle de taille $n \times m$:

— n : nombre de qubits (lignes) ;

— m : profondeur du circuit (colonnes).

Chaque cellule contient une porte quantique décrite par ses attributs : nom (e.g., R_Y, $CNOT$), qubits affectés, qubits de contrôle, qubits cibles et paramètres (par exemple, un angle θ pour les portes rotationnelles).

Ce modèle repose sur un processus évolutif inspiré des algorithmes génétiques classiques :

1. **Initialisation** : génération aléatoire d'une population d'individus (circuits quantiques).
2. **Évaluation** : chaque individu est évalué selon une fonction de fitness, par exemple :

$$F = |\langle \psi_{\text{target}} | \psi_{\text{output}} \rangle|^2 \tag{4.2.23}$$

   pour mesurer la fidélité à un état quantique cible.
3. **Sélection** : individus sélectionnés pour reproduction selon plusieurs stratégies : aléatoire, tournoi, ou roue de la fortune.
4. **Croisement** : génération d'enfants via différentes méthodes (single-point, multi-point, blockwise), avec gestion automatique des circuits de tailles différentes.
5. **Mutation** : application de modifications aléatoires sur les circuits (flip de porte, permutation de colonnes, ajustement de paramètres, ajout/suppression de qubits/gates).
6. **Survivor selection** : sélection de la prochaine génération.

Ce cycle est répété pendant un nombre fixe de générations. Le processus est contrôlé par un fichier de configuration dans lequel l'utilisateur spécifie les paramètres du modèle (taille de population, taux de mutation, profondeur maximale, ensemble de portes autorisées, etc.).

La figure 4.4 décrit le processus d'optimisation fait par GA4QCO.

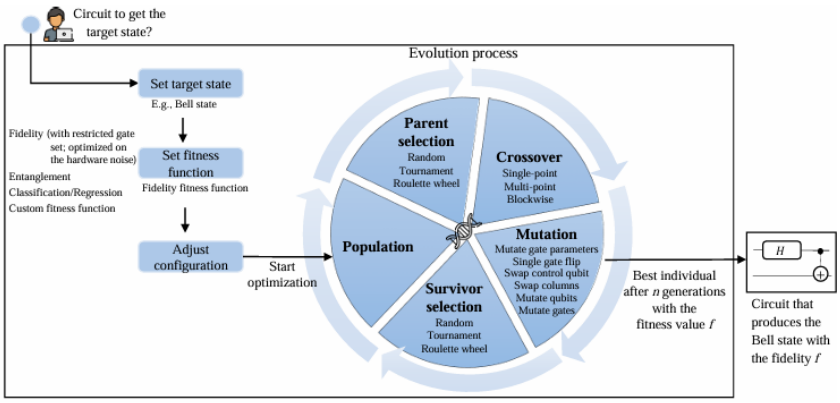


FIGURE 4.4 – Optimisation des circuits quantiques dans le cadre GA4QCO.

Les objectifs d'optimisation dans GA4QCO sont définis à travers une fonction de fitness que l'utilisateur peut personnaliser. Parmi les fonctions proposées, on trouve notamment la fidélité à un état cible $|\psi_{\text{target}}\rangle$, mesurée par le chevauchement avec l'état généré. Le cadre permet également de rechercher des états fortement intriqués, l'intrication étant quantifiée par l'entropie de von Neumann. Un autre objectif est la précision obtenue par un modèle quantique de classification ou de régression, entraîné au sein du processus évolutif. Enfin, GA4QCO offre la possibilité d'intégrer des fonctions de fitness entièrement personnalisées, ce qui élargit son champ d'application à de nombreux cas d'usage spécifiques.

Résultats expérimentaux Les auteurs appliquent GA4QCO à une tâche de préparation d'état sur 4 qubits avec une profondeur de 20. Deux configurations sont comparées :

— **Sans contrainte de portes** : la fidélité moyenne augmente graduellement et atteint ≈ 0.8 après 1000 générations.

— **Avec contrainte** (portes limitées à `{id, rz, sx, x, cx}`) : meilleure fidélité obtenue de 0.994, contre une moyenne de 0.29 pour une génération aléatoire.

Ces résultats montrent que GA4QCO est capable de produire des circuits courts, performants, et compatibles avec les restrictions physiques imposées par les dispositifs NISQ.

4.2.4 Analyse comparative des méthodes étudiées

Les tableaux suivants 4.1 et 4.2 récapitulent les travaux réalisés concernant la conception et l'optimisation de circuits quantiques à l'aide d'algorithmes génétiques, qu'ils soient classiques ou à objectifs multiples :

Référence	**Objectifs optimisés**	**Fonction de fitness / Distance**	**Représentation du chromosome**	**Méthode / Spécificités**	**Spécificités techniques**	**Innovation / Contribution clé**
Potoček et al. (2018)	$E_{\text{avg}}, E_{\max}$, gates, oracle,types	Fidélité moyenne et maximale ; coût structurel	Séquence de portes à longueur variable	AG multi-objectifs	Mutations discrètes et continues ; fusions automatiques	Découverte automatique de circuits canoniques (QFT, Grover) à partir d'E/S
Wu et al. (2020)	Nombre de CNOTs, fidélité, bruit	Corrélation réduction CNOT / fidélité ; simulation bruitée	Ensemble de sous-circuits (blocs de 3–4 qubits)	Compilation hiérarchique (QGo)	Découpage du circuit en blocs + recomposition	Synthèse évolutive scalable avec réduction CNOT jusqu'à 50 %
Rindell et al. (2023)	Fidélité, coût pondéré ($N_1 + 10N_2$)	Fidélité sous bruit réaliste IBM Fake-Vigo	Liste séquentielle de portes avec types et angles	NSGA-II	Évaluation bruitée + front de Pareto	Compromis fidélité théorique vs robustesse matérielle
Ding et Spector (2023)	$-R(C), \Delta(C), N(C)$	Récompense RL, bruit, taille du modèle	Chaîne de blocs x_i (rotation, entanglement, encodage, mesure)	NSGA-II + UMAD	RL dans environnements standards d'OpenAI ; descente de gradient	Architecture PQC multitâche robuste au bruit
Rahman et al. (2025)	Erreur E, profondeur D, nb CNOT G_2	Distance de Hilbert-Schmidt	Couches parallèles avec paramètres θ et qubits mappés	NSGA-II + logique floue	Mutation adaptative ; fusion de portes ; BFGS local	Contrôle dynamique des mutations ; structure modulaire NISQ-compatible

Table 4.2 – Comparaison des approches multi-objectifs pour la synthèse de circuits quantiques

L'analyse du tableau 4.1 met en lumière la diversité des stratégies évolutionnaires appliquées à la synthèse de circuits quantiques. Les travaux fondateurs, tels que ceux de Yabuki et Iba (2000) et de Lukac et Perkowski (2002), se sont d'abord concentrés sur des cas spécifiques ou sur de petits circuits, illustrant la faisabilité de générer automatiquement des circuits fonctionnels à partir d'exemples ou de spécifications. Par la suite, des approches plus structurelles ont émergé, comme QEQEA Krylov et Lukac (2018), qui introduit une représentation hybride basée sur qubits et qutrits, et Davis et al. (2020), dont la méthode tient compte de la topologie physique des processeurs NISQ.

Plusieurs méthodes mettent également l'accent sur la modularité, comme l'algorithme en îles de Miranda et al. (2023), ou la représentation configurable à base de matrices proposée dans GA4QCO (Sünkel et al., 2023). Toutefois, bien que ces travaux introduisent des idées puissantes pour la réduction de la profondeur des circuits ou la diminution du nombre de portes bruitées, la majorité reste limitée à des circuits de petite taille (souvent $n \leq 7$). Cela souligne une contrainte majeure : l'explosion combinatoire de l'espace de recherche dès que la taille du circuit augmente.

Référence	**Cible**	**Approche**	**Représentation des circuits**	**Stratégie de fitness**	**Objectif principal**	**Réduction profondeur**	**Scalabilité**	**Réduction portes bruitées**
Yabuki et Iba (2000)	Téléportation quantique	AG structuré par étapes	Étapes fonctionnelles codées avec portes élémentaires (type EPR → Alice → Bob)	Erreur d'amplitude (ratio)	Synthèse fonctionnelle minimale	Oui	Limité à 1 qubit	Non
Lukac et Perkowski (2002)	Circuits arbitraires	AG avec encodage par blocs	Blocs parallèles de portes quantiques élémentaires	Distance matricielle + pénalité longueur	Synthèse générale de circuits	Partielle	Jusqu'à 4 qubits	Non
Giraldi et al. (2004)	Matrices unitaires et QGA	AG + modèle quantique (Algorithme génétique quantique (AGQ))	Matrices unitaires continues	Norme ℓ_1 sur exemples I/O	Apprentissage d'opérateurs	Non abordé	Modéré	Non
Krylov et Lukac (2018)	Synthèse partielle de circuits	QEQEA (qubit + qutrit)	Qubits (θ) + qutrits (axes de rotation)	Fidélité matricielle via trace	Modèle partiellement quantique	Partielle	Faible	Non
Davis et al. (2020)	Circuits pour IBM Q	A*-search + optimisation locale	Séquence de portes natives (U3, CNOT)	Distance unitaire + CNOT count	Compilation topologie-aware	Oui (jusqu'à 5.3×)	Forte (reconfigurable)	Oui
Tandeitnik et Guerreiro (2022)	Circuits correcteurs (QECC)	AG sur portes Clifford	Matrices de portes Clifford	Corrigibility C et profondeur D	Codes stabilisateurs (CSS, Steane, Shor)	Oui	Jusqu'à 7 qubits	Oui (min CNOT)
Miranda et al. (2023)	Portes unitaires (Toffoli, Fredkin)	Island AG + mutation structurée	Blocs unitaires (B_1, B_2, …)	Trace fidelity, erreur ε	Synthèse modulaire sans qubit auxiliaire	Oui	Oui (migration îles)	Oui
Creevey et al. (2023)	Préparation d'états $\lvert\psi\rangle$	AG (GASP) + SLSQP	Séquence de gènes : [qt, G, qc, θ]	$\lvert\langle\psi_{\text{target}}\lvert\psi(\theta)\rangle\rvert^2$	États cibles avec circuit minimal	Oui (vs Qiskit)	Modéré	Oui
Sünkel et al. (2023)	Circuits arbitraires	AG modulaire configurable	Matrice $n \times m$ de portes paramétrées	Fidélité à l'état cible ; entropie ; précision ML	Génération automatique sous contraintes NISQ	Oui	Moyenne (4 qubits, prof. 20)	Oui (portes compatibles NISQ)

Table 4.1 – Comparaison des méthodes évolutionnaires pour la synthèse de circuits quantiques

C'est dans ce contexte que la communauté a progressivement basculé vers des approches multi-objectifs, comme le montre le tableau 4.2. Ces méthodes — notamment celles de (Potoček et al., 2018), (Rindell et al., 2023), (Ding & Spector, 2023), (Rahman et al., 2025) — permettent d'optimiser simultanément plusieurs critères souvent antagonistes : la fidélité, la complexité structurelle, la profondeur ou encore la robustesse face au bruit. Cette orientation répond à la nécessité de produire non pas un circuit parfait, mais un circuit exécutable sur les machines quantiques réelles, sujettes aux erreurs.

Néanmoins, plusieurs défis persistent. La plupart des approches utilisent des représentations de chromosomes modulaires (par blocs ou par couches), qui facilitent l'exploration, mais compliquent le contrôle fin de la profondeur et de la topologie. De plus, l'intégration du bruit, bien que présente dans certains travaux comme ceux de Rindell et al. (2023), reste souvent simplifiée ou stationnaire, ne prenant pas en compte des effets dynamiques ou corrélés. Par ailleurs, même des méthodes adaptatives comme celles basées sur la logique floue (Rahman et al., 2025) reposent encore sur des heuristiques empiriques, dont la robustesse n'est pas toujours prouvée à grande échelle.

Enfin, la comparaison des approches classiques (Tableau 4.1) et multi-objectifs (Tableau 4.2) montre une progression claire vers plus de réalisme, d'adaptabilité et de prise en compte des contraintes matérielles. C'est précisément cette complexité croissante des objectifs — fidélité, bruit, coût — qui a motivé l'adoption d'approches multi-objectifs. Ces dernières se sont imposées comme une réponse naturelle aux limites des méthodes mono-objectif, d'où notre choix de nous inscrire dans cette voie pour le développement de notre propre contribution au chapitre suivant.

4.3 Les algorithmes d'intelligence en essaim

4.3.1 Apport des algorithmes d'intelligence en essaim à la conception de circuits quantiques

Les algorithmes d'intelligence en essaim (Swarm Intelligence), tels que Particle Swarm Optimization PSO, Ant Colony Optimization (ACO), Grey Wolf Optimization (Grey Wolf Optimization (GWO)), ou encore Improved Self-Organizing Migrating Algorithm (iSOMA), constituent des outils pertinents pour la synthèse automatique de circuits quantiques.Inspirées de phénomènes collectifs observés dans la nature (essaims d'oiseaux, colonies de fourmis, meutes de loups), ces méthodes permettent d'explorer efficacement un espace de recherche combinatoire et d'identifier des solutions adaptées tout en optimisant l'utilisation des ressources matérielles. L'étude menée par Horáčková et Zelinka (2025) met en évidence trois axes principaux d'application des algorithmes d'intelligence en essaim dans ce domaine :

1. **Génération de circuits fonctionnels en espace de recherche libre :** Dans ce contexte, les algorithmes tels que PSO, GWO ou iSOMA sont utilisés pour explorer sans contraintes la structure des circuits. Cette approche permet de démontrer leur capacité à produire des solutions viables et parfois inédites, révélant un fort potentiel d'« inventivité algorithmique ».
2. **Reproduction et optimisation de circuits existants sous contraintes :** Ici, les méthodes d'intelligence en essaim doivent reconstruire des circuits imposant certaines conditions, comme la position fixe des portes de mesure. Les résultats obtenus montrent que ces algorithmes, en particulier PSO et iSOMA, sont capables de

générer des architectures équivalentes ou fonctionnellement identiques à celles conçues manuellement, confirmant leur robustesse face à des restrictions de conception.

3. **Réduction du nombre de qubits et optimisation des ressources :** En introduisant volontairement des qubits redondants dans les expériences, les chercheurs ont évalué la capacité des algorithmes à éviter leur utilisation inutile. Les résultats mettent en évidence la faculté de ces approches à proposer automatiquement des circuits économes en qubits, ce qui constitue un atout crucial dans le contexte actuel, où les dispositifs quantiques demeurent limités en termes de ressources.

Dans l'ensemble, les travaux montrent que le PSO se distingue comme l'algorithme le plus performant en termes de diversité et d'optimisation des circuits, tandis que l'ACO apparaît moins efficace dans ce contexte. Ces résultats soulignent que l'intelligence en essaim constitue une voie méthodologique prometteuse pour l'automatisation et l'optimisation de la synthèse de circuits quantiques, en complément des approches évolutionnaires déjà étudiées.

4.3.2 Origines de l'application des algorithmes d'intelligence en essaim (SI) à la synthèse de circuits quantiques

L'intégration des algorithmes d'intelligence en essaim (SI) dans le domaine des circuits réversibles et quantiques s'est véritablement concrétisée au début du XXIe siècle, dans un contexte marqué par la nécessité d'explorer et d'optimiser des espaces de conception hautement combinatoires.

Les premières contributions significatives remontent à la période **2005–2010**, avec notamment les travaux de Sasamal et al. (2005), qui ont appliqué le *Particle Swarm Optimization* (PSO) et l'*Ant Colony Optimization* (ACO) à la synthèse de circuits logiques réversibles (Sasamal et al., 2005). Ces recherches pionnières ont mis en évidence la capacité des SI à réduire le nombre de portes tout en respectant les contraintes de réversibilité, élément fondamental pour le développement du calcul quantique futur. Cette étape constitue ainsi la première incursion des algorithmes d'intelligence en essaim dans la conception de circuits réversibles, lesquels représentent un jalon essentiel vers les circuits quantiques. Plus récemment, une avancée décisive est réalisée en **2023** par **Ivan Zelinka et ses collaborateurs**, qui publient dans *Applied Soft Computing* la première étude consacrée à l'application d'un algorithme d'intelligence en essaim — l'*improved Self-Organizing Migrating Algorithm* (iSOMA) — spécifiquement dédié à la synthèse de circuits quantiques (Zelinka et al., 2023). Cette contribution inaugure l'usage explicite des SI pour la génération automatique de circuits quantiques optimisés. L'année suivante, en **2024**, ces travaux sont approfondis et élargis dans l'*International Journal of Parallel, Emergent and Distributed Systems*, au travers d'une comparaison systématique de plusieurs algorithmes d'intelligence en essaim (PSO, ACO, GWO et iSOMA) (Horáčková & Zelinka, 2025). Cette démarche consolide leur rôle pionnier et confirme leur pertinence méthodologique dans la conception et l'optimisation de circuits quantiques.

4.3.3 Typologie des travaux réalisés

4.3.3.1 Optimisation par Essaim de Particules PSO pour la synthèse de circuits quantiques

Hidayat et Cheah (2025) proposent une approche basée sur PSO pour la conception automatique de circuits quantiques, avec pour objectif la génération de solutions optimales au problème *MaxOne*. Le PSO est inspiré du comportement collectif d'animaux sociaux, tel que le vol d'oiseaux ou le déplacement de bancs de poissons (Kennedy & Eberhart, 1995), et repose sur l'actualisation itérative de la *position* et de la *vitesse* de chaque particule de l'essaim. Chaque particule i est caractérisée par :

$$\mathbf{x}_i(t) \quad \text{(position courante)} \quad \text{et} \quad \mathbf{v}_i(t) \quad \text{(vitesse courante)},$$

où la position encode un circuit quantique sous forme de séquence de portes, avec leurs qubits cibles et paramètres d'angle.

L'évolution de l'essaim suit les équations classiques du PSO :

$$\mathbf{v}_i(t+1) = w\,\mathbf{v}_i(t) + c_1 r_1 \left(\mathbf{pbest}_i - \mathbf{x}_i(t)\right) + c_2 r_2 \left(\mathbf{gbest} - \mathbf{x}_i(t)\right), \tag{4.3.1}$$

$$\mathbf{x}_i(t+1) = \mathbf{x}_i(t) + \mathbf{v}_i(t+1), \tag{4.3.2}$$

où :

- w est le **poids d'inertie**, contrôlant l'impact de la vitesse précédente,
- c_1 et c_2 sont les coefficients d'attraction cognitive et sociale,
- r_1 et r_2 sont des nombres aléatoires uniformes dans $[0, 1]$,
- $\mathbf{pbest}_i$ est la meilleure position atteinte par la particule i,
- **gbest** est la meilleure position globale trouvée par l'essaim.

La **fonction de fitness** proposée combine trois critères :

$$f(C) = \alpha \cdot \mathrm{Acc}(C) - \beta \cdot L(C) - \gamma \cdot D(C),$$

où :

- $\mathrm{Acc}(C)$ est l'exactitude fonctionnelle du circuit C (proportion de sorties correctes),
- $L(C)$ est le nombre total de portes,
- $D(C)$ est la profondeur du circuit,
- α, β, γ sont des poids définis expérimentalement.

Comme illustré à la Fig. 4.5, le processus PSO pour la synthèse de circuits évolue en itérations en mettant à jour position et vitesse des particules jusqu'à satisfaction d'un critère d'arrêt (Hidayat & Cheah, 2025).

Les auteurs utilisent un simulateur Qiskit et comparent le PSO à un algorithme génétique (GA) sur le même problème. Les principaux paramètres du PSO sont :

$$w = 0.7, \quad c_1 = 1.5, \quad c_2 = 1.5, \quad \text{taille de l'essaim} = 30.$$

Chaque expérience est répétée 30 fois pour mesurer la stabilité des résultats.

Les expériences montrent que le PSO converge vers des solutions valides en moyenne **35% plus rapidement** que le AG. Le nombre moyen de portes dans les circuits obtenus par PSO est inférieur d'environ **12%** à celui des circuits produits par le AG, et la profondeur

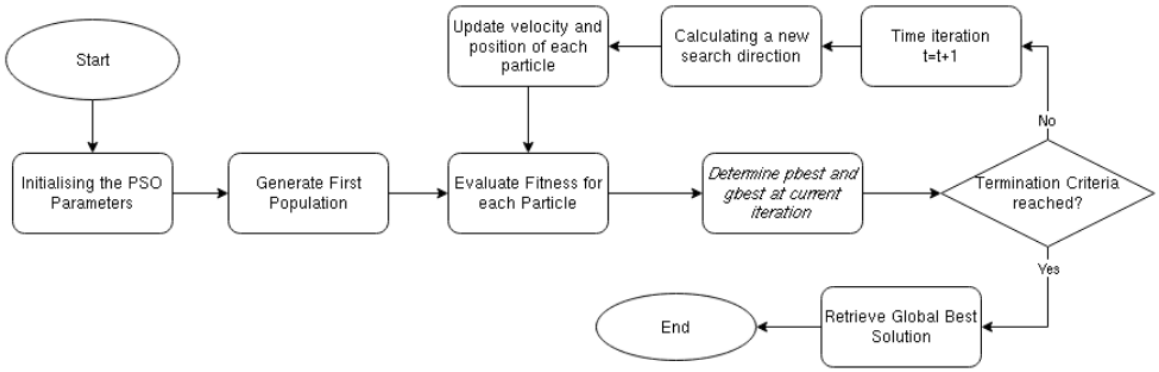


FIGURE 4.5 – Schéma de flux du PSO pour la synthèse de circuits quantiques (HIDAYAT & CHEAH, 2025).

moyenne est réduite de **15%**. De plus, l'écart-type des résultats est plus faible, indiquant une plus grande stabilité inter-expérimentations. Toutefois, les performances du PSO restent sensibles aux réglages de w, c_1 et c_2, nécessitant une calibration fine pour éviter la stagnation.

Le PSOapparaît comme une alternative efficace aux approches génétiques pour la synthèse automatique de circuits quantiques, grâce à sa rapidité de convergence et sa capacité à produire des circuits plus compacts. Les auteurs suggèrent d'explorer des extensions hybrides (PSO–AG) ou multi-essaims afin d'améliorer l'exploration de l'espace des solutions et de limiter le risque de convergence prématurée.

Datta et al. (2012) proposent une méthode de synthèse de circuits réversibles fondée sur l'Optimisation par Essaim de Particules PSO, motivée par les enjeux thermodynamiques (principe de Landauer) et l'intérêt de la réversibilité pour les architectures quantiques (BENNETT, 1973 ; FEYNMAN, 1985 ; FREDKIN & TOFFOLI, 1982 ; LANDAUER, 1961 ; TOFFOLI, 1980). Chaque particule encode une *cascade de portes* de la librairie CNT (NOT, CNOT, Toffoli généralisée) sous la forme d'un tableau de taille K ; une porte n-entrées est codée par un entier dans l'intervalle $[0,\ n \cdot 2^{n-1}-1]$, avec la valeur `-1` réservée à une *porte factice* (pass-through). La taille maximale K est initialisée à 10 puis *augmentée par pas de* 5 si aucune solution n'est trouvée. La fonction objectif agrège trois composantes

$$F(P_i) \;=\; c_1\,\mathsf{mismatch}(f_i, f_{\text{des}}) \;+\; c_2\,\mathsf{hamm}(f_i, f_{\text{des}}) \;+\; c_3\,\mathsf{length}(P_i),$$

où f_i est la permutation réalisée par la cascade représentée par la particule P_i, f_{des} la permutation cible, mismatch compte les éléments différents, hamm évalue un *heuristique Hamming* bit-à-bit pondéré (plus le bit "se trompe souvent", plus la distance augmente), et length comptabilise le nombre de portes effectives en ignorant les `-1` et les portes consécutives identiques. L'exploration applique des mises à jour stochastiques (insertion/échange/suppression/remplacement de portes, éventuellement *importées* d'un voisin), puis accepte une détérioration avec une faible probabilité $p = 0.05$ (règle *R1*) pour éviter la stagnation, et accepte toute amélioration (règle *R2*). Comme en PSO standard (KENNEDY & EBERHART, 1995), les meilleurs historiques individuels (*pbest*) et global (*gbest*) guident la recherche. Les expériences (implémentation C, 2.8 GHz, 4 Go) sur des *benchmarks réversibles* (jusqu'à 5 entrées, RevLib et classiques) montrent des solutions **optimales**

ou quasi-optimales avec des **temps de synthèse très courts** (souvent $< 1\,\mathrm{s}$) et des métriques reportées en *Gate Count* (GC), *Quantum Cost* (QC) et *Transistor Cost* (TC). Par exemple, les circuits `peres` (GC= 2, QC= 6, TC= 24) et `fredkin` (GC= 3, QC= 7, TC= 32) sont atteints en temps négligeable, `1bitadder` est optimal (GC= 4, QC= 12, TC= 48, $\sim 0.7\,\mathrm{s}$), tandis qu'une permutation plus difficile `(3 6 2 5 7 1 0 4)` reste *près de l'optimum* (GC= 8 vs 6 optimal, QC= 20, TC= 80, $\sim 0.7\,\mathrm{s}$). Globalement, la méthode surpasse plusieurs approches heuristiques antérieures en *gate count* et rivalise avec des méthodes exactes SAT/groupes sur de nombreux cas, avec l'avantage de **temps de calcul** très faibles ; les auteurs soulignent toutefois que l'optimalité n'est pas garantie sur tous les benchmarks et suggèrent de combiner PSO et méthodes de théorie des groupes pour bénéficier d'un guidage structurel préalable.

L'article de Li et al. (2025) traite du *mapping* de circuits quantiques sur du matériel NISQ à connectivité limitée en combinant une allocation initiale par Discrete Particle Swarm Optimization (DPSO) et un ordonnancement/insertion de `SWAP` par Deep Reinforcement Learning (DRL) ; l'idée est d'explorer globalement l'espace des affectations logique→physique, puis de raffiner localement les décisions pour minimiser simultanément le nombre de `SWAP` et la profondeur. Pour formaliser le problème, on modélise le processeur par un graphe de couplage $G = (V, E)$ où V est l'ensemble des qubits physiques et E les liaisons disponibles, et l'on note $\mathcal{G}_2$ l'ensemble des portes à deux qubits du circuit logique ; si π est une affectation des qubits logiques aux qubits physiques, alors la *distance de routage* d'une porte $(i, j) \in \mathcal{G}_2$ vaut $d_G\big(\pi(i), \pi(j)\big)$ (longueur du plus court chemin dans G), ce qui induit un *borne inférieure* sur les `SWAP` nécessaires :

$$\mathrm{LB}_{\mathrm{SWAP}}(\pi) \;=\; \sum_{(i,j)\in\mathcal{G}_2} \Big(d_G(\pi(i), \pi(j)) - 1\Big). \tag{4.3.3}$$

Dans la pratique, on combine cette contrainte topologique à une pénalisation de la profondeur (longueur de la *critical path*) pour définir un objectif à l'initialisation par DPSO, par exemple

$$J_{\mathrm{map}}(\pi) \;=\; \alpha \sum_{(i,j)\in\mathcal{G}_2} \Big(d_G(\pi(i), \pi(j)) - 1\Big) \;+\; \beta\,\mathrm{depth}(\pi), \tag{4.3.4}$$

où $\alpha, \beta > 0$ pondèrent respectivement le coût en `SWAP` et la profondeur estimée du circuit mappé.

Pour chercher une bonne affectation π, chaque particule du PSO code une permutation à n éléments (les n qubits logiques) ; une façon classique de gérer la nature discrète est l'*encodage par clés aléatoires* : on maintient un vecteur réel $\mathbf{z} \in \mathbb{R}^n$ et on obtient la permutation par tri $\pi = \mathrm{argsort}(\mathbf{z})$. L'évolution suit alors les équations PSO (Kennedy & Eberhart, 1995) dans l'espace continu,

$$\mathbf{v}(t{+}1) = \omega\,\mathbf{v}(t) + c_1 r_1\Big(\mathbf{pbest} - \mathbf{z}(t)\Big) + c_2 r_2\Big(\mathbf{gbest} - \mathbf{z}(t)\Big), \tag{4.3.5}$$

$$\mathbf{z}(t{+}1) = \mathbf{z}(t) + \mathbf{v}(t{+}1), \tag{4.3.6}$$

puis un *passage discret* par tri fournit la permutation candidate ; la fitness de chaque particule est évaluée via (4.3.4), ce qui pousse l'essaim vers des mappings où les portes bi-qubits sont déjà proches dans G, réduisant le besoin de `SWAP` ultérieurs.

Une fois l'affectation initiale fixée, l'ordonnancement et l'insertion de `SWAP` sont modélisés comme un processus décisionnel séquentiel résolu par DRL : à chaque étape t, l'état

s_t encode des caractéristiques du circuit restant (frontière de portes exécutables, positions des qubits physiques, congestion sur G, etc.) et l'agent choisit une action a_t (exécuter une porte admissible ou insérer un `SWAP` sur une arête de G). La récompense *intermédiaire* est définie pour pénaliser les coûts structurels effectivement induits à l'étape,

$$r_t \;=\; -\lambda_1\,\Delta\mathrm{SWAP}_t \;-\; \lambda_2\,\Delta\mathrm{depth}_t \;-\; \lambda_3\,\Delta\mathrm{stall}_t, \tag{4.3.7}$$

où $\Delta\mathrm{SWAP}_t$ compte les `SWAP` ajoutés, $\Delta\mathrm{depth}_t$ la variation de profondeur (ou du temps de fin planifié), et $\Delta\mathrm{stall}_t$ une pénalité de « blocage » quand aucune porte utile n'avance ; les poids $\lambda_k > 0$ équilibrent ces objectifs. L'agent est entraîné par *Double DQN* (van Hasselt et al., 2016) afin de stabiliser l'estimation de la valeur d'action ; l'équation cible pour l'apprentissage est

$$y_t \;=\; r_t \;+\; \gamma\, Q_{\bar\theta}\Big(s_{t+1}, \arg\max_a Q_\theta(s_{t+1}, a)\Big), \tag{4.3.8}$$

avec facteur d'actualisation $\gamma \in (0,1)$, réseau en ligne Q_θ, réseau cible $Q_{\bar\theta}$ et perte quadratique $L(\theta) = \Big(y_t - Q_\theta(s_t, a_t)\Big)^2$; un *rejeu priorisé* améliore la vitesse de convergence en rééchantillonnant les transitions les plus informatives.

Au total, la décomposition « *DPSO* pour π globalement bon $\rightarrow$ *DRL* pour le raffinement local » permet de (i) réduire dès le départ les distances de routage grâce à (4.3.3)–(4.3.4), puis (ii) choisir, au fil de l'ordonnancement, des insertions de `SWAP` réellement nécessaires via (4.3.7) et (4.3.8), ce qui se traduit empiriquement par moins de `SWAP`, une profondeur plus faible et des temps de compilation moindres que des heuristiques monolithiques et même que des compilateurs industriels, tout en restant compatible avec les contraintes topologiques des dispositifs NISQ (Li et al., 2025).

Comme illustré à la Fig. 4.6, le processus PSO pour la synthèse de circuits quantiques.

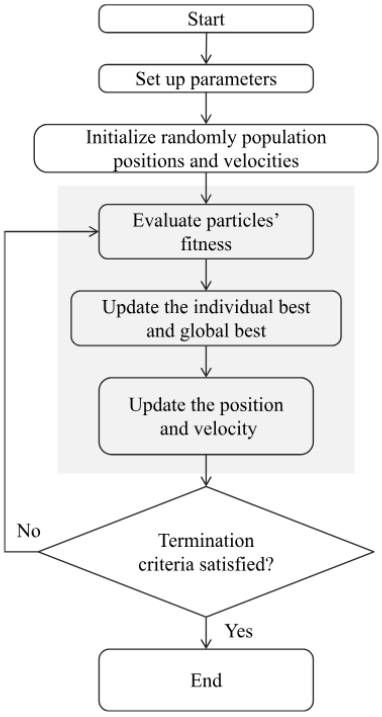


Figure 4.6 – Schéma de flux du PSO pour la synthèse de circuits quantiques (Li et al., 2025).

Sur le plan empirique, la combinaison DPSO pour l'allocation initiale et DRL pour le scheduling des `SWAP` se traduit par des gains substantiels sur des benchmarks variés et des topologies IBM (p. ex., Q20, Q127) : par rapport à une heuristique de référence (SI-FDS), les auteurs rapportent en moyenne une **réduction du nombre de `SWAP` de 29,1 %** et une **diminution du temps de compilation de 41,95 %** ; face à un planificateur *RL* de type *MCTS*, la méthode hybride abaisse simultanément `SWAP` et profondeur d'environ **33 %** pour les petits circuits et **19 %** pour les grands, confirmant l'intérêt de l'exploration globale (DPSO) suivie d'un raffinement local (DRL) ; enfin, comparée à des compilateurs industriels, elle insère **14,73 %** de `SWAP` en moins que `t`$|$`ket`$\rangle$ et **16,55 %** en moins que `Qiskit`, avec des bénéfices qui s'accentuent à mesure que la taille du circuit et les contraintes de connectivité augmentent. Si l'entraînement *off-line* du modèle DRL induit un coût initial non négligeable, l'étape d'inférence demeure rapide et réutilisable, ce qui rend l'approche particulièrement praticable pour des familles de circuits similaires (Li et al., 2025).

4.3.4 Analyse comparative des méthodes étudiées

Le tableau 4.3 met en parallèle deux configurations représentatives. D'un côté, la synthèse « générique »(Hidayat & Cheah, 2025) où une particule code directement un circuit (portes, qubits cibles, angles) et où la recherche s'effectue dans un espace continu avec la mise à jour standard du PSO (4.3.1)–(4.3.2) ; de l'autre, la synthèse *réversible* (Datta et al., 2012) où une particule code une cascade de portes CNOT via un encodage entier, plus adapté à la combinatoire discrète. Dans les deux cas, la fonction objectif couple *exactitude fonctionnelle* et *coûts structurels* (nombre de portes, profondeur), avec des métriques dédiées à la réversibilité (GC, QC, TC) lorsque pertinent. L'idée directrice est la même : guider l'essaim vers des circuits à la fois corrects et *compacts*, en tirant parti de la dynamique « cognitive/sociale » du PSO pour accélérer la convergence.

Référence	**Contexte / Tâche**	**Encodage / Recherche**	**Objectif (fitness) & métriques**	**Résultats & limites (extraits)**
Hidayat & Cheah (2025) (Hidayat & Cheah, 2025)	Synthèse de circuits (ex. MaxOne)	Une particule = un circuit (séquence de portes, qubits cibles, angles). PSOcontinu (inertie w, c_1, c_2) sur l'espace des paramètres ; mise à jour (4.3.1)–(4.3.2).	$f(C) = \alpha\,\mathrm{Acc}(C) - \beta\,L(C) - \gamma\,D(C)$: exactitude fonctionnelle, #portes, profondeur.	**Convergence** en moyenne **35 % plus rapide** qu'un GA ; **-12 %** de portes et **-15 %** de profondeur vs AG (Qiskit sim). *Limite* : sensibilité à w, c_1, c_2 (calibration).
Datta, Sengupta & Rahaman (2012) (Datta et al., 2012)	**Synthèse réversible** (librairie CNT : NOT, CNOT, Toffoli gén.)	Une particule = cascade de K portes (codage entier ; `-1`=pass-through). Mouvements stochastiques (insertion/ échange/ suppression/ remplacement) guidés par *pbest*/*gbest*.	$F = c_1\,\mathsf{mismatch} + c_2\,\mathsf{hamm} + c_3\,\mathsf{length}$; report des coûts *Gate Count* (GC), *Quantum Cost* (QC), *Transistor Cost* (TC).	**Solutions optimales ou quasi-optimales** très rapides (< 1 s) sur benchmarks (p. ex. `peres` GC=2, QC=6 ; `fredkin` GC=3, QC=7). *Limite* : cas test jusqu'à 5 entrées (taille modérée).

Table 4.3 – Approches PSO « seules » pour la conception automatique de circuits (quantiques et réversibles).

Les résultats confirment que le PSO est particulièrement efficace lorsque la compacité et la vitesse d'itération sont prioritaires. En synthèse générique, Hidayat & Cheah rapportent en moyenne **+35 %** de vitesse de convergence par rapport à un GA, avec **-12 %** de portes et **-15 %** de profondeur sur un benchmark de type *MaxOne* — des gains cohérents avec une fitness qui pénalise explicitement taille et profondeur. L'encodage réel (paramètres d'angles) favorise ici une exploration continue et lisse, mais rend la performance *sensible* aux hyperparamètres (w, c_1, c_2), ce qui impose une calibration soignée pour éviter stagnation et oscillations. En logique réversible, l'encodage *discret* par entiers et la dynamique *pbest*/*gbest* permettent d'atteindre des solutions *optimales ou quasi-optimales* en *temps très court* (souvent < 1 s), avec des cas emblématiques comme `peres` (GC=2, QC=6) et `fredkin` (GC=3, QC=7). La limite principale tient à l'*échelle des benchmarks* (jusqu'à 5 entrées dans l'étude), ce qui suggère que le PSO seul excelle pour des circuits *petits à moyens* ou fortement structurés, tandis que des problèmes plus grands ou multi-contraints pourront bénéficier d'extensions (p. ex. PSO hybride, ou intégration d'objectifs multiples). En synthèse, le tableau 4.3 justifie l'usage du PSO seul comme *baseline robuste et rapide* pour générer des circuits compacts, et comme *initialiseur* naturel avant un affinage plus riche en contraintes (multi-objectifs, prise en compte explicite du bruit) dans les sections

comparatives suivantes.

4.4 Synthèse comparative

Cette section met en regard les performances du PSO utilisé seul et celles des algorithmes évolutionnaires — aussi bien en version mono-objectif (GA classiques) qu'en version multi-objectifs (NSGA-II et variantes). Le tableau 4.4 synthétise les critères les plus pertinents pour la conception/optimisation de circuits quantiques en contexte NISQ : vitesse de convergence, qualité structurelle (nombre de portes et profondeur), fidélité fonctionnelle, robustesse au bruit, scalabilité et facilité de mise en œuvre. Les références mobilisées couvrent des usages contrastés — de la synthèse « générique » de circuits et d'états cibles (Creevey et al., 2023 ; Davis et al., 2020 ; Miranda et al., 2023) à la synthèse réversible (Datta et al., 2012) — ainsi que des cadres explicitement multi-objectifs intégrant coûts structurels et bruit (Ding & Spector, 2023 ; Potoček et al., 2018 ; Rindell et al., 2023). L'objectif est de dégager, au-delà des résultats ponctuels, des tendances opérationnelles permettant de choisir la famille d'algorithmes la plus adaptée aux contraintes matérielles et aux objectifs du chapitre.

Critère	**PSO(seul)**	**AG(mono-objectif)**	**AG multi-objectifs (NSGA-II, etc.)**
Vitesse de convergence	Rapide sur espaces continus/mixtes ; ex. : **+35 %** vs GA (Hidayat & Cheah, 2025).	Variable ; dépend encodage/croisement. *GASP* montre de bonnes vitesses pour états cibles (Creevey et al., 2023).	Plus coûteux (évaluation multi-critères), mais front de Pareto utile (**PotocÌŇek2018** ; Ding & Spector, 2023 ; Rindell et al., 2023).
Qualité structurelle (portes, profondeur)	Réduction typique **-12 %** portes, **-15 %** profondeur vs AG sur MaxOne (Hidayat & Cheah, 2025) ; optimalité fréquente en réversible (Datta et al., 2012).	Fort potentiel : ex. réduction jusqu'à 5.3× **CNOT** vs UniversalQ, **-36 %** longueur vs Qiskit (Davis et al., 2020) ; circuits plus courts que Qiskit avec *GASP* (Creevey et al., 2023).	Maîtrise fine des compromis (profondeur/CNOT/fidélité) ; améliore la robustesse et la compacité simultanément (Ding & Spector, 2023 ; Rindell et al., 2023).
Fidélité / erreur fonctionnelle	Bonne si fitness bien pondérée ; focus « compacité+exactitude » (Hidayat & Cheah, 2025).	Très bonne quand cible unitaire connue (trace-fidelity, etc.) (Miranda et al., 2023).	Optimise explicitement fidélité + coûts ; ex. : **+38 %** de fidélité effective vs baseline LRSP sous bruit (Rindell et al., 2023).
Robustesse au bruit NISQ	Indirecte (via pénalités de profondeur/CNOT).	Peut intégrer contraintes matérielles/topologiques (ex. topologie-aware) (Davis et al., 2020).	Naturellement traitée via objectifs multiples (fidélité bruitée, coût CNOT) (Rindell et al., 2023).
Scalabilité / encodage	Encodage simple ; efficace pour petits/moyens cas ; sensible aux hyperparamètres (w, c_1, c_2).	Encodages modulaires (blocs/îles) pour limiter la stagnation, bonne diversité (Miranda et al., 2023).	Front de Pareto utile pour grands circuits mais coût d'évaluation plus élevé (**PotocÌŇek2018** ; Ding & Spector, 2023).
Mise en œuvre	Peu de métaparamètres, pipeline simple ; calibration nécessaire.	Plus d'opérateurs (sélection, crossover, mutation) à régler ; forte flexibilité.	Nécessite le choix/poids des objectifs et l'archivage Pareto ; plus *heavy*, mais plus expressif.
Messages clés	**PSO** excelle sur la compacité et la vitesse d'itération ; très efficace en réversible.	**AG** peut atteindre des circuits très compétitifs/compacts, et s'adapter à la topologie.	**NSGA-II et dérivés** offrent les meilleurs compromis pratiques (fidélité/coût/profondeur) en NISQ.

Table 4.4 – PSO *vs* algorithmes évolutionnaires (AG/NSGA-II) pour la conception/optimisation de circuits quantiques.

Les données indiquent d'abord que le **PSO** se distingue par une *convergence rapide* et une *bonne compacité* des circuits lorsque la fitness pénalise explicitement la taille et la profondeur : sur un cas de synthèse de type *MaxOne*, on observe typiquement **+35 %** de vitesse par rapport à un GA, avec **-12 %** de portes et **-15 %** de profondeur en moyenne (Hidayat & Cheah, 2025). En logique réversible, le PSO atteint fréquemment des solutions optimales ou quasi-optimales avec des temps de synthèse très faibles (Datta et al., 2012). En revanche, sa *sensitivity* aux hyperparamètres (w, c_1, c_2) impose une calibration fine pour éviter stagnation et dérive. Les **AG mono-objectif** montrent une *flexibilité structurelle* élevée grâce aux opérateurs de croisement/mutation et à des encodages modulaires (par blocs, îles), ce qui peut conduire à des circuits très compétitifs : réduction jusqu'à 5.3× du nombre de CNOT par rapport à une baseline UniversalQ et **-36 %** de longueur face à Qiskit dans une approche de type recherche guidée + optimisation locale (Davis et al., 2020), ou encore des circuits plus courts que Qiskit pour la préparation d'états cibles avec *GASP* (Creevey et al., 2023). Toutefois, l'absence d'objectifs multiples explicites peut rendre plus difficile l'arbitrage entre fidélité et coût structurel dans des environnements bruités. À l'opposé, les **AG multi-objectifs** (NSGA-II et dérivés) *rendent visibles et pilotables* ces compromis via un front de Pareto, ce qui améliore la *robustesse effective* sur matériel NISQ : par exemple, sous bruit réaliste IBM, on rapporte jusqu'à **+38 %** de fidélité effective par rapport à une baseline *LRSP* grâce à l'optimisation conjointe fidélité/coût CNOT (Rindell et al., 2023), et des architectures PQC plus parcimonieuses et moins sensibles au bruit dans un cadre tri-objectif (Ding & Spector, 2023). En synthèse, **PSO** seul est un excellent choix lorsque l'on vise la *compacité rapide* (prototypage, réversible, petits à moyens circuits), tandis que **AG/NSGA-II** s'avèrent préférables dès que la *prise en compte simultanée* de la fidélité, du coût structurel (profondeur/CNOT) et de la *robustesse au bruit* devient critique — typiquement le cas en NISQ. Un schéma hybride (PSO pour une initialisation « globale », AG/NSGA-II pour l'affinage multi-critères) constitue enfin une voie pragmatique pour bénéficier des atouts des deux familles.

Conclusion

Ce chapitre a montré que les métaheuristiques bio-inspirées constituent un levier crédible et polyvalent pour la conception automatique de circuits quantiques. D'un côté, le **PSO** se distingue comme *baseline* rapide et frugale : avec un encodage soigné et une fitness qui pénalise explicitement la taille et la profondeur, il produit des circuits compacts et converge vite, y compris en logique réversible où des solutions optimales sont souvent atteintes. De l'autre, les **algorithmes évolutionnaires** — des AG classiques aux variantes **multi-objectifs** (NSGA-II) — se révèlent particulièrement adaptés dès qu'il faut arbitrer, de manière explicite, entre fidélité, CNOT, profondeur et robustesse au bruit : les fronts de Pareto rendent ces compromis visibles et pilotables, et les encodages modulaires (blocs, îles) favorisent une exploration diversifiée tout en limitant la stagnation.

Ces atouts n'effacent pas plusieurs limites structurantes : une *scalabilité* encore contrainte (croissance exponentielle, dépendance aux encodages), des *modèles de bruit* souvent stationnaires ou simplifiés, une *sensibilité* des performances à la calibration des hyperparamètres (notamment pour le PSO), et l'intégration parfois incomplète des contraintes de *topologie matérielle* et de mapping. Les frameworks récents (synthèse hiérarchique par blocs, intégration du bruit, hybridation avec optimisation locale) attestent néanmoins de la maturation du domaine et de sa pertinence pratique en contexte NISQ.

Conclusion

L'informatique quantique représente aujourd'hui l'un des axes majeurs d'innovation scientifique et technologique. En exploitant des propriétés uniques telles que la superposition, l'intrication et l'interférence, elle promet de surmonter certaines limites fondamentales des architectures classiques. Toutefois, la mise en œuvre pratique de cette nouvelle approche reste fortement contrainte par les caractéristiques des dispositifs actuels de type NISQ, où le bruit, la faible cohérence et les contraintes matérielles imposent une optimisation fine des circuits.

Dans ce contexte, ce mémoire a exploré le rôle des **métaheuristiques bio-inspirées**, et plus particulièrement des **algorithmes évolutionnaires** (AG, NSGA-II) et de l'**intelligence en essaim** (PSO), pour la **conception automatique de circuits quantiques**. L'étude a montré que ces méthodes offrent une alternative robuste aux approches déterministes classiques, grâce à leur capacité à :

— explorer efficacement de vastes espaces de recherche,

— optimiser plusieurs critères simultanément (fidélité, profondeur, coût structurel),

— générer des circuits adaptés aux contraintes matérielles des processeurs quantiques actuels.

La comparaison des différentes approches a mis en évidence des complémentarités notables : les **algorithmes génétiques** se distinguent par leur flexibilité et leur aptitude à générer des solutions diversifiées, tandis que le **PSO** favorise une convergence rapide vers des solutions prometteuses. Les approches multi-objectifs, telles que **NSGA-II**, se révèlent particulièrement adaptées pour équilibrer simultanément fidélité et complexité structurelle.

En définitive, les travaux réalisés confirment que les algorithmes évolutionnaires constituent une voie prometteuse pour **automatiser la conception de circuits quantiques optimisés**. Néanmoins, de nombreuses perspectives s'ouvrent pour consolider ces résultats, notamment :

— l'intégration de modèles hybrides combinant évolutionnaires et apprentissage par renforcement,

— l'exploitation de représentations tensoriales et de méthodes de réduction de complexité pour traiter des circuits de grande échelle,

— la validation expérimentale des solutions générées sur des plateformes matérielles réelles afin de mesurer leur robustesse face au bruit et aux erreurs physiques.

Ainsi, ce travail contribue à tracer les contours d'un champ de recherche en pleine expansion, où l'alliance entre **inspiration naturelle** et **technologies quantiques** apparaît comme une approche féconde pour accompagner la transition vers des systèmes quantiques plus fiables et performants.

Bibliographie

Abdou, W., Bloch, C., Charlet, D., & Spies, F. (2013). Algorithme génétique multi-objectifs adaptatif. *5èmes Journées Doctorales/Journées Nationales MACS*.

AI, G. Q. (2020). Cirq : A Python Framework for Creating, Editing, and Invoking Quantum Circuits.

Andrae, A. S. G., & Edler, T. (2015). On global electricity usage of communication technology : trends to 2030. *Challenges*, *6*(1), 117-157.

Arora, S., & Barak, B. (2009). *Computational Complexity : A Modern Approach*. Cambridge University Press.

Bäck, T., & Schwefel, H.-P. (1993). An Overview of Evolutionary Algorithms for Parameter Optimization. *Evolutionary Computation*, *1*, 1-23. https://doi.org/10.1162/evco.1993.1.1.1

Barnier, N., & Brisset, P. (1999). Optimisation par algorithme génétique sous contraintes. *Revue des Sciences et Technologies de l'Information-Série TSI : Technique et Science Informatiques*, *18*(1), pp-1.

Bell, J. S. (1964). On the Einstein Podolsky Rosen paradox. *Physics Physique Fizika*, *1*(3), 195-200.

Benioff, P. (1980). The computer as a physical system : A microscopic quantum mechanical Hamiltonian model of computers as represented by Turing machines. *Journal of Statistical Physics*, *22*(5), 563-591. https://doi.org/10.1007/BF01011339

Benkirat, A. (2019). *Vers une stratégie collaborative de foraging multi-robots* [Mémoire de fin d'études (Master)]. Université 8 Mai 1945 – Guelma.

Bennett, C. H. (1973). Logical Reversibility of Computation. *IBM Journal of Research and Development*, *17*, 525-532.

Bennett, C. H., Brassard, G., Crépeau, C., Jozsa, R., Peres, A., & Wootters, W. K. (1993). Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels. *Physical Review Letters*, *70*(13), 1895-1899.

Bhandari, S., Nichele, S., Denysov, S., & Lind, P. G. (2024). How quantum and evolutionary algorithms can help each other : two examples. *arXiv preprint arXiv :2408.00448*.

Blum, C. (2024). Ant colony optimization : A bibliometric review. *Physics of Life Reviews*.

Blum, C., & Roli, A. (2003). Metaheuristics in combinatorial optimization : Overview and conceptual comparison. *ACM computing surveys (CSUR)*, *35*(3), 268-308.

Chuang, I. L. (2002). Quantum computation : a new scientific paradigm. *Physics World*, *15*(1), 33-38.

Computing, R. (2022). *Annonce d'un processeur quantique 128 qubits*. https://www.rigetti.com

Creevey, E., Stanfield, R., White, D. R., Rangamani, A., Abbasloo, E., Tavakkoli, A., & Bradley, M. J. (2023). GASP : A Genetic Algorithm for State Preparation on Quantum Computers. *Scientific Reports*, *13*(1), 10898. https://doi.org/10.1038/s41598-023-37767-w

Datta, K., Sengupta, I., & Rahaman, H. (2012). Particle Swarm Optimization based Circuit Synthesis of Reversible Logic. *2012 International Symposium on Electronic System Design (ISED)*, 226-230.

Davis, M. G., Smith, E., Tudor, A., Sen, K., Siddiqi, I., & Iancu, C. (2020). Towards Optimal Topology Aware Quantum Circuit Synthesis. *IEEE International Conference on Quantum Computing and Engineering (QCE)*, 306-317. https://doi.org/10.1109/QCE49297.2020.00036

de Andoin, M. G., & Echanobe, J. (2022). Implementable hybrid quantum ant colony optimization algorithm. *Quantum Machine Intelligence*, *4*(2), 12.

Deb, K. (1999). Multi-objective genetic algorithms : Problem difficulties and construction of test problems. *Evolutionary computation*, *7*(3), 205-230.

Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm : NSGA-II. *IEEE Transactions on Evolutionary Computation*, *6*(2), 182-197.

Deutsch, D. (1985). Quantum theory, the Church–Turing principle and the universal quantum computer. *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences*, *400*(1818), 97-117. https://doi.org/10.1098/rspa.1985.0070

Deutsch, D., & Jozsa, R. (1992). Rapid solution of problems by quantum computation. *Proceedings of the Royal Society A*, *439*(1907), 553-558.

Ding, L., & Spector, L. (2023). Multi-Objective Evolutionary Architecture Search for Parameterized Quantum Circuits. *Entropy*, *25*(1), 93. https://doi.org/10.3390/e25010093

Dorigo, M., & Socha, K. (2018, mai). An Introduction to Ant Colony Optimization. In T. F. Gonzalez (Éd.), *Handbook of Approximation Algorithms and Metaheuristics, Second Edition* (1re éd., p. 395-408). Chapman ; Hall/CRC. https://doi.org/10.1201/9781351236423-23

Einstein, A., Podolsky, B., & Rosen, N. (1935). Can Quantum-Mechanical Description of Physical Reality Be Considered Complete ? *Physical Review*, *47*, 777-780.

et al., K. M. S. (2018). Q# : Enabling Scalable Quantum Computing and Development with a High-level DSL. *Proceedings of the Real World Domain Specific Languages Workshop 2018*.

Evolutionary Algorithms in Engineering Design Optimization. (2022, mars). MDPI. https://doi.org/10.3390/books978-3-0365-2715-4

Ezratty, O. (2019). *Understanding Quantum Technologies* [Édition mise à jour en continu]. Self-published. https://www.xn--quantum-bibliothque-5yb.com

Farhi, E., Goldstone, J., & Gutmann, S. (2014). A Quantum Approximate Optimization Algorithm. *arXiv preprint*.

Feynman, R. (1985). Quantum Mechanical Computers. *Optics News*, *11*, 11-20.

Fowler, A. G., Mariantoni, M., Martinis, J. M., & Cleland, A. N. (2012). Surface codes : Towards practical large-scale quantum computation. *Physical Review A*, *86*, 032324.

Fredkin, E., & Toffoli, T. (1982). Conservative Logic. *International Journal of Theoretical Physics*, *21*, 219-253.

Gambs, S. (2008). Introduction to Quantum Computing : A Survey for Computer Scientists. *ACM Computing Surveys*, *40*(3), 1-43.

Gershenfeld, N. A., & Chuang, I. L. (1997). Bulk spin-resonance quantum computation. *Science*, *275*(5298), 350-356. https://doi.org/10.1126/science.275.5298.350

Giraldi, G. A., Portugal, R., & Thess, R. N. (2004). Genetic Algorithms and Quantum Computation. *arXiv preprint cs/0403003*. https://arxiv.org/abs/cs/0403003v1

Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.

Goldberg, D. E., & Lingle, R. (1985). Alleles, loci, and the traveling salesman problem. *Proceedings of the First International Conference on Genetic Algorithms*.

Gondran, M., & Gondran, A. (2020). *Introduction à l'informatique quantique*. Ellipses.

Grover, L. K. (1996). A fast quantum mechanical algorithm for database search, 212-219. https://doi.org/10.1145/237814.237866

Hidayat, M., & Cheah, K. (2025). Particle Swarm Optimization for Quantum Circuit Synthesis : Performance Analysis and Insights. *arXiv preprint arXiv :2507.02898*.

Holand, J. H. (1975). Adaptation in natural and artificial systems. *Ann Arbor : The University of Michigan Press*, *32*.

Holland, J. H. (1984). Genetic algorithms and adaptation. *Adaptive control of ill-defined systems*, 317-333.

Horáčková, L., & Zelinka, I. (2025). Advanced swarm intelligence algorithms in quantum circuit design. *International Journal of Parallel, Emergent and Distributed Systems*, *40*(1), 1-20.

IEEE Rebooting Computing Initiative. (2019).

Imani, V., Sevilla-Salcedo, C., Moradi, E., Fortino, V., Tohka, J., Initiative, A. D. N., et al. (2024). Multi-objective genetic algorithm for multi-view feature selection. *Applied Soft Computing*, *167*, 112332.

Iwai, H. (2009). Roadmap for 22 nm and beyond. *Microelectronic Engineering*, *86*(7-9), 1520-1528. https://doi.org/10.1016/j.mee.2009.03.129

Jaffali, F. (2020). Cours d'introduction à l'informatique quantique [Université de Tunis]. https://moodle.isg.rnu.tn

Katoch, S., Chauhan, S. S., & Kumar, V. (2021). A review on genetic algorithm : past, present, and future. *Multimedia tools and applications*, *80*, 8091-8126.

Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proceedings of ICNN'95 - International Conference on Neural Networks*, *4*, 1942-1948.

Kolodziej, M. (2021). Qubit Toaster : A Fast Quantum Circuit Simulator in C++.

Koomey, J. G. (2011). Growth in data center electricity use 2005 to 2010. *Analytics Press*.

Krylov, L., & Lukac, M. (2018). Quantum Encoded Quantum Evolutionary Algorithm for the Design of Quantum Circuits. *Proceedings of the 2018 International Conference on Computing and Network Communications (CoCoNet)*, 90-96. https://doi.org/10.1109/CoCoNet.2018.8510727

Kumar, S., & Pathak, S. (2019). A comprehensive review on swarm intelligence & particle swarm optimization. *Indian Journal of Scientific Research*, *18*(2), 288-294.

Landauer, R. (1961). Irreversibility and Heat Generation in the Computing Process. *IBM Journal of Research and Development*, *5*, 183-191.

Li, Y.-Z., Liu, W., Xu, G.-S., Li, M.-D., Chen, K., & He, S.-L. (2025). Quantum circuit mapping based on discrete particle swarm optimization and deep reinforcement learning. *Swarm and Evolutionary Computation*, *95*, 101923. https://doi.org/10.1016/j.swevo.2025.101923

Linke, N. M., Maslov, D., Roetteler, M., Debnath, S., Figgatt, C., Landsman, K. A., Wright, K., & Monroe, C. (2017). Experimental comparison of two quantum computing architectures. *Proceedings of the National Academy of Sciences*, *114*(13), 3305-3310. https://doi.org/10.1073/pnas.1618020114

Lukac, M., & Perkowski, M. (2002). Evolving quantum circuits using genetic algorithm. *Proceedings 2002 NASA/DoD Conference on Evolvable Hardware*, 177-185.

Maslov, D. (2016). Advantages of using relative-phase Toffoli gates with an application to multiple control Toffoli optimization. *Physical Review A*, *93*(2), 022311. https://doi.org/10.1103/PhysRevA.93.022311

Miranda, F. T., Balbi, P. P., & Costa, P. C. (2023). Synthesis of Quantum Circuits with an Island Genetic Algorithm. *arXiv preprint arXiv :2106.03115*. https://arxiv.org/abs/2106.03115

Mitchell, M. (1998). *An Introduction to Genetic Algorithms*. MIT press.

Moore, G. E. (1975). Progress in Digital Integrated Electronics. *Proceedings of the IEEE International Electron Devices Meeting (IEDM)*, 11-13.

Murali, P., Baker, J. M., Javadi-Abhari, A., Chong, F. T., & Martonosi, M. (2019). Noise-adaptive compiler mappings for noisy intermediate-scale quantum computers. *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*, 1015-1029. https://doi.org/10.1145/3297858.3304075

Nakamura, Y., Pashkin, Y. A., & Tsai, J. S. (1999). Coherent control of macroscopic quantum states in a single-Cooper-pair box. *Nature*, *398*, 786-788. https://doi.org/10.1038/19718

National Academies of Sciences, Engineering, and Medicine. (2019). *Quantum Computing : Progress and Prospects*. National Academies Press. https://doi.org/10.17226/25196

Nielsen, M. A. (2003). Implementation of a Quantum Controlled-NOT Gate with Trapped Ions. *Nature*, *422*, 412-415.

Nielsen, M. A., & Chuang, I. L. (2000). *Quantum Computation and Quantum Information*. Cambridge University Press.

Nielsen, M. A., & Chuang, I. L. (2010). *Quantum Computation and Quantum Information*. Cambridge University Press.

Pednault, E., Gunnels, J., Nannicini, G., et al. (2017). Breaking the 49-qubit barrier in the simulation of quantum circuits. *arXiv preprint*. https://arxiv.org/abs/1710.05867

Peruzzo, A., McClean, J., Shadbolt, P., Yung, M.-H., Zhou, X.-Q., Love, P. J., Aspuru-Guzik, A., & O'Brien, J. L. (2014). A variational eigenvalue solver on a photonic quantum processor. *Nature Communications*, *5*, 4213. https://doi.org/10.1038/ncomms5213

Pop, E. (2010). Energy dissipation and transport in nanoscale devices. *Nano Research*, *3*, 147-169.

Potoček, V., Reynolds, A. P., Fedrizzi, A., & Corne, D. W. (2018). Multi-objective evolutionary algorithms for quantum circuit discovery. *arXiv preprint arXiv :1812.04458*. https://arxiv.org/abs/1812.04458

Preskill, J. (1998). Reliable quantum computers. *Proceedings of the Royal Society of London. Series A : Mathematical, Physical and Engineering Sciences*, *454*(1969), 385-410.

Preskill, J. (2018). Quantum computing in the NISQ era and beyond. *Quantum*, *2*, 79. https://doi.org/10.22331/q-2018-08-06-79

Rahman, M. S. I., Zhang, S., Haque, M. R., & Shatabda, S. (2025). Quantum Circuit Synthesis Using Fuzzy-Logic-Assisted Genetic Algorithms. *Algorithms*, *18*(4), 178. https://doi.org/10.3390/a18040178

Research, I. (2016). *IBM Quantum Experience : premier processeur supraconducteur 5 qubits*. https://research.ibm.com/ibm-q

Research, I. (2021). *IBM Eagle : processeur supraconducteur 127 qubits*. https://www.ibm.com/quantum/blog/127-qubit-quantum-processor-eagle

Richard & Feynman. (1982). Simulating physics with computers. *International Journal of Theoretical Physics*, *21*(6-7), 467-488. https://doi.org/10.1007/BF02650179

Rindell, T., Yenilen, B., Halonen, N., Pönni, A., Tittonen, I., & Raasakka, M. (2023). Exploring the optimality of approximate state preparation quantum circuits with a genetic algorithm. *arXiv preprint*. https://arxiv.org/abs/2210.06411

Sasamal, T., et al. (2005). Reversible Circuit Synthesis Using Evolutionary Algorithms. *Proceedings of International Conference on VLSI Design*, xxx-xxx.

Schrödinger, E. (1935). Die gegenwärtige Situation in der Quantenmechanik [Traduction anglaise : *The present situation in quantum mechanics*]. *Naturwissenschaften*, *23*, 807-812, 823-828, 844-849.

Shami, T. M., El-Saleh, A. A., Alswaitti, M., Al-Tashi, Q., Summakieh, M. A., & Mirjalili, S. (2022). Particle Swarm Optimization : A Comprehensive Survey. *IEEE Access*, *10*, 10031-10062. https://doi.org/10.1109/ACCESS.2022.3142859

Shende, V. V., Bullock, S. S., & Markov, I. L. (2006). Synthesis of Quantum Logic Circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, *25*(6), 1000-1010. https://doi.org/10.1109/TCAD.2005.855930

Shor, P. W. (1994). Algorithms for quantum computation : discrete logarithms and factoring. *Proceedings 35th Annual Symposium on Foundations of Computer Science*, 124-134.

Shor, P. W. (1995). Scheme for reducing decoherence in quantum computer memory. *Physical Review A*, *52*(4), R2493-R2496.

Smith, R. S., Curtis, M. J., & Zeng, W. J. (2016). A Practical Quantum Instruction Set Architecture. *Proceedings of the 2016 ACM Conference on Programming Language Design and Implementation (PLDI)*.

Steane, A. M. (1996). Error Correcting Codes in Quantum Theory. *Physical Review Letters*, *77*, 793-797.

Steiger, D. S., Häner, T., & Troyer, M. (2018). ProjectQ : An Open Source Software Framework for Quantum Computing. *Quantum*, *2*, 49. https://doi.org/10.22331/q-2018-01-31-49

Sünkel, L., Martyniuk, D., Mattern, D., Jung, J., & Paschke, A. (2023). Ga4qco : genetic algorithm for quantum circuit optimization. *arXiv preprint arXiv :2302.01303*.

Systems, D.-W. (2019). *Quantum Annealing for Combinatorial Optimization*. https://www.dwavesys.com

Syswerda, G. (1989). Uniform crossover in genetic algorithms. *Proceedings of the 3rd International Conference on Genetic Algorithms*, 2-9.

Talbi, E.-G. (2009). *Metaheuristics : from design to implementation*. John Wiley & Sons.

Tandeitnik, D., & Guerreiro, T. (2022). Evolving Quantum Circuits. *arXiv preprint arXiv :2210.05058*. https://arxiv.org/abs/2210.05058

Tanenbaum, A. S., & Austin, T. (2013). *Structured Computer Organization* (6e éd.). Pearson.

Team, Q. D. (2023). Qiskit : An Open-source Framework for Quantum Computing.

Terhal, B. M. (2015). Quantum error correction for quantum memories. *Reviews of Modern Physics*, *87*, 307-346.

Theis, T. N., & Wong, H.-S. P. (2017). The end of Moore's Law : A new beginning for information technology. *Computing in Science & Engineering*, *19*(2), 41-50. https://doi.org/10.1109/MCSE.2017.29

Toffoli, T. (1980). *Reversible Computing* (rapp. tech. N° MIT/LCS/TM-151). MIT Laboratory for Computer Science.

Umeano, C. (2021). *Optimisation of Quantum Circuits Using Evolutionary Algorithms* [MSc Dissertation]. Imperial College London. https://www.imperial.ac.uk/media/imperial-college/research-centres-and-groups/theoretical-physics/msc/dissertations/2021/Chukwudubem-Umeano-Dissertation.pdf

van Hasselt, H., Guez, A., & Silver, D. (2016). Deep Reinforcement Learning with Double Q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, *30*, 2094-2100.

Vert, T. (2021). Introduction au calcul quantique. https://thomasvert.fr/quantum

von Neumann, J. (1945). *First Draft of a Report on the EDVAC* (rapp. tech.) (Rapport fondateur de l'architecture de Von Neumann). University of Pennsylvania.

Wu, X.-C., Davis, M. G., Chong, F. T., & Iancu, C. (2020). QGo : Scalable quantum circuit optimization using automated synthesis. *arXiv preprint arXiv :2012.09835*.

Yabuki, T., & Iba, H. (2000). Genetic algorithms for quantum circuit design-evolving a simpler teleportation circuit. *Late Breaking Papers at the 2000 Genetic and Evolutionary Computation Conference*, 421-425.

Yepez, J. (2013). Lecture notes : Qubit representations and rotations [Department of Physics and Astronomy, University of Hawai'i at Manoa, Lecture 1, January 9, 2013].

Young, T. (1804). The Bakerian Lecture : Experiments and Calculations relative to Physical Optics. *Philosophical Transactions of the Royal Society of London*, *94*, 1-16.

Zelinka, I., Kojecký, L., Lampart, M., Nowaková, J., & Plucar, J. (2023). iSOMA swarm intelligence algorithm in synthesis of quantum computing circuits. *Applied Soft Computing*, *142*, 110350. https://doi.org/10.1016/j.asoc.2023.110350

Zhirnov, V. V., Cavin, R. K., Hutchby, J. A., & Bourianoff, G. I. (2003). Limits to binary logic switch scaling—A Gedanken model. *Proceedings of the IEEE*, *91*(11), 1934-1939.